그래프신경망 중간고사

교수님이 말한 슬라이드 (필기용)

10/15 수업에서 11문제를 말할 때 짚은 쪽, 그 뒤에 '추가' 쪽 (혹시 몰라 더 보면 좋은 것). 한 쪽에 두 장, 아래는 손필기 칸.

---

01 메시지 패싱: 인접행렬 정규화, GCN 한 층, 순열 등변성 2쪽
2주차 p.20-25, 2주차 p.35-38, 2주차 p.46-48
추가 (9쪽부터): 2주차 p.4, 2주차 p.27-31, 2주차 p.34

02 WL relabeling 과 GIN 13쪽
3주차 p.28-45, 3주차 p.46-58
추가 (29쪽부터): 3주차 p.4-6, 3주차 p.16-22, 3주차 p.59-60, 3주차 p.65-66

03 ESAN 과 1-WL 넘는 방법 비교 36쪽
3주차 p.63-64, 3주차 p.69-72, 3주차 p.73-75, 3주차 p.79-82
추가 (43쪽부터): 3주차 p.67-68, 3주차 p.76-78

04 계산 그래프와 계산 부담, full batch 46쪽
4주차 p.5-6, 4주차 p.9-12
추가 (49쪽부터): 4주차 p.13-14, 4주차 p.21

05 Cluster-GCN 과 그 문제점 51쪽
4주차 p.45-58
추가 (58쪽부터): 4주차 p.44, 4주차 p.59-63

06 과평활, 과압축, under-reaching 과 저역통과 필터 61쪽
5주차 p.14-31, 6주차 p.6
추가 (71쪽부터): 5주차 p.9-13, 5주차 p.32-36, 6주차 p.4-5

07 GCNII: initial residual, identity mapping 77쪽
5주차 p.47-51
추가 (80쪽부터): 5주차 p.37-39, 5주차 p.41, 5주차 p.53-54

08 동질성 지표 (homophily metrics) 83쪽
6주차 p.9-17
추가 (88쪽부터): 6주차 p.3-8, 6주차 p.18-22, 6주차 p.28-29

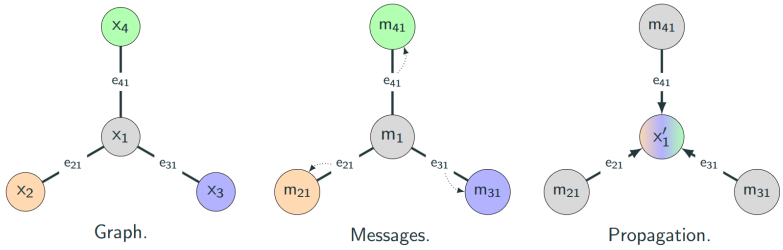
09 H2GCN 95쪽
6주차 p.51-56
추가 (98쪽부터): 6주차 p.30-31, 6주차 p.64

10 GAT 어텐션과 GAT 대 GATv2 100쪽
7주차 p.14-25, 7주차 p.27-33
추가 (110쪽부터): 7주차 p.9-12, 7주차 p.38-41

11 HAN: 동작과 어텐션을 두 번 하는 이유 114쪽
7주차 p.70-84
추가 (122쪽부터): 7주차 p.69, 6주차 p.18-19

The Basics: Graph Neural Networks 20

Core Idea: Propagate node features by systematically exchanging information between adjacent nodes in the graph.

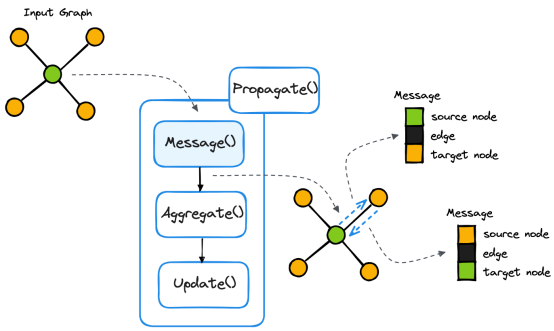


References:
- William L. Hamilton, Rex Ying, Jure Leskovec: Representation Learning on Graphs: Methods and Applications. IEEE Data Eng. Bull. 40(3): 52-74 (2017)
- Daniele Grattarola, University of Lugano: Graph Neural Networks. Graph Deep Learning 2021

The Basics: Graph Neural Networks 21

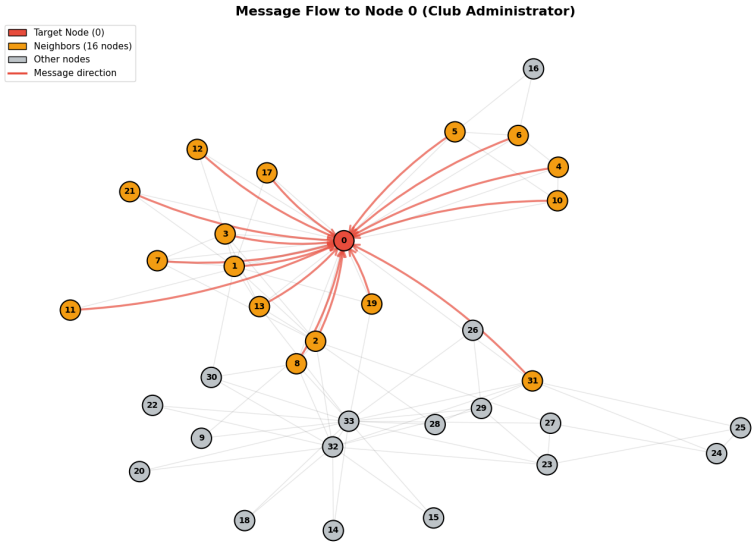
Core Idea: Propagate node features by systematically exchanging information between adjacent nodes in the graph.

1. **Message Generation**:
   - Prepare a specific message (feature representation) at each node that will be broadcast to its neighbors.



References:
- William L. Hamilton, Rex Ying, Jure Leskovec: Representation Learning on Graphs: Methods and Applications. IEEE Data Eng. Bull. 40(3): 52-74 (2017)
- Futia, G. (2024, September 2). The Expressive Power of GNNs — The Message Passing Neural Network. Medium; TDS Archive. https://medium.com/data-science/the-expressive-power-of-gnns-the-message-passing-neural-network-e4b5f8f6c01f

The Basics: Graph Neural Networks 22



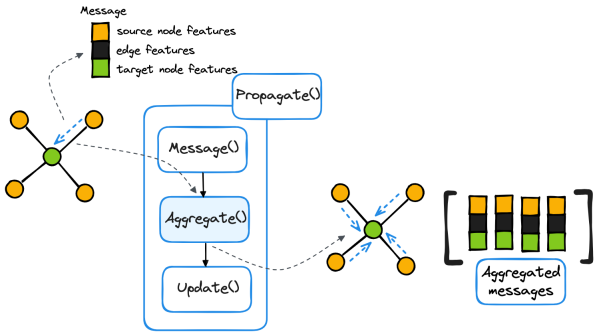
Open in Colab

Example: Neighborhood aggregation for Node 0 in the Zachary karate club dataset.

The Basics: Graph Neural Networks 23

Core Idea: Propagate node features by systematically exchanging information between adjacent nodes in the graph.

2. **Aggregate and Update**
   - **Aggregate**: Collect and combine all incoming messages from a node's local neighborhood.

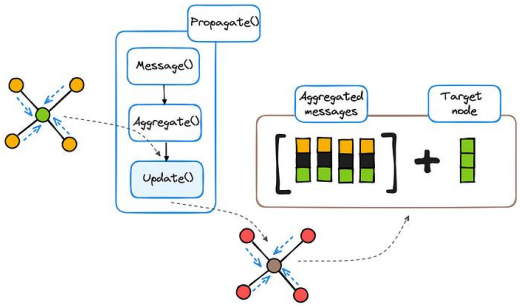


References:
- William L. Hamilton, Rex Ying, Jure Leskovec: Representation Learning on Graphs: Methods and Applications. IEEE Data Eng. Bull. 40(3): 52-74 (2017)
- Futia, G. (2024, September 2). The Expressive Power of GNNs — The Message Passing Neural Network. Medium; TDS Archive. https://medium.com/data-science/the-expressive-power-of-gnns-the-message-passing-neural-network-e4b5f8f6c01f

The Basics: Graph Neural Networks 24

Core Idea: Propagate node features by systematically exchanging information between adjacent nodes in the graph.

2. **Aggregate and Update**
   - **Aggregate**: Collect and combine all incoming messages from a node's local neighborhood.
   - **Update**: Compute the new feature representation for the node using its previous state and the aggregated neighborhood message.



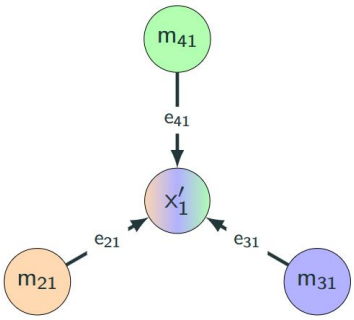
References:
- William L. Hamilton, Rex Ying, Jure Leskovec: Representation Learning on Graphs: Methods and Applications. IEEE Data Eng. Bull. 40(3): 52-74 (2017)
- Futia, G. (2024, September 2). The Expressive Power of GNNs — The Message Passing Neural Network. Medium; TDS Archive. https://medium.com/data-science/the-expressive-power-of-gnns-the-message-passing-neural-network-e4b5f8f6c01f

The Basics: Graph Neural Networks 25

$$\mathbf{h}_v^{(l+1)} = \text{UPDATE}^{(l)}\left(\mathbf{h}_v^{(l)}, \text{AGGREGATE}^{(l)}\left(\left\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\right\}\right)\right)$$

where:

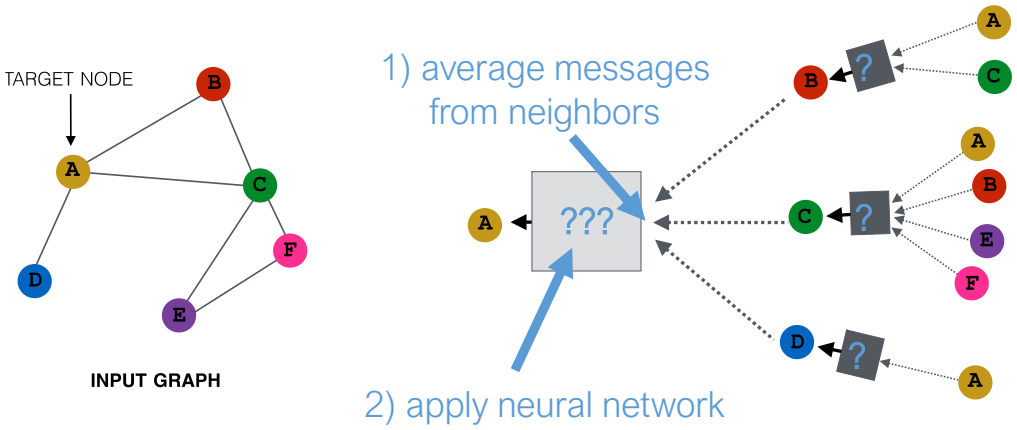
- $\mathbf{h}_u^{(l)}$: message, representing the features of neighbor u at layer l;
- $\mathcal{N}(v)$ represents the set of neighbors of node v.
- $\text{AGGREGATE}^{(l)}$: aggregation function (sum, average, max, or something else...);
- $\text{UPDATE}^{(l)}$: update function, final transformation to obtain new attributes after aggregating messages.



References:
- Daniele Grattarola, University of Lugano: Graph Neural Networks. Graph Deep Learning 2021

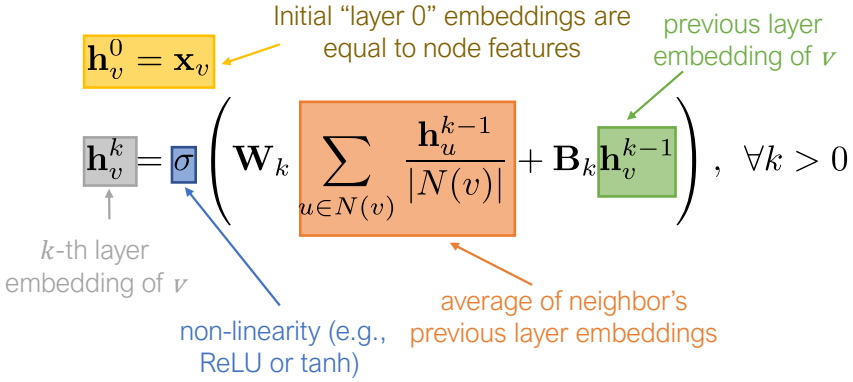
Neighborhood Aggregation 35

➢ Basic approach: Average neighbor information and apply a neural network.



Neighborhood Aggregation: Mathematics 36

➢ Basic approach: Average neighbor messages and apply a neural network.



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighborhood Aggregation: Permutation Invariance 37

- **The Constraint**: Because graph neighborhoods are unordered and variable in size, we cannot simply concatenate feature vectors.
- **The Solution**: We must use a permutation invariant operation.
- **Definition**: An operation that produces the **exact same output** regardless of the order in which the inputs are processed.

  A node function $f: \mathbb{R}^{|V|\times|V|} \times \mathbb{R}^{|V|\times m} \rightarrow \mathbb{R}^{|V|\times d}$ is said to be **permutation-equivariant** if, for any permutation matrix P, the following condition holds:

  $$Pf(A,X) = f(PAP^T, PX)$$

  Where:

  - $A \in \mathbb{R}^{|V|\times|V|}$ represents the graph's adjacency matrix.
  - $X \in \mathbb{R}^{|V|\times m}$ represents the node feature matrix (with m-dimensional input features).
  - f maps each node in the vertex set V to a new d-dimensional embedding.

References:
- Daniele Grattarola, University of Lugano: Graph Neural Networks. Graph Deep Learning 2021
- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighborhood Aggregation: Permutation Invariance 38

- The GNN Aggregation Formula:

  $$m_{N(v)}^{(l)} = \text{AGGREGATE}^{(l)}\left(\left\{h_u^{(l)} : u \in N(v)\right\}\right)$$

- To achieve permutation invariance, GNNs typically use one of the following operations to process neighbor vectors:
  - **Sum**: $\sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(l)}$ - Add all neighbor vectors together.
  - **Mean**: $\frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(l)}$ - Take the element-wise average of the vectors.
  - **Max**: $\max_{u\in\mathcal{N}(v)}\left(\mathbf{h}_u^{(l)}\right)$ - Take the element-wise maximum across all vectors.
- **Result**: A summary representing the *"collective wisdom"* of the local environment.

References: Thor, W.-M. (2026). The Neighborhood Aggregation Idea. Apxml.Com; ApX Machine Learning.
https://apxml.com/courses/introduction-to-graph-neural-networks/chapter-2-the-message-passing-mechanism/neighborhood-aggregation-idea

Graph Convolutional Networks 46

➢ GCN are slight variation on the neighborhood aggregation idea.

Basic Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1}\right), \forall k > 0$$

GCN Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

same matrix for self and neighbor embeddings

per-neighbor normalization

References: Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017

Graph Convolutional Networks 47

➢ Empirically, this configuration to give the best results

- More parameter sharing.
- Down-weights high degree neighbors.

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

Use the same transformation matrix for self and neighbor embeddings

Instead of a simple average, normalization varies across neighbors

References: Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017

Graph Convolutional Networks: Batch Implementation 48

- Can be efficiently implemented using sparse batch operations:

$$\mathbf{H}^{(k+1)} = \sigma\left(\mathbf{D}^{-\frac{1}{2}}\tilde{\mathbf{A}}\mathbf{D}^{-\frac{1}{2}}\mathbf{H}^{(k)}\mathbf{W}_k\right)$$

where:

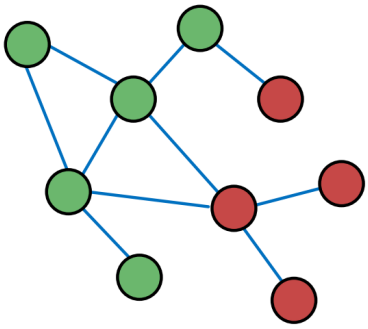
- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$
- $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{i,j}$
- $\mathbf{A}$: Adjacency matrix
- $\mathbf{I}$: Identity matrix

- And $O(|E|)$ time complexity overall.

References: Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017

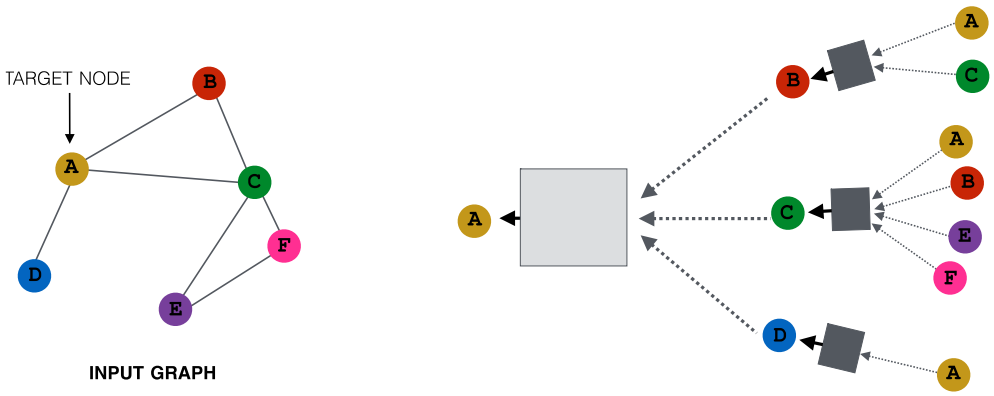
Message Passing Intuition 4

- **Intuition**: Correlations (dependency) exist in network
  - Similar nodes are connected.
  - Homophily: individual character → social connections.
  - Influence: social connections → individual character.
- **Key concept**: collective classification, assign labels to all nodes in a network together.
- We will look at 2 **techniques**:
  - Relational classification.
  - Iterative classification.



Neighborhood Aggregation 27

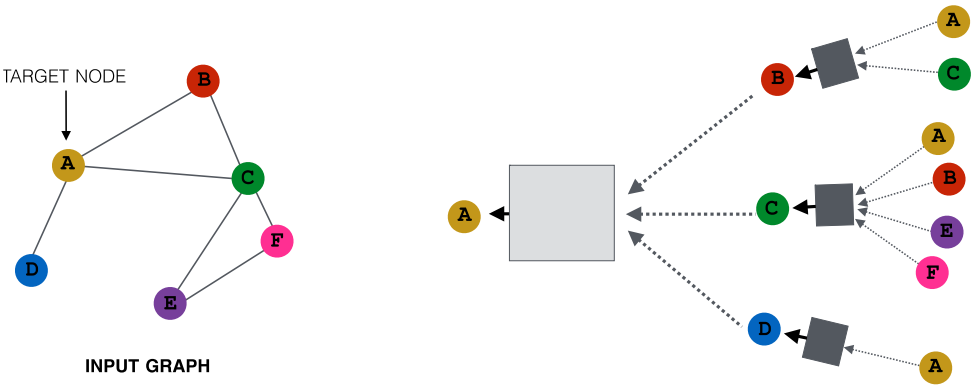
- **Key idea**: Generate node embeddings based on local neighborhoods.



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighborhood Aggregation 28

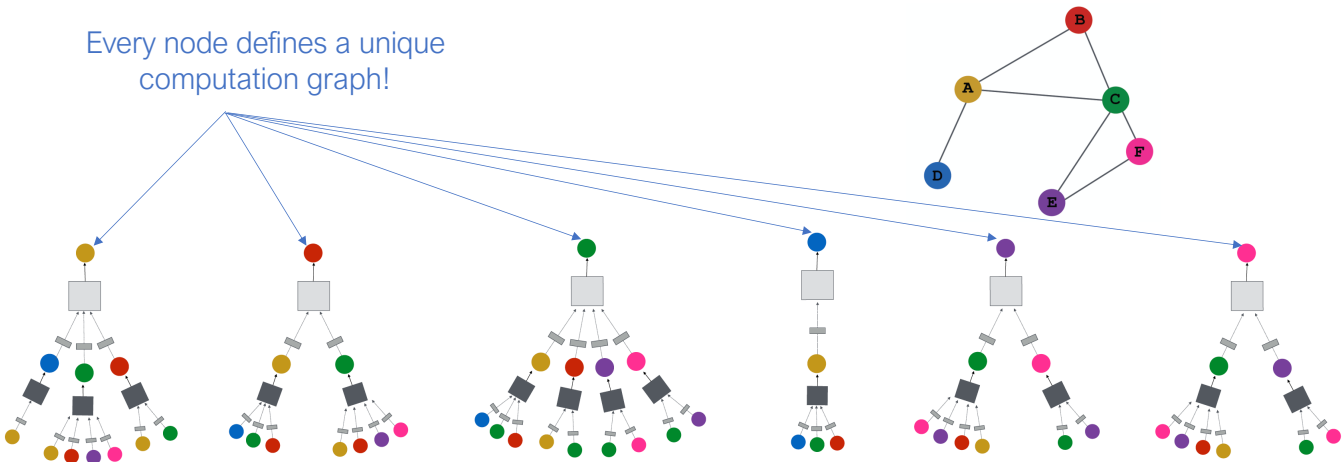
➢ **Intuition**: Nodes aggregate information from their neighbors using neural networks



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

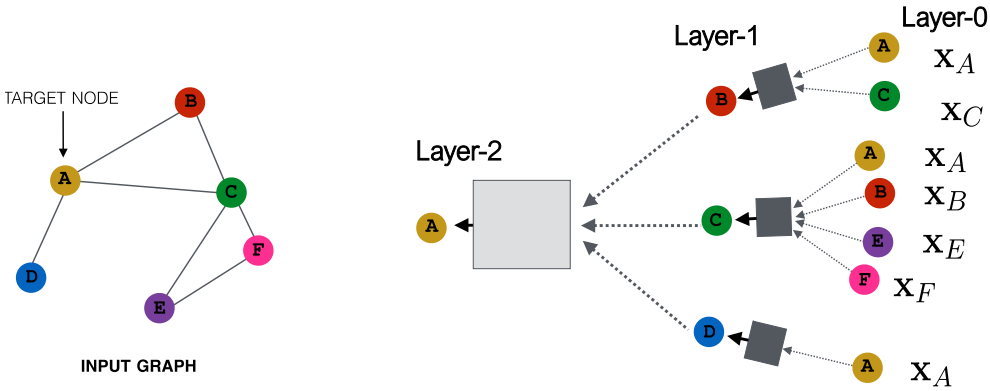
Neighborhood Aggregation 29

➢ **Intuition**: Network neighborhood defines a computation graph.



Neighborhood Aggregation 30

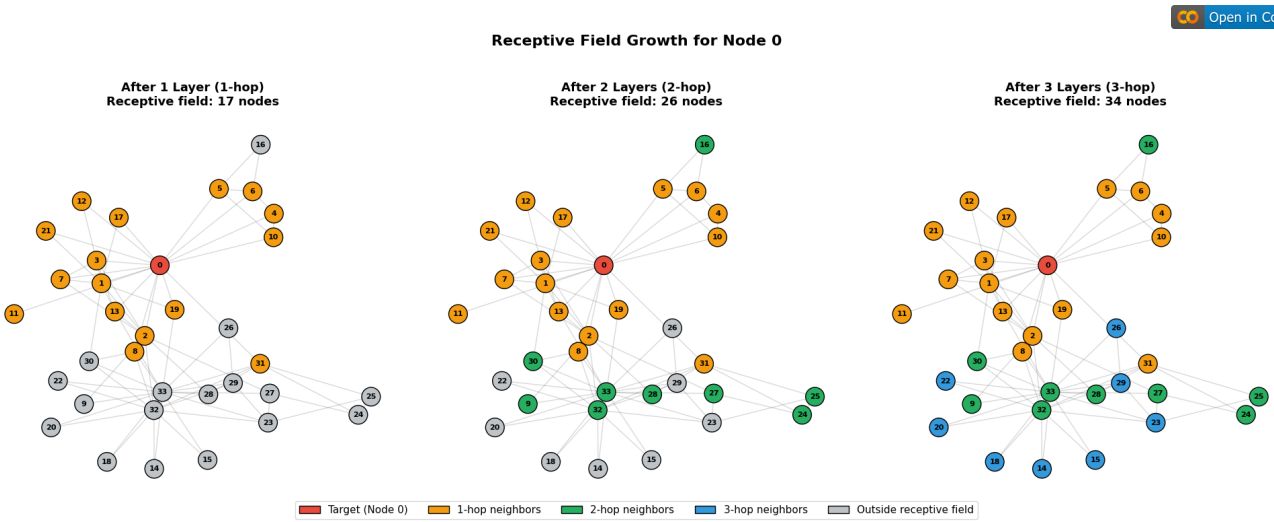
- Nodes have embeddings at each layer.
- Model can be arbitrary depth.
- "layer-0" embedding of node u is its input feature, i.e. x_u



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighborhood Aggregation 31

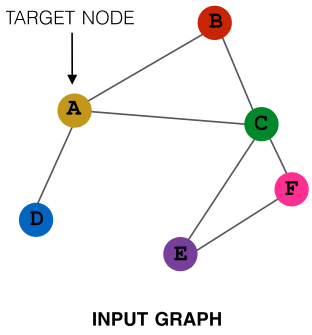
Open in Colab



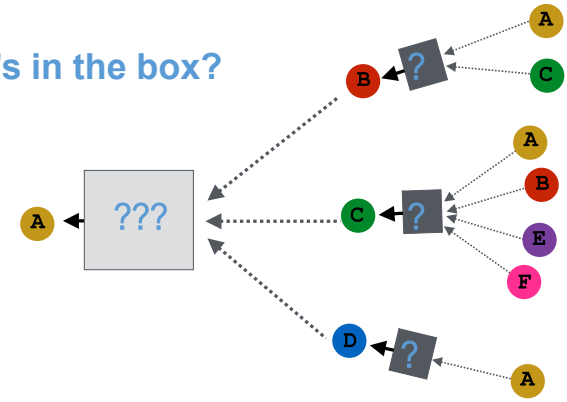
Example: Visualization of the k-hop neighborhood growth for Node 0 in the Zachary karate club dataset.

Neighborhood Aggregation 34

➢ Key distinctions are in how different approaches aggregate information across the layers.



INPUT GRAPH



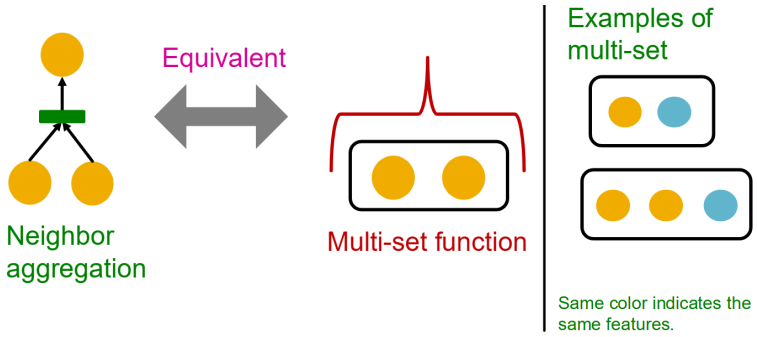
Expressive Power of GNNs 28

- **Key observation**: The expressive power of GNN can be characterized by that of the neighbor aggregation functions they use.
  - A more expressive aggregation function leads to a more expressive GNN.
  - An **injective aggregation function** leads to the most expressive GNN.
- **Next**:
  - Theoretically analyze the expressive power of aggregation functions.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Expressive Power of GNNs 29

- **Observation:** Neighbor aggregation can be abstracted as a function over a multi-set (a set with repeating elements).



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation 30

- **GCN (mean-pool)** [Kipf & Welling, ICLR 2017]
  - Uses element-wise mean pooling over neighboring node features

$$\text{Mean}\left(\{x_u\}_{u\in N(v)}\right)$$

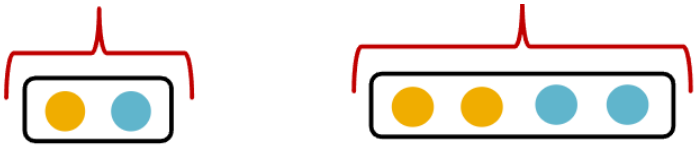
- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
  - Uses element-wise max pooling over neighboring node features

$$\text{Max}\left(\{x_u\}_{u\in N(v)}\right)$$

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 31

- **GCN (mean-pool)** [Kipf & Welling, ICLR 2017]
  - Take **element-wise mean**, followed by linear function and ReLU activation, i.e., $\max(0, x)$.
  - **Theorem** [Xu et al. ICLR 2019]
    - GCN's aggregation function cannot **distinguish different multi-sets with the same color proportion.**
  - **Failure case**



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 32

- For simplicity, we assume node features (colors) are represented by one-hot encoding
  - Example: If there are two distinct colors:

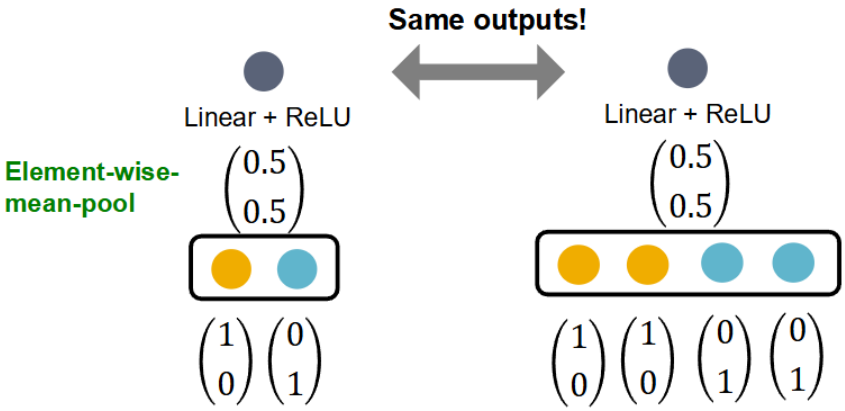
$$\text{●} = \begin{pmatrix}1\\0\end{pmatrix} \qquad \text{●} = \begin{pmatrix}0\\1\end{pmatrix}$$

  - This assumption is sufficient to illustrate how GCN fails.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 33

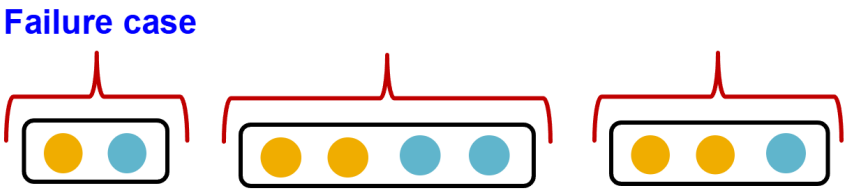
- **GCN (mean-pool)** [Kipf & Welling, ICLR 2017]
  - **Failure case illustration**



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 34

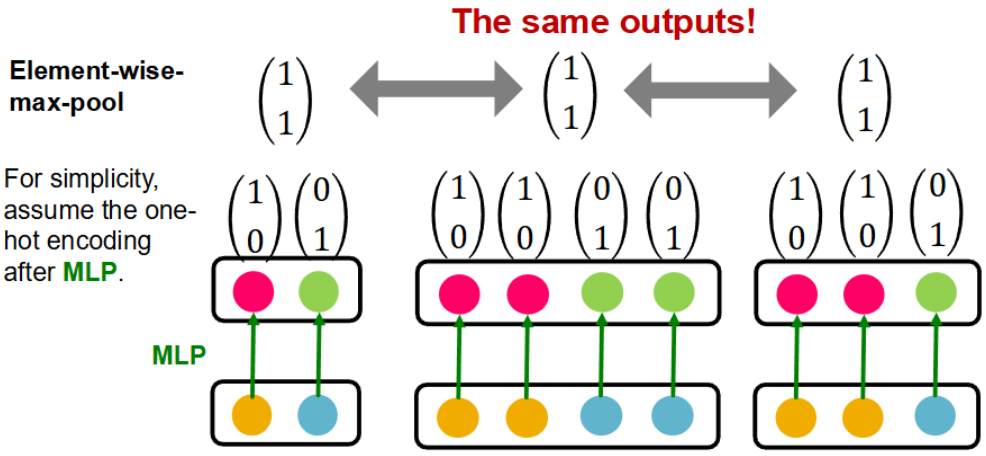
- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
  - Apply an MLP, then take element-wise max.
  - Theorem [Xu et al. ICLR 2019]
    - GraphSAGE's aggregation **function cannot distinguish different multisets with the same set of distinct colors.**



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 35

- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
  - **Failure case illustration**



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Neighbor Aggregation: Case Study 36

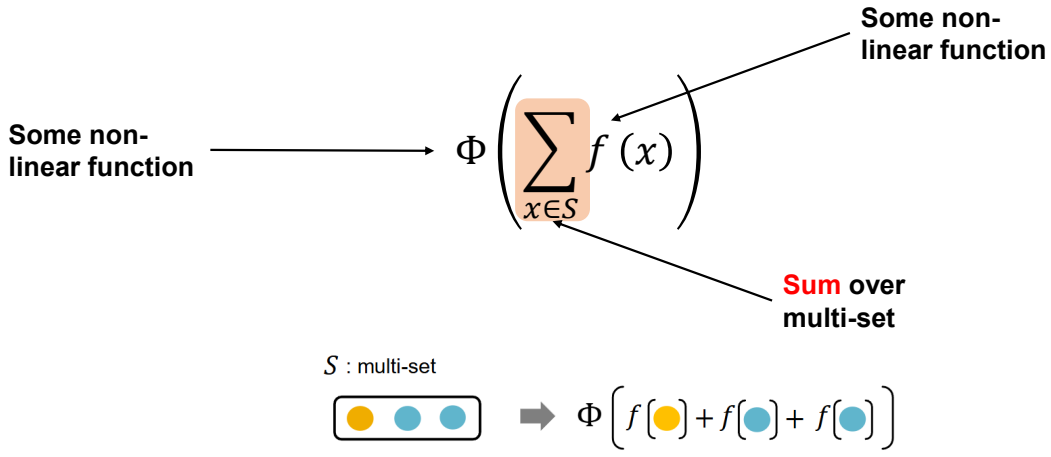
- We analyzed the **expressive power of GNNs.**
- **Main takeaways:**
  - The expressive power of GNNs can be characterized by that of the neighbor aggregation function.
  - Neighbor aggregation is a function over multi-sets(sets with repeating elements)
  - GCN and GraphSAGE's aggregation functions fail to distinguish some basic multisets; hence, they are not injective.
  - Therefore, GCN and GraphSAGE are not maximally powerful GNNs.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Injective Multi-Set Function 37

- **Theorem** [Xu et al. ICLR 2019]

  Any injective multi-set function can be expressed as:



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Designing Most Expressive GNNs 38

- **Our goal**: Design maximally powerful GNNs in the class of message-passing GNNs.
- This can be achieved by designing an **injective neighbor aggregation function over multi-sets.**
- Here, we design a neural network that can model an injective multiset function.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

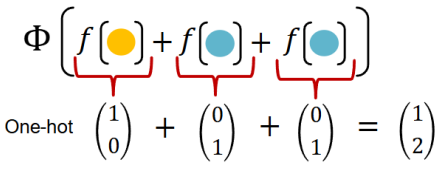
Injective Multi-Set Function 39

- **Proof Intuition:** [Xu et al. ICLR 2019]

  f produces one-hot encodings of colors. Summation of the one-hot encodings retains all the information about the input multi-set:

$$\Phi\left(\sum_{x \in S} f(x)\right)$$

Example:

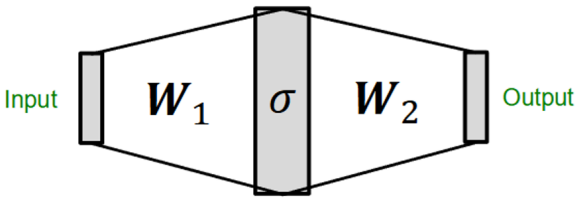


References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Universal Approximation Theorem 40

- How to model Φ and f in $\Phi\left(\sum_{x \in S} f\,(x)\right)$?
- We use a **Multi-Layer Perceptron (MLP)**.
- **Theorem**: Universal Approximation Theorem [Hornik et al., 1989]

  1-hidden-layer MLP with sufficiently-large hidden dimensionality and appropriate non-linearity $\sigma(\cdot)$ (including ReLU and sigmoid) can **approximate any continuous function to an arbitrary accuracy.**



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Injective Multi-Set Function 41

- We have arrived at a neural network that can model any injective multiset function.

$$\mathrm{MLP}_{\Phi}\left(\sum_{x \in S} \mathrm{MLP}_{f}\,(x)\right)$$

- In practice, MLP hidden dimensionality of 100 to 500 is sufficient.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Most Expressive GNN 42

- **Graph Isomorphism Network (GIN)** [Xu et al. ICLR 2019].
  - Apply an MLP, element-wise sum, followed by another MLP.

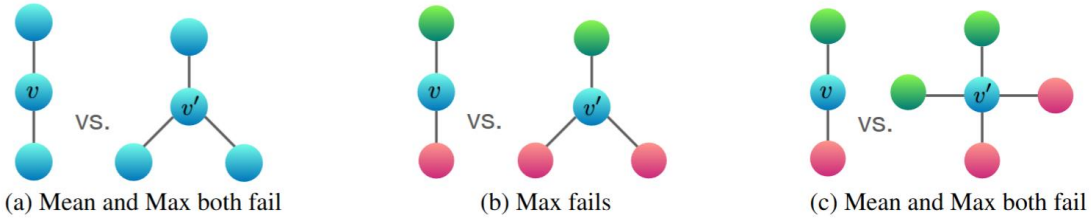
$$\mathrm{MLP}_{\Phi}\left(\sum_{x \in S} \mathrm{MLP}_{f}(x)\right)$$

- **Theorem** [Xu et al. ICLR 2019]
  - GIN's neighbor aggregation function is injective.
- **No failure cases!**
- **GIN is THE most expressive GNN** in the class of message-passing GNNs we have introduced!

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

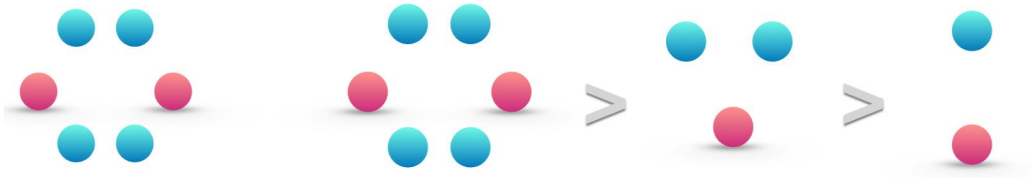
Discussion: The Power of Pooling 43

- **Failure cases for mean and max pooling:**



(a) Mean and Max both fail (b) Max fails (c) Mean and Max both fail

- **Ranking by discriminative power:**



References:
- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu
- Keyulu Xu, Weihua Hu, Jure Leskovec, Stefanie Jegelka: How Powerful are Graph Neural Networks? ICLR 2019

Why sum ≻ mean ≻ max 44

SUM

injective (countable X)

- $\{\{a,b\}\} \neq \{\{a,a,b,b\}\}$
- $\{\{a,b\}\} \neq \{\{a,b,b\}\}$

Retains both which elements occur and how many times.

MEAN

blind to multiplicity

- $\{\{a,b\}\} = \{\{a,a,b,b\}\}$
- $\{\{a,b\}\} \neq \{\{a,b,b\}\}$

Two neighbourhoods with the same proportions collapse - it cannot see degree.

MAX

blind to counts entirely

- $\{\{a,b\}\} = \{\{a,a,b,b\}\}$
- $\{\{a,b\}\} = \{\{a,b,b\}\}$

Only the distinct element types survive; it is a set function.

$$Sep(\text{MAX}-\text{GNN}) \subsetneq Sep(\text{MEAN}-\text{GNN}) \subsetneq Sep(\text{SUM}-\text{GNN}) = Sep(1-\text{WL})$$

References: Keyulu Xu, Weihua Hu, Jure Leskovec, Stefanie Jegelka: How Powerful are Graph Neural Networks? ICLR 2019

Full Model of GIN 45

- **So far:** We have described the neighborhood aggregation part of GIN.
- We now describe the full model of GIN by relating it to the **WL graph kernel** (traditional way of obtaining graph-level features).
  - We will see how GIN is a “neural network” version of the WL graph kernel.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Recap: Graph Isomorphism problem 46

➢ **A classical expressivity test is graph isomorphism.**

- Given a pair of graphs, can we tell if they are isomorphic?



- Open problem (a polynomial time algorithm **does not exist**, has not been proven **NP-complete**).

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Relation to WL Graph Kernel 47

Color refinement algorithm in WL kernel.

➢ **Given**: A graph G with a set of nodes V.

- Assign an initial color $c^{(0)}(v)$ to each node v.
- Iteratively refine node colors by

$$c^{(k+1)}(v) = \text{HASH}\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right),$$

where HASH maps different inputs to different colors.

- After K steps of color refinement, **$c^{(K)}(v)$ summarizes the structure of K-hop neighborhood**

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

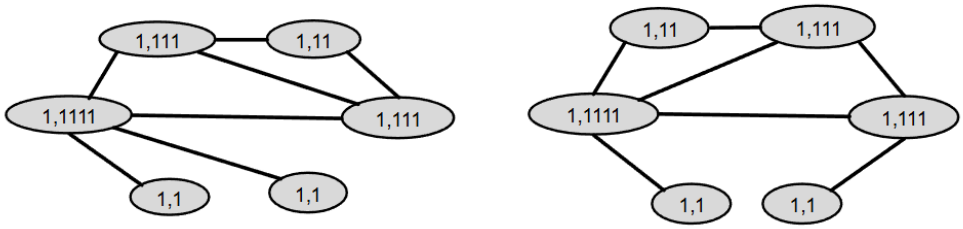
Color Refinement (1) 48

Example of color refinement given <u>two graphs</u>

- Assign initial colors



- Aggregate neighboring colors



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

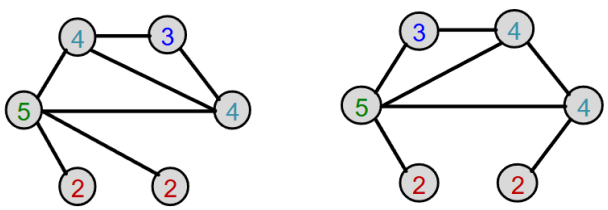
Color Refinement (2) 49

Example of color refinement given <u>two graphs</u>

- Aggregated colors:



- **Injectively** HASH the aggregated colors



HASH table: **Injective!**

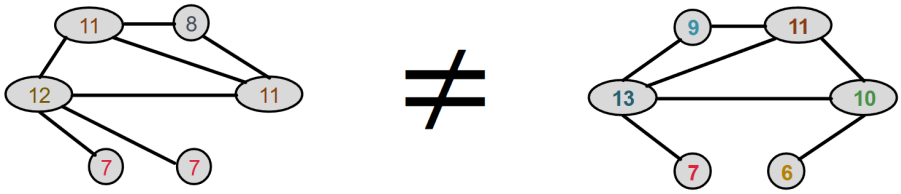
1,1	-->	2
1,11	-->	3
1,111	-->	4
1,1111	-->	5

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Color Refinement (3) 50

Example of color refinement given <u>two graphs</u>

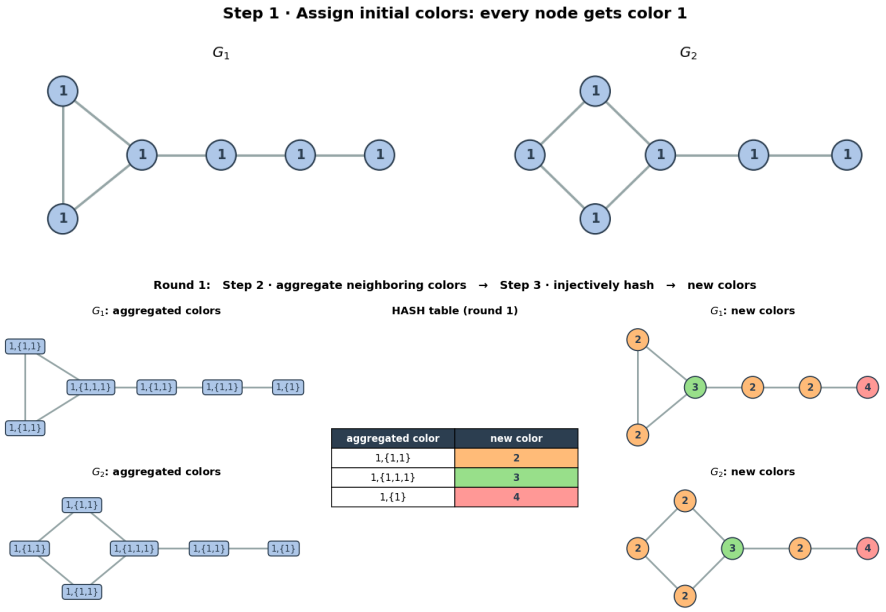
- Process continues until a stable coloring is reached
- Two graphs are considered isomorphic if they have the same set of colors.



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

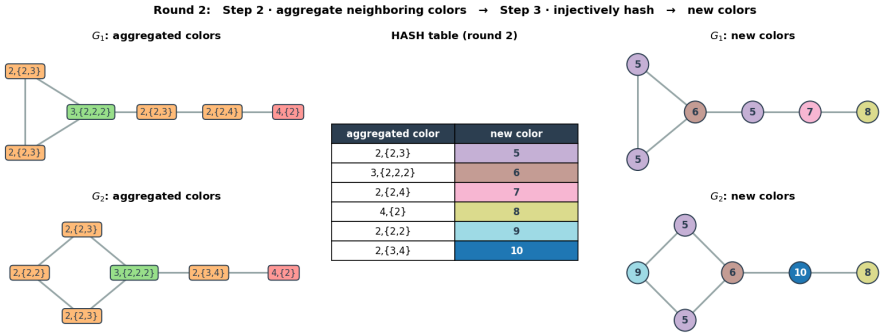
Another Example 51

Open in Colab



Another Example 52

Open in Colab



aggregated color	new color
2,{2,3}	5
3,{2,2,2}	6
2,{2,4}	7
4,{2}	8
2,{2,2}	9
2,{3,4}	10

Round	Graph G1	Graph G2
0	{1: 6}	{1: 6}
1	{2: 4, 3: 1, 4: 1}	{2: 4, 3: 1, 4: 1}
2	{2: 5, 3: 1, 7: 1, 8: 1}	{2: 5, 3: 1, 8: 1, 9: 1, 10: 1}

Result: Non-isomorphic - the histograms differ at **Round 2**.

The Complete GIN Model 53

- GIN uses a neural network to model the injective HASH function

$$c^{(k+1)}(v) = \text{HASH}\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right)$$

- Specifically, we will model the injective function over the tuple:

$$\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right)$$

Root node features — **Neighboring node colors**

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

The Complete GIN Model 54

➢ **Theorem** (Xu et al. ICLR 2019)

Root node features $\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right)$ **Neighboring node colors**

Any injective function over the tuple

$$\text{MLP}_\Phi\left((1+\epsilon)\cdot \text{MLP}_f\left(c^{(k)}(v)\right) + \sum_{u \in N(v)} \text{MLP}_f\left(c^{(k)}(u)\right)\right)$$

where ϵ is a learnable scalar.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

The Complete GIN Model 55

➢ If input feature $c^{(0)}(v)$ is represented as one-hot, direct summation is injective.

Example: $\Phi\left(\begin{pmatrix}1\\0\end{pmatrix} + \begin{pmatrix}0\\1\end{pmatrix} + \begin{pmatrix}0\\1\end{pmatrix}\right) = \begin{pmatrix}1\\2\end{pmatrix}$

➢ We only need to ensure the injectivity.

$$\text{GINConv}\left(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right) = \text{MLP}_\Phi\left((1+\epsilon)\cdot c^{(k)}(v) + \sum_{u \in N(v)} c^{(k)}(u)\right)$$

Root node features

Neighboring node features

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

The Complete GIN Model 56

- GIN's node embedding updates
- **Given**: A graph G with a set of nodes V.
  - Assign an **initial vector** $c^{(0)}(v)$ to each node v.
  - Iteratively update node vectors by

$$c^{(k+1)}(v) = \text{GINConv}\left(\left\{c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)}\right\}\right),$$

Differentiable color HASH function

where **GINConv** maps different inputs to different embeddings.

  - After K steps of GIN iterations, $c^{(K)}(v)$ **summarizes the structure of K-hop neighborhood**.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

GIN and WL Graph Kernel 57

- GIN can be understood as differentiable neural version of the WL graph Kernel:

	Update target	**Update function**
WL Graph Kernel	Node colors (one-hot)	HASH
GIN	Node embeddings (low-dim vectors)	GINConv

- Advantages of GIN over the WL graph kernel are:
  - Node embeddings are **low-dimensional**; hence, they can capture the fine-grained similarity of different nodes.
  - Parameters of the update function can be **learned for the downstream tasks**

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Expressive Power of GIN

58

- Because of the relation between GIN and the WL graph kernel, their expressiveness is exactly the same.
  - If two graphs can be distinguished by GIN, they can also be distinguished by the WL kernel, and vice versa.
- **How powerful is this?**
  - WL kernel has been both theoretically and empirically shown to distinguish most of the real-world graphs [Cai et al. 1992].
  - Hence, GIN is also powerful enough to distinguish most of the real graphs!

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Definitions 4

➢ **Two ways to say "expressive"**

1. Separation (distinguishing) power

A model f separates the pair (G, H) if $f(G) \neq f(H)$. Collect all pairs it can separate:

$$Sep(F) = \{(G, H): G \not\cong H, \exists f \in F, f(G) \neq f(H)\}$$

$$F \leq G \Leftrightarrow Sep(F) \subseteq Sep(G)$$

2. Approximation (universality)

Can the class approximate every continuous invariant target on a compact set?

$$\forall \varepsilon > 0, \forall g \in C_{\text{inv}}(K), \exists f \in F:$$

$$\sup_{G \in K} |f(G) - g(G)| < \varepsilon$$

EQUIVALENCE

Theorem (Chen, Villar, Chen & Bruna, NeurIPS 2019).

For permutation-invariant classes closed under suitable arithmetic, F is universal ⟺ F separates every pair of non-isomorphic graphs.

Therefore, by answering the simpler combinatorial question of which graphs we can distinguish, we obtain the analytic solution naturally. For the remainder of this lecture, "expressive power" will be synonymous with separation power.

References: Chen, Z., Villar, S., Lei, C., & Bruna, J. (2019). On the equivalence between graph isomorphism testing and function approximation with GNNs. Advances in Neural Information Processing Systems, 32. https://proceedings.neurips.cc/paper/2019/hash/71ee911dd06428a96c143a0b135041a4-Abstract.html

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Graph Isomorphism problem 5

➢ **A classical expressivity test is graph isomorphism.**

- Given a pair of graphs, can we tell if they are isomorphic?



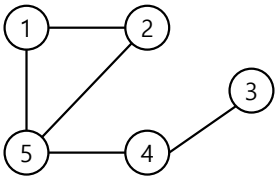
- Open problem (a polynomial time algorithm **does not exist**, has not been proven **NP-complete**).

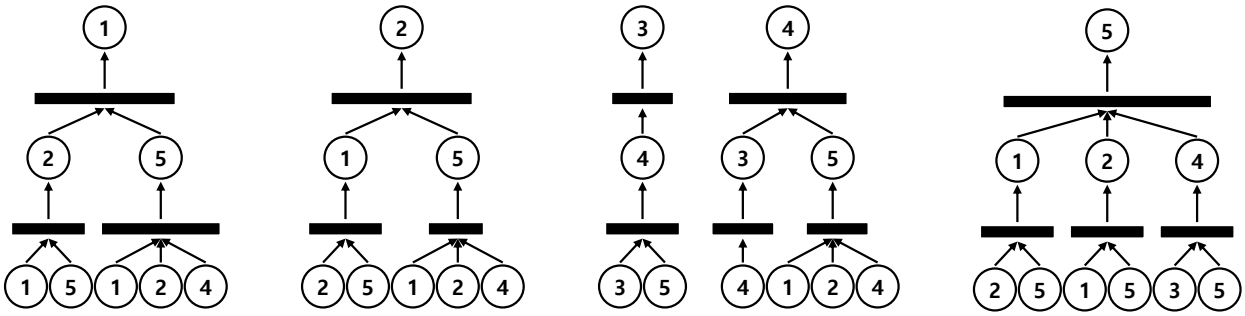
References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

A simpler problem 6

- **Given a pair of nodes with different neighborhood structures. Is there a GNN that can always tell them apart?**

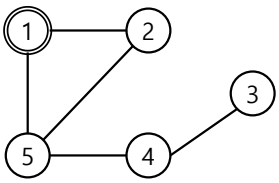


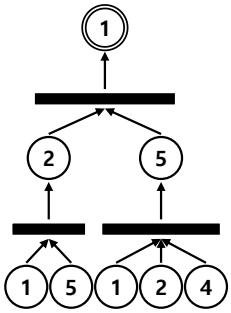


References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph (1) 16

- In each layer, a GNN aggregates neighboring node embeddings.
- A GNN generates node embeddings through a computational graph defined by the neighborhood.
  - **Example:** Node 1's computational graph (2-layer GNN)

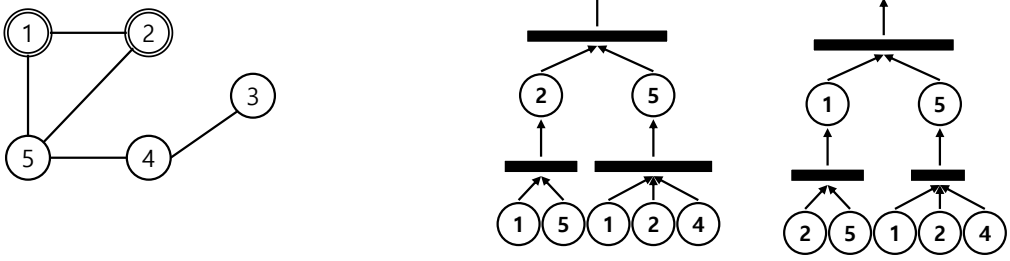




References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph (2) 17

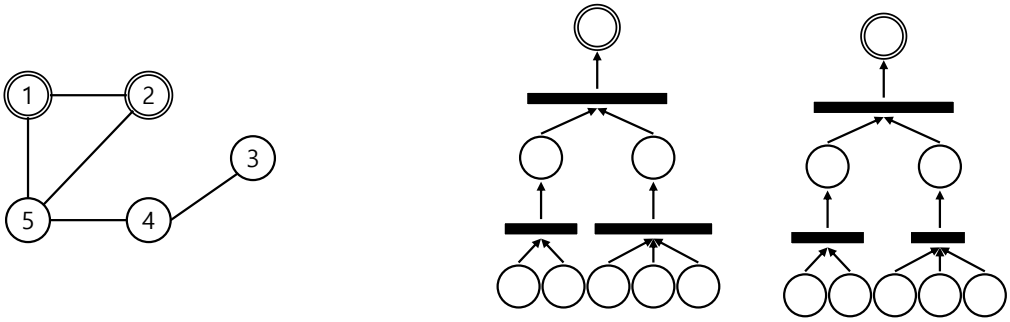
- **Example:** Nodes 1 and 2's computational graphs.



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph (3) 18

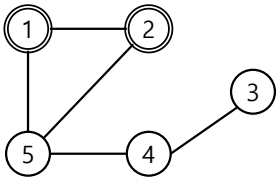
- **Example:** Nodes 1 and 2's computational graphs.
- But GNN only sees node features (not IDs)



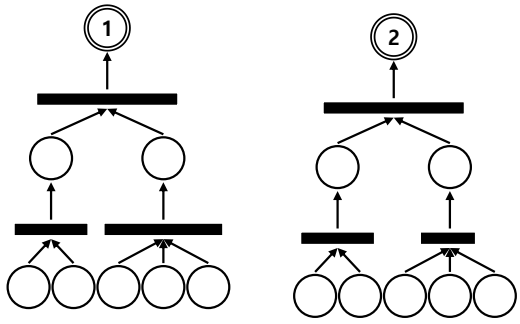
References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph (3) 19

- A GNN will generate the same embedding for nodes 1 and 2 because:
  - Computational graphs are the **same**
  - Node features (colors) are **identical**.



GNN won't be able to distinguish nodes 1 and 2



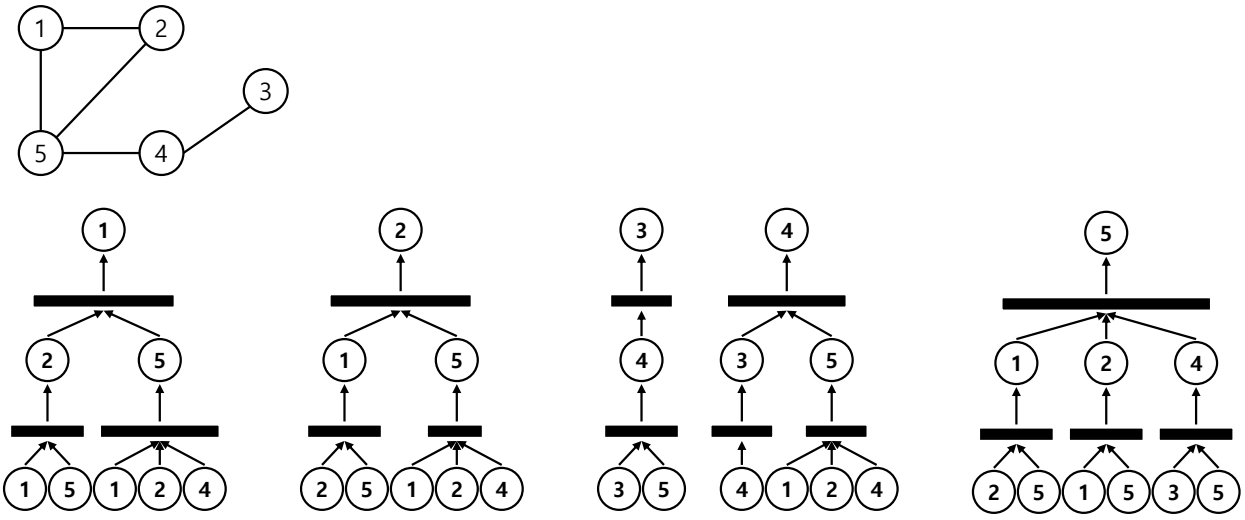
Notes

- GNN does not care about node IDs, it just aggregates feature vectors of different nodes.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph 20

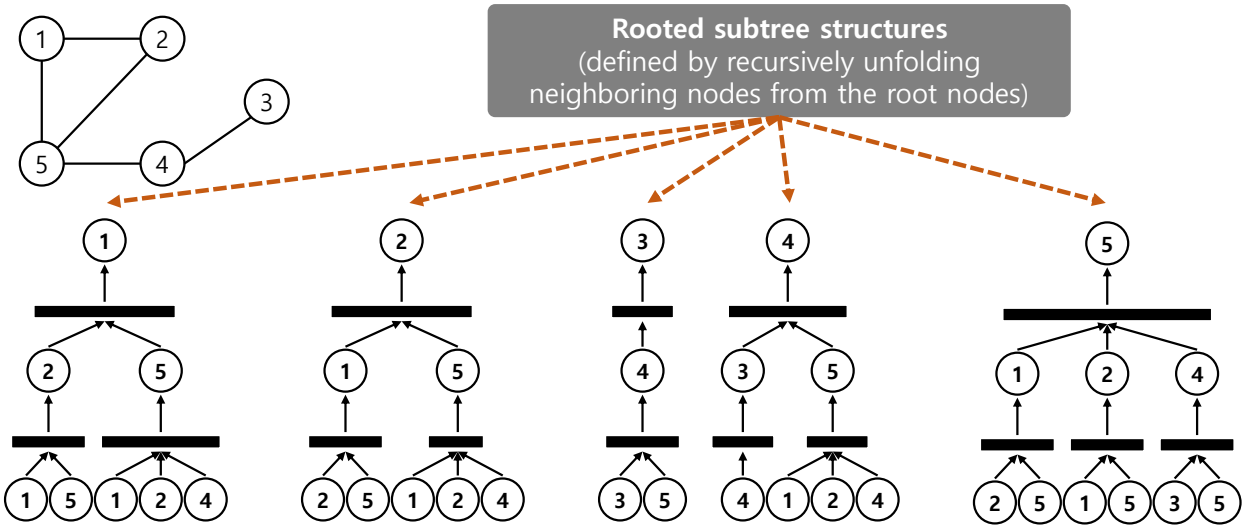
- In general, different local neighborhoods define different computational graphs



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph 21

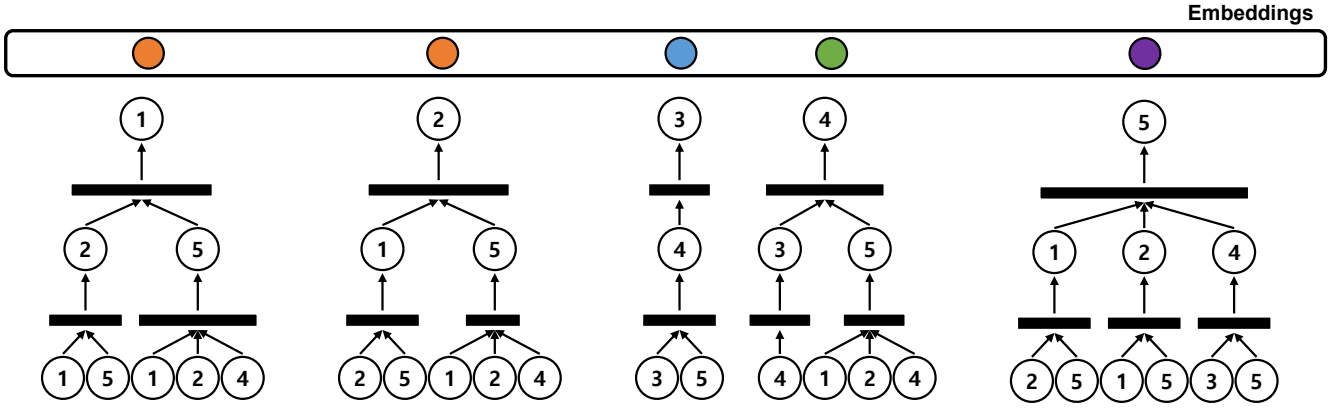
- Computational graphs are identical to **rooted subtree structures** around each node.



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Computational Graph 22

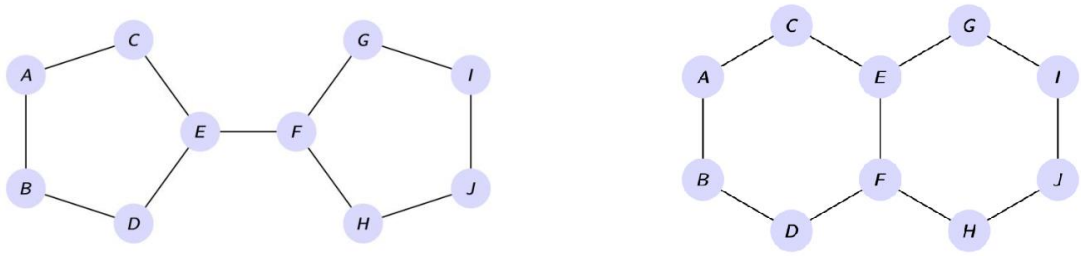
- GNN's node embeddings capture **rooted subtree structures**.
- Most expressive GNN maps different **rooted subtrees** into different node embeddings (represented by different colors).



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Where 1-WL fails: regular graphs 59

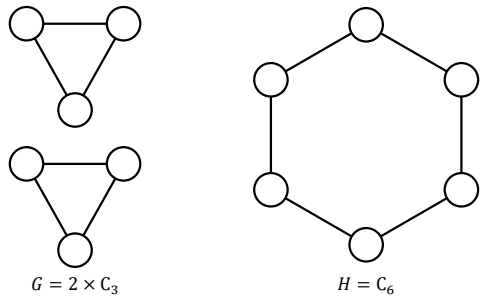
- The WL kernel cannot distinguish between some basic graph structures, e.g.,



- The WL kernel cannot count important sub-structures, e.g., cycles and cliques.

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Where 1-WL fails: regular graphs 60



Both are 2-regular on 6 vertices, so $c_v^{(t)} = \gamma_t$ for every v in either graph, $\forall t$.
Histograms agree at every round ⇒ 1-WL is silent.

No d-regular graphs at all
If G and H are both d-regular with n vertices, 1-WL assigns one colour to everything: the test is meaningless. The smallest non-isomorphic such pair already appears at $n = 6$.

Cannot count triangles
G has clustering coefficient 1; H has 0. No MPNN can report this difference - a fact with real consequences in chemistry and social network analysis.

Molecular twins
Decalin and bicyclopentyl (both $C_{10}H_{18}$) have identical 1-WL colorings but different chemistry - fused rings vs. a single bond.

CSL graphs
Circulant skip-link graphs (Murphy et al., 2019): a whole benchmark family of 4-regular graphs on which 1-WL models score at chance.

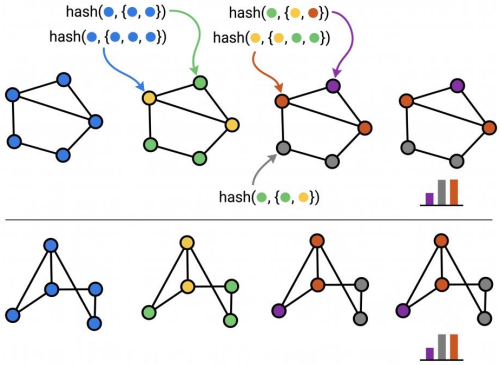
Recap: The 1-Weisfeiler-Lehman Test (1-WL) 65

- Often called *Color Refinement*, 1-WL operates on individual nodes.

$$c_v^{(t)} = \text{HASH}\left(c_v^{(t-1)}, \left\{\left\{c_u^{(t-1)} \mid u \in \mathcal{N}(v)\right\}\right\}\right)$$

Where:

- $c_v^{(t)}$ is the color of node v at iteration t.
- $\mathcal{N}(v)$ is the set of neighbors of v.
- $\{\{\ldots\}\}$ denotes a multiset (which allows duplicate colors).



Defining k-WL: Tuples & Neighbors 66

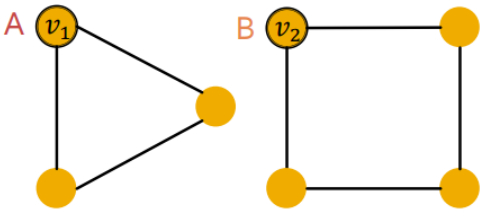
- **Idea**: Instead of looking at a node's topological neighbors, k-WL looks at **"neighboring tuples"** formed by swapping out exactly one node in the tuple for another node in the graph.
  - **Initialization**: Let $\mathbf{v} = (v_1, \ldots, v_k) \in V^k$ be a k-tuple of nodes. The initial color $c_{\mathbf{v}}^{(0)}$ encodes the isomorphism type of the subgraph induced by those k nodes
  - **Definition**: To update a tuple $\mathbf{v}$, we consider every vertex w in the entire graph V. For each w, we generate k new tuples by substituting w into each of the k positions of $\mathbf{v}$.

$\mathbf{v}_{j\leftarrow w} = (v_1, \ldots, v_{j-1}, w, v_{j+1}, \ldots, v_k)$ represent the tuple where the j-th node is replaced by w.

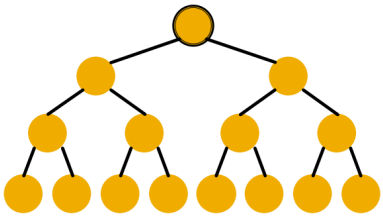
$$c_{\mathbf{v}}^{(t)} = \text{HASH}\left(c_{\mathbf{v}}^{(t-1)}, \left\{\left(c_{\mathbf{v}_{1\leftarrow w}}^{(t-1)}, \ldots, c_{\mathbf{v}_{k\leftarrow w}}^{(t-1)}\right) \mid w \in V\right\}\right)$$

Improving GNNs' Power 63

- Can the expressive power of GNNs be improved?
- There are basic graph structures that existing GNN frameworks cannot distinguish, such as differences in cycles.



Computational graphs for nodes v_1 and v_2 :



- GNNs' expressive power **can be improved** to resolve the above problem. [You et al. AAAI 2021, Li et al. NeurIPS 2020, Kanatsoulis et al. ICLR 2024]

References: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, http://cs224w.stanford.edu

Breaking the 1-WL ceiling 64

A	**Enrich the features**	Positional and structural encodings, random node initialization, and substructure counts. Cheapest option; message passing is untouched.	**≈ 3-WL and beyond, probabilistically**
B	**Subgraph and Ego-Network**	Run the GNN on a bag of subgraphs, on ego-networks, or on a lifted cell/simplicial complex instead of on G itself.	**> 1-WL; node-based subgraph GNNs ≤ 3-WL**
C	**Higher-Order GNNs**	k-GNN, invariant graph networks, PPGN: learn equivariant maps on tensors indexed by tuples of nodes.	**= k-WL exactly**
D	**Break the symmetry**	DropGNN, random features: perturb or randomise, then average over runs. Invariance holds only in distribution.	**universal with high probability**
E	**Go global**	Graph transformers with structural bias: all-pairs attention, expressivity carried entirely by the encoding.	**depends on the encoding used**

There is no free lunch: each family pays in compute, in guaranteed invariance, or in sample complexity.

From a Combinatorial Test to a Neural Architecture 69

➢ **1-Dimensional GNNs (1-GNNs) as Message Passing**: Standard GNNs operate by iteratively aggregating feature information over local node neighborhoods. A general 1-GNN update step at layer t for a node v is formalized as:

$$f^{(t)}(v) = f_{W_1}^{\text{merge}}\left(f^{(t-1)}(v), f_{W_2}^{\text{aggr}}(\{\{f^{(t-1)}(w) \mid w \in N(v)\}\})\right)$$

where $f_{W_1}^{\text{merge}}$ and $f_{W_2}^{\text{aggr}}$ are **differentiable**, **permutation-invariant** functions.

Theorem 1 (Upper Bound):

For any initial feature encoding and weight parameters $W^{(t)}$, the coloring $c_l^{(t)}$ induced by 1-WL always refines the features $f^{(t)}$ induced by a 1-GNN.

Theorem 2 (Equivalence)

There exists a sequence of parameters $W^{(t)}$ such that 1-GNNs have exactly the same power in distinguishing non-isomorphic graphs as the 1-WL test ($c_l^{(t)} \equiv f^{(t)}$).

References: Morris, Christopher, et al. "Weisfeiler and leman go neural: Higher-order graph neural networks." Proceedings of the AAAI conference on artificial intelligence. Vol. 33. No. 01. 2019..

Weisfeiler and leman go neural: Higher-order graph neural networks 70

➢ **Set-Based k-WL for Scalability**:

To overcome 1-WL limitations, *k-GNNs* leverage the k-*WL* algorithm by operating on k-element subsets of nodes, $[V(G)]^k$, rather than individual nodes.

➢ **Neighborhood Definitions for a k-set s:**

The broader neighborhood is $N(s) = \{t \in [V(G)]^k \mid |s \cap t| = k-1\}$. This is split into:

- **Local Neighborhood** ($N_L(s)$): $t \in N(s)$ where an edge $(v,w) \in E(G)$ exists for the unique $v \in s \setminus t$ and $w \in t \setminus s$.
- **Global Neighborhood** ($N_G(s)$): $N(s) \setminus N_L(s)$.

➢ ***k*-GNN Feature Computation:**

In each layer $t > 0$, the feature vector for a k-set s is updated as:

$$f_k^{(t)}(s) = \sigma\left(f_k^{(t-1)}(s) \cdot W_1^{(t)} + \sum_{u \in N_L(s) \cup N_G(s)} f_k^{(t-1)}(u) \cdot W_2^{(t)}\right)$$

References: Morris, Christopher, et al. "Weisfeiler and leman go neural: Higher-order graph neural networks." Proceedings of the AAAI conference on artificial intelligence. Vol. 33. No. 01. 2019..

The Hierarchical k-GNN Architecture 71

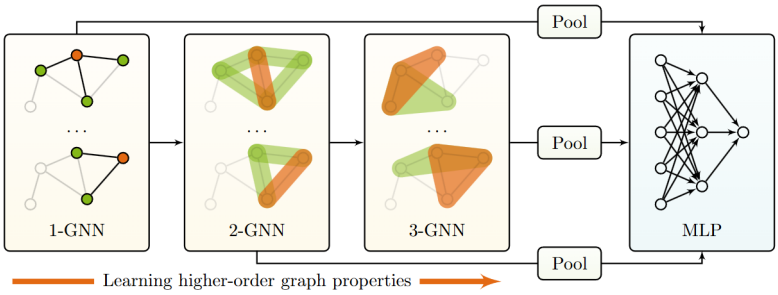
➢ **Inductive Bias for Hierarchical Structures:**

Instead of using simple one-hot indicator vectors of isomorphism types as initial inputs, the hierarchical variant connects representations learned at different dimensions.

➢ **Hierarchical Initialization:**

Features learned by a $(k-1)$-dimensional GNN are used as the initial features $f_k^{(0)}(s)$ for the k-GNN layer:

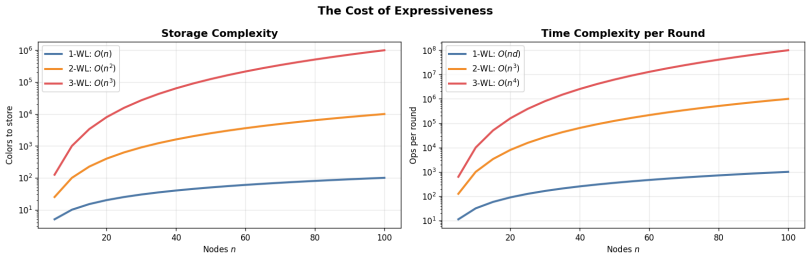
$$f_k^{(0)}(s) = \sigma\left(\left[f_{\text{iso}}(s), \sum_{u \subset s} f_{k-1}^{(T_{k-1})}(u)\right] \cdot W_{k-1}\right)$$



References: Morris, Christopher, et al. "Weisfeiler and leman go neural: Higher-order graph neural networks." Proceedings of the AAAI conference on artificial intelligence. Vol. 33. No. 01. 2019..

The Cost of Going k-WL and k-GNNs 72

WL Level	Storage	Time / round
1-WL	$O(n)$	$O(n \cdot d)$
2-WL	$O(n^2)$	$O(n^3)$
3-WL	$O(n^3)$	$O(n^4)$
k-WL	$O(n^k)$	$O(n^{k+1})$



⚠ **Practical note** For $n = 100$, 3-WL needs 10^6 triples and 10^8 operations per round.
Subgraph GNNs and equivariant architectures aim for higher expressiveness at lower cost.

Graph Substructure Networks (GSN) 73

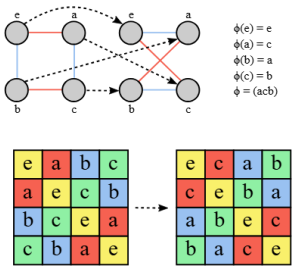
➢ **Preliminaries**:

To make GNNs structure-aware, we must mathematically define a vertex or edge's "structural role" within a given substructure $\mathcal{H}$.

➢ **Automorphism**: An isomorphism that maps a graph $\mathcal{H}$ onto itself. The set of all automorphisms forms the group $\mathrm{Aut}(\mathcal{H})$.

➢ **Orbits**: The automorphism group partitions the vertices of $\mathcal{H}$ into disjoint subsets called orbits. This groups vertices by their structural equivalency.

$$\mathcal{O}_{\mathcal{H}}(v) = \{u \in V_{\mathcal{H}} \mid \exists g \in \mathrm{Aut}(\mathcal{H}) \text{ s.t. } g(u) = v\}$$



References: Bouritsas, Giorgos, et al. "Improving graph neural network expressivity via subgraph isomorphism counting." IEEE Transactions on Pattern Analysis and Machine Intelligence 45.1 (2022): 657-668.

Graph Substructure Networks (GSN) 74

➢ **Key Concepts**:

- **Automorphisms and Structural Roles**: To ensure permutation equivariance, GSN counts the appearances of predefined small graphs $\mathcal{H} = \{H_1, \ldots, H_K\}$ by mapping nodes/edges to their structural roles. These roles are defined by the orbits of the automorphism group of H (denoted $O_{H,i}$).
- **Vertex Structural Features**: For a vertex v, the feature vector counts how many times v appears in subgraph H acting in the specific structural role of orbit i:

$$x^V_{H,i}(v) = \left|\{G_S \cong H : v \in V_{G_S}, f(v) \in O^V_{H,i}\}\right|$$

- **Edge Structural Features**: Similarly, for an edge (u, v) mapping to edge orbit i:

$$x^E_{H,i}(u,v) = \left|\{G_S \cong H : (u,v) \in E_{G_S}, \big(f(u), f(v)\big) \in O^E_{H,i}\}\right|$$

where:

- G_S is a subgraph of the main graph G.
- f is the isomorphism mapping the subgraph G_S to the target substructure H.
- $O^V_{H,i}$ and $O^E_{H,i}$ are the i-th vertex and edge orbits of H.

References: Bouritsas, Giorgos, et al. "Improving graph neural network expressivity via subgraph isomorphism counting." IEEE Transactions on Pattern Analysis and Machine Intelligence 45.1 (2022): 657-668.

Graph Substructure Networks (GSN) 75

➢ **Integrating Features into GNNs:**

GSN extends standard MPNNs by injecting the structural features x_v^V (for GSN-v) directly into the message generation phase.

1. **Message Generation:**

$$m_v^{t+1} = \mathcal{M}_{t+1}\left(\left\{\left(h_v^t, h_u^t, x_v^V, x_u^V, e_{u,v}\right)\right\}_{u\in\mathcal{N}(v)}\right)$$

Where $\mathcal{M}$ is a permutation-invariant aggregation function (e.g., Sum + MLP).

2. **Node State Update:**

$$h_v^{t+1} = \mathrm{UP}_{t+1}(h_v^t, m_v^{t+1})$$

Where UP is an arbitrary function approximator (e.g., an MLP).

For a detailed proof, see URL.

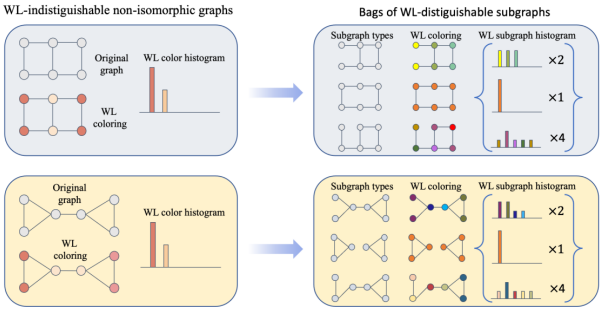
References: Bouritsas, Giorgos, et al. "Improving graph neural network expressivity via subgraph isomorphism counting." IEEE Transactions on Pattern Analysis and Machine Intelligence 45.1 (2022): 657-668.

Equivariant Subgraph Aggregation (ESAN) 79

➢ **The Core Intuition**: While two entire graphs G_1 and G_2 might not be distinguishable by an MPNN, they often contain localized substructures (subgraphs) that are **distinguishable**.

➢ **The ESAN Approach**: Instead of processing the original graph G, the model applies a subgraph selection policy π to map the graph into a bag (multiset) of subgraphs, denoted as:

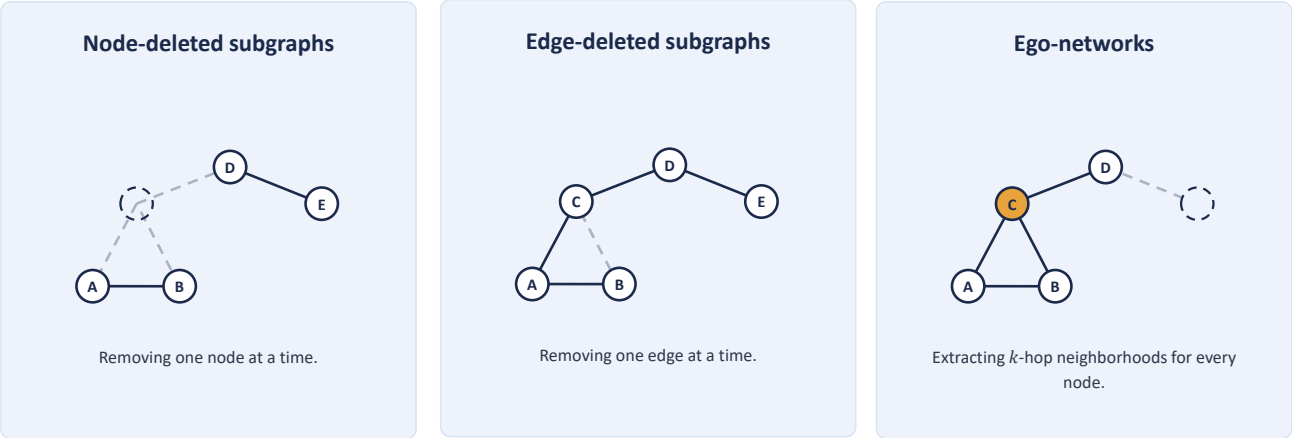
$$S_G = \{\{G_1, G_2, \ldots, G_m\}\}$$



References: Beatrice Bevilacqua, Fabrizio Frasca, Derek Lim, Balasubramaniam Srinivasan, Chen Cai, Gopinath Balamurugan, Michael M. Bronstein, Haggai Maron: Equivariant Subgraph Aggregation Networks

Equivariant Subgraph Aggregation (ESAN) 80

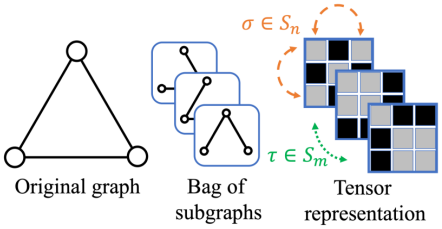
- **Subgraph Selection Policies**: This structural mapping is deterministic and typically relies on policies such as:



References: Beatrice Bevilacqua, Fabrizio Frasca, Derek Lim, Balasubramaniam Srinivasan, Chen Cai, Gopinath Balamurugan, Michael M. Bronstein, Haggai Maron: Equivariant Subgraph Aggregation Networks

Equivariant Subgraph Aggregation (ESAN) 81

- **The Symmetry Group**: The multiset of subgraphs S_G forms a highly structured mathematical object. Assuming m subgraphs and n nodes, it can be represented by adjacency matrices $A \in \mathbb{R}^{m \times n \times n}$ and node features $X \in \mathbb{R}^{m \times n \times d}$. The architecture must be exactly equivariant to the direct product group $S_m \times S_n$ (permutations of the m subgraphs and n nodes).



- **DSS-GNN (Deep Sets for Subgraphs):** To capture this equivariance, the network updates the hidden representation H_i of each subgraph G_i at layer t using two graph encoders ($L^1$and L^2):

$$H_i^{(t+1)} = L^1\left(H_i^{(t)}, A_i\right) + L^2\left(\sum_{j=1}^{m} H_j^{(t)}, A_{agg}\right)$$

References: Beatrice Bevilacqua, Fabrizio Frasca, Derek Lim, Balasubramaniam Srinivasan, Chen Cai, Gopinath Balamurugan, Michael M. Bronstein, Haggai Maron: Equivariant Subgraph Aggregation Networks

Equivariant Subgraph Aggregation (ESAN) 82

➢ **DS-GNN (Deep Sets without Sharing):** A computationally simpler variant that disables the information-sharing component (L^2) during the intermediate layers. It processes all subgraphs in parallel and applies a global set-pooling at the readout phase:

$$F(G) = \mathrm{MLP}\left(\sum_{i=1}^{m} \mathrm{GNN}\left(H_i^{(0)}, A_i\right)\right)$$

Strictly More Powerful than MPNNs:

$$1\text{−WL} < \mathrm{DS}\text{−WL} \subseteq \mathrm{DSS}\text{−WL}$$

Theorem (Expressive Power)

- DSS-GNN is at least as powerful as the DSS-WL test [1].
- DS-GNN is at least as powerful as the DS-WL test [1].

[1] DSS-WL and DS-WL Tests: To bound the theoretical power of the new architectures, the authors introduce two novel color refinement procedures inspired by the standard WL test, designed to operate over multisets of subgraphs.

References: Beatrice Bevilacqua, Fabrizio Frasca, Derek Lim, Balasubramaniam Srinivasan, Chen Cai, Gopinath Balamurugan, Michael M. Bronstein, Haggai Maron: Equivariant Subgraph Aggregation Networks

2-Weisfeiler-Lehman Test (2-WL) 67

- The general k-WL equation applied specifically where $k = 2$.
- **Scope**: Operates on node pairs (v_1, v_2), effectively coloring every **edge** and **non-edge**.

$$c^{(t)}_{(v_1,v_2)} = \text{HASH}\left(c^{(t-1)}_{(v_1,v_2)}, \left\{\left\{\left(c^{(t-1)}_{(w,v_2)}, c^{(t-1)}_{(v_1,w)}\right) \mid w \in V\right\}\right\}\right)$$

- **Initialization**: $c^{(0)}_{(v_1,v_2)}$ encodes whether the pair is a self-loop $(v_1 = v_2)$, a connected edge, or a disconnected non-edge.
- **Definition**: A pair (v_1, v_2) is updated by sweeping over every node w in the graph, substituting w into the first position (w, v_2), then the second position (v_1, w), and hashing the resulting colors.

Notes

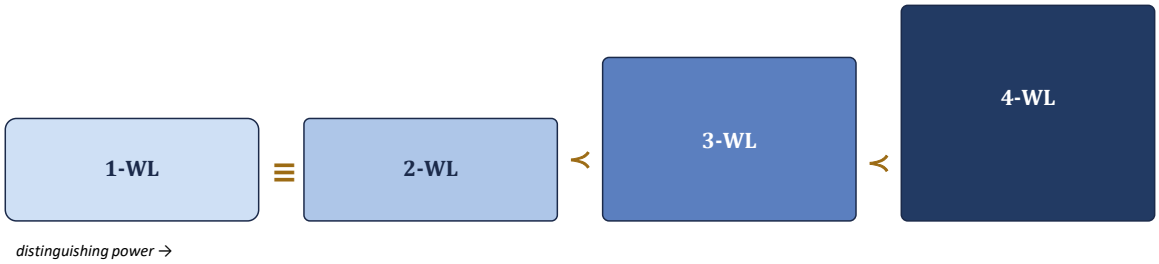
You may sometimes encounter "Folklore" k-WL in literature, denoted k-FWL. Folklore 2-FWL operates slightly differently and is equivalent in power to standard 3-WL

Beyond 1-WL: The k-WL Hierarchy 68

$$k\text{−WL} \preceq (k+1)\text{−WL}, \quad \text{strict for } k \geq 2$$

$$1\text{−WL} \equiv 2\text{−WL}$$

1-WL and 2-WL have equivalent expressive power- strict gains require $k \geq 3$.



References: Cai, Jin-Yi, Martin Fürer, and Neil Immerman. "An optimal lower bound on the number of variables for graph identification." Combinatorica 12.4 (1992): 389-410.

From Stars to Subgraphs 76

- **The Star-Topology Limitation**: The communication pattern in standard MPNNs strictly follows a **star-shaped topology** (a central node connected only to its 1-hop neighbors). The node update mechanism is generalized as:

$$h_v^{(t)} = \text{UPDATE}^{(t)}\left(h_v^{(t-1)}, \text{AGG}^{(t)}\left(\left\{\left\{h_u^{(t-1)} : u \in \mathcal{N}(v)\right\}\right\}\right)\right)$$

- **From Stars to k-Egonets**: To shatter the 1-WL limit without resorting to unscalable higher-order tensors, the GNN-AK (GNN As Kernel) framework shifts the localized receptive field from a star to a general induced subgraph, such as a k-hop egonet denoted by $G[\mathcal{N}_k(v)]$.
- **Subgraph-1-WL**: Instead of hashing a star graph, Subgraph-1-WL hashes the entire k-hop induced subgraph:

$$c_v^{(t)} = \text{HASH}\left(G^{(t)}[\mathcal{N}_k(v)]\right)$$

References: Lingxiao Zhao, Wei Jin, Leman Akoglu, Neil Shah: From Stars to Subgraphs: Uplifting Any GNN with Local Structure Awareness. ICLR 2022

From Stars to Subgraphs 77

- To fully exploit the subgraph structure, the **GNN-AK+** architecture calculates the new embedding for a node j by concatenating three distinct theoretical encodings:
  - **Centroid Encoding**: Captures the node's explicit role as the root of its own egonet.

$$h_{\text{cent}}^{(j)} = \text{Emb}(j|\text{Sub}[j])$$

  - **Subgraph Encoding**: Captures the global structural signature of j's encompassing egonet.

$$h_{\text{sub}}^{(j)} = \sum_{i \in \text{Sub}[j]} \text{Emb}\left(i|\text{Sub}[j]\right)$$

  - **Context Encoding**: Captures how node j is perceived externally across overlapping neighboring egonets.

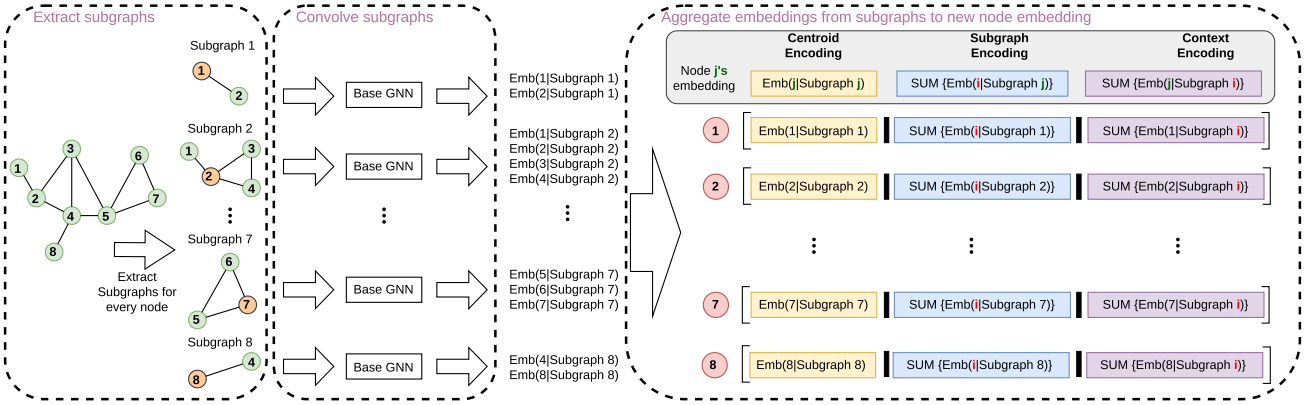
$$h_{\text{ctx}}^{(j)} = \sum_{i:j \in \text{Sub}[i]} \text{Emb}\left(j|\text{Sub}[i]\right)$$

- **Aggregation**: These three matrices are concatenated and updated through subsequent layers, effectively lifting the base GNN to a highly expressive local-structure-aware model.

References: Lingxiao Zhao, Wei Jin, Leman Akoglu, Neil Shah: From Stars to Subgraphs: Uplifting Any GNN with Local Structure Awareness. ICLR 2022

From Stars to Subgraphs

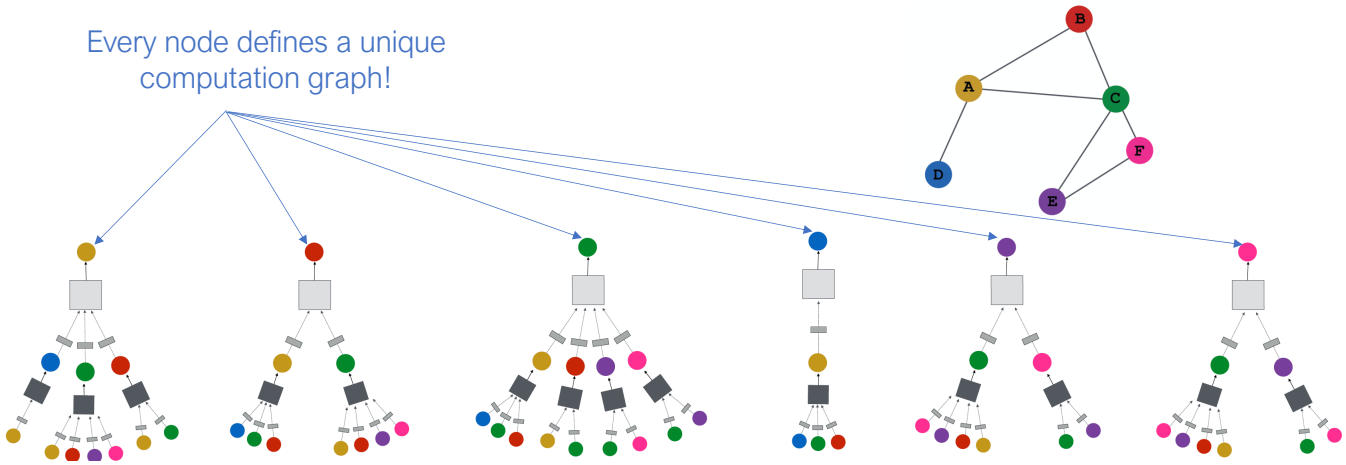
78



References: Lingxiao Zhao, Wei Jin, Leman Akoglu, Neil Shah: From Stars to Subgraphs: Uplifting Any GNN with Local Structure Awareness. ICLR 2022

Recap: Neighborhood Aggregation 5

➢ **Intuition**: Network neighborhood defines a computation graph



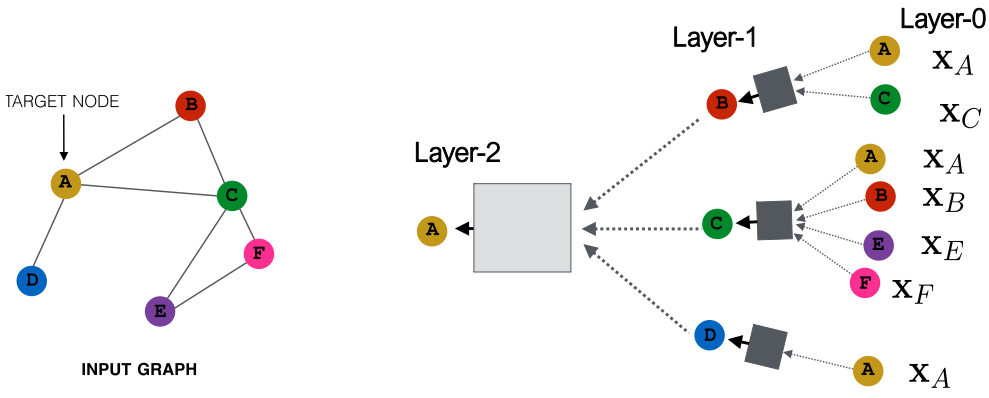
Representation Learning on Networks, snap.stanford.edu/proj/embeddings-www, WWW 2018

Recap: Neighborhood Aggregation 6

➢ Nodes have embeddings at each layer.

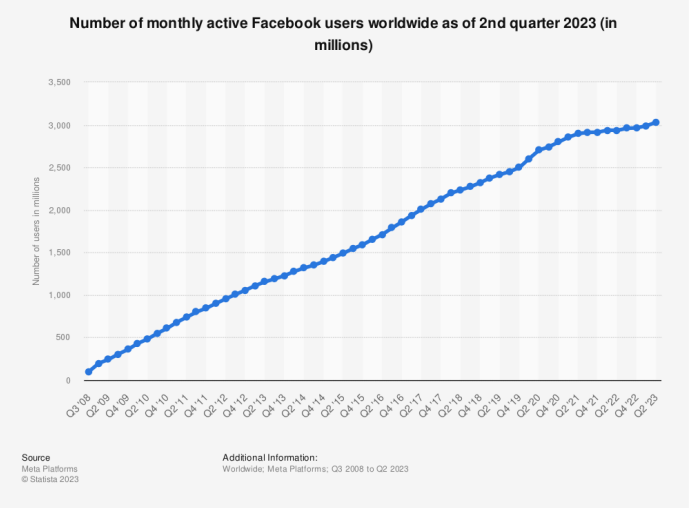
➢ Model can be arbitrary depth.

➢ "layer-0" embedding of node A is its input feature, i.e. x_A

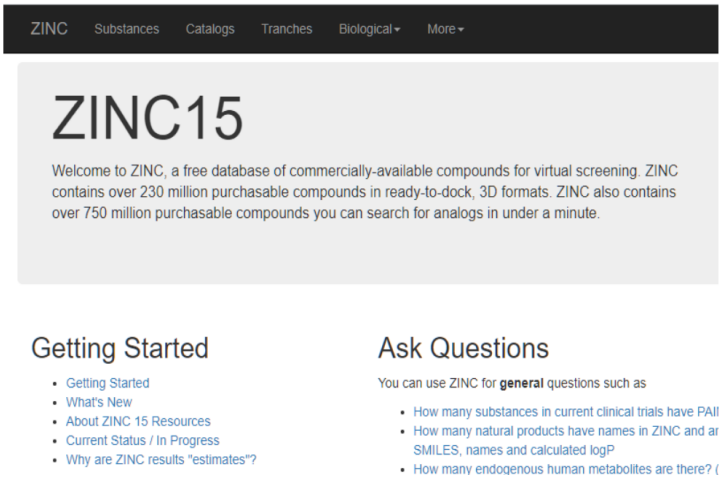


Graphs in the real-world can be very large 9

Large scale



Large number



References: https://www.statista.com/statistics/264810/number-of-monthly-active-facebook-users-worldwide/

Benchmark graphs are already huge 10

ogbn-papers100M	Large scale	111,059,956 nodes 1,615,685,872 edges
ogbn-products	Medium scale	2,449,029 nodes 61,859,140 edges
ogbn-proteins	Medium scale	132,534 nodes 39,561,252 edges
ogbn-mag	Medium scale	1,939,743 nodes 21,111,007 edges
ogbn-arxiv	Small scale	169,343 nodes 1,166,243 edges

Bars on a log scale starting at 10^5. Node-classification datasets of the Open Graph Benchmark (OGB).

111M

nodes in **ogbn-papers100M**, joined by more than **1.6 billion** edges

Even the “small” **ogbn-arxiv** has 169K nodes and 1.17M edges.

References: Ma, Hehuan, Yu Rong, and Junzhou Huang. "Graph neural networks: Scalability." Graph neural networks: foundations, frontiers, and applications. Singapore: Springer Nature Singapore, 2022. 99-119.

Graph Convolutional Network Limitations 11

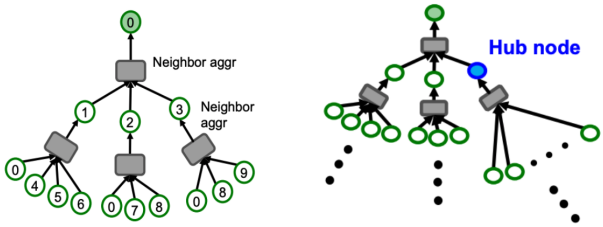
➢ Computationally Expensive:

- We need to generate the complete K-hop neighborhood computational graph and then need to aggregate plenty of information from its surroundings.
- As we go deeper into the neighborhood, the computation graph becomes exponentially large.

  → problem while fitting these computational graphs inside the GPU memory.

➢ The curse of Hub nodes or Celebrity nodes:

- Hub nodes are those nodes that are very high-degree nodes in the graph.



Two obstacles block the original GNN 12

1. Memory

Most GNNs keep the **entire adjacency matrix** and the **featur e/embedding matrices** of all nodes in memory.

For very large graphs this simply does not fit, so the GNN can not be applied directly.

2. Gradient updates

Every training iteration updates the gradient of **every node**, r ecursively through its neighbors.

That is as slow as full-batch gradient descent on a huge data set.

GD: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{n} \sum_{i=1}^{n} \ell_i$

Precise but computationally expensive
(processes all n samples)

SGD: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell_i$

Fast but introduces high variance
(processes a small batch $\mathcal{B}$)

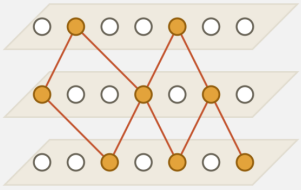
Three sampling paradigms 13



Node-wise

Sample around each target node

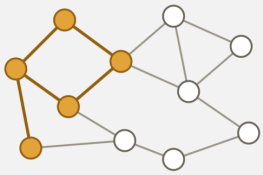
Pick a subset of each node's neighbors, recursively. GraphSAGE, VR-GCN



Layer-wise

Sample per convolutional layer

Pick a fixed set of nodes per layer, shared by the batch. FastGCN, ASGCN



Graph-wise

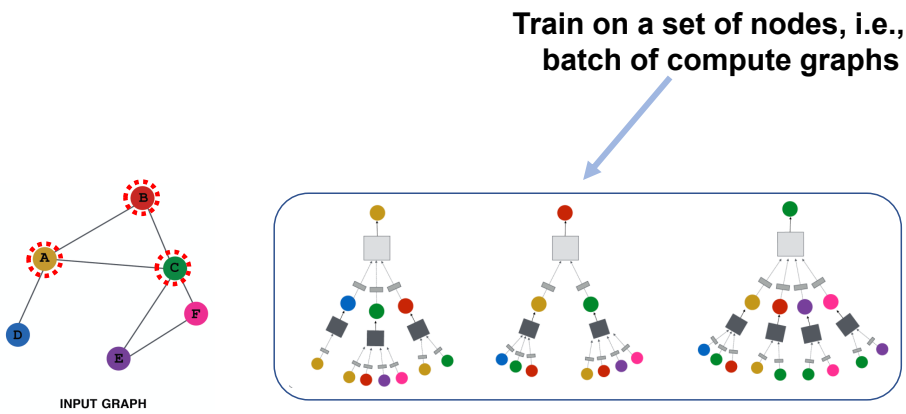
Sample whole sub-graphs

Cut the graph into sub-graphs; run the full GNN on each. Cluster-GCN, GraphSAINT

References: Ma, Hehuan, Yu Rong, and Junzhou Huang. "Graph neural networks: Scalability." Graph neural networks: foundations, frontiers, and applications. Singapore: Springer Nature Singapore, 2022. 99-119.

Two issues towards the large-scale GNNs 14

- How to design efficient sampling algorithm?
- How to guarantee the sampling quality?

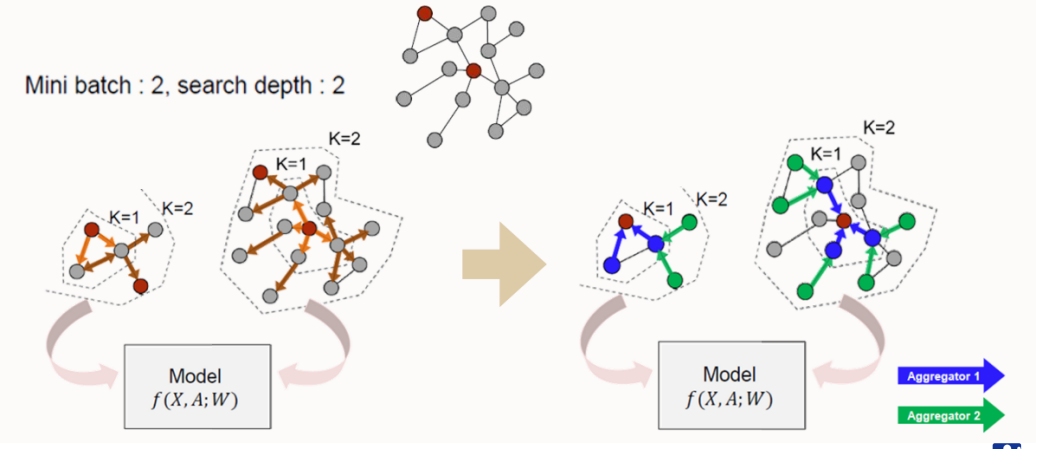


GraphSAGE: Mini batch

21

➢ **Mini batch:** There are three steps:

- Sample neighbourhood.
- Aggregate feature information from neighbours.
- Predict graph context and label using aggregated information.

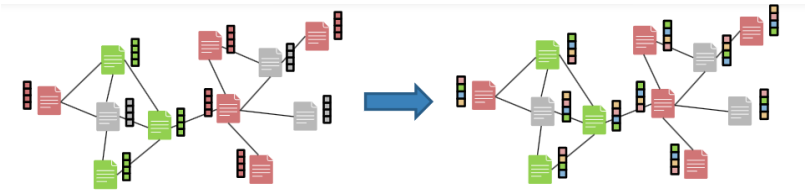


Graph-wise Sampling: ClusterGCN 45

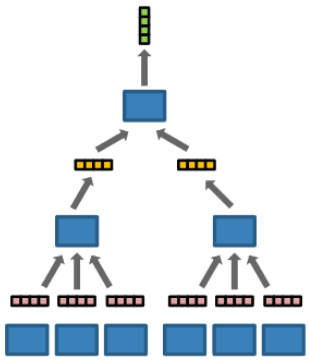
➢ **GCN is not trivial:**

In standard neural networks (e.g., CNN), loss function can be decomposed as $\sum_{i=0}^{N} loss(x_i, y_i)$

- In GCN, loss on a node not only depends on itself but all its neighbors.
- This dependency brings difficulties when performing **SGD on GCN.**



What we expect



What truly happen

References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

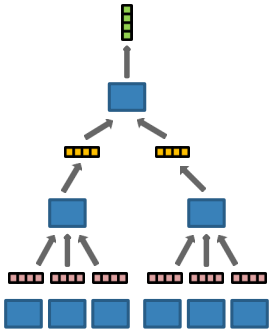
Graph-wise Sampling: ClusterGCN 46

➢ Issues come from **high computation costs**.

➢ Suppose we desire to calculate a target node's loss with a 2-layer GCN.

➢ To obtain its final representation, needs all node embeddings in its 2-hop neighborhood.

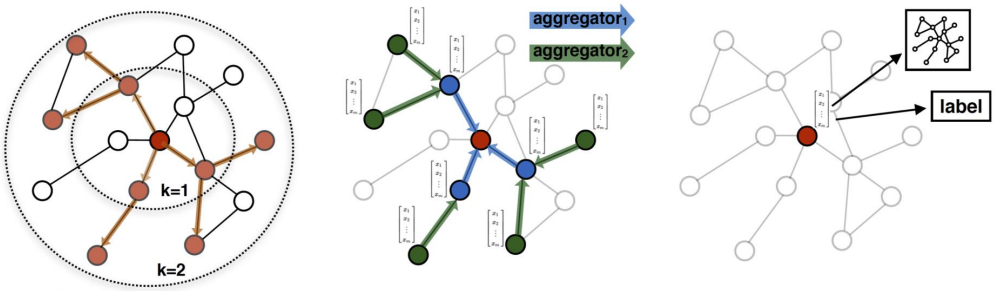
➢ **For example:** 9 nodes' embeddings needed but only get 1 loss (low utilization).



Graph-wise Sampling: ClusterGCN 47

➢ **Idea**: Subsample a *smaller number* of neighbors.

- For example, GraphSAGE (NeurIPS'17) considers a subset of neighbors per node.
- But it still suffers from **recursive neighborhood expansion**.

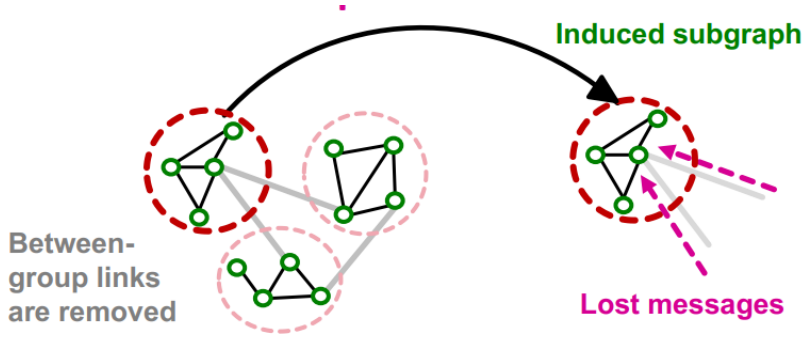


1. Sample neighborhood
2. Aggregate feature information from neighbors
3. Predict graph context and label using aggregated information

Graph-wise Sampling: ClusterGCN 48

➢ **Problems**:

- The induced subgraph removes between group links.
- As a result, messages from other groups will be lost during message passing, which could hurt the GNN's performance.



References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

Graph-wise Sampling: ClusterGCN 49

➢ **Ways to improve the Embedding Utilization**:

- If considering all losses at one time (full-batch), 9 nodes' embedding used and got 9 losses.

$$GCN_{2-layer}(A, X) = A\sigma(AXW^{(0)})W^{(1)}$$

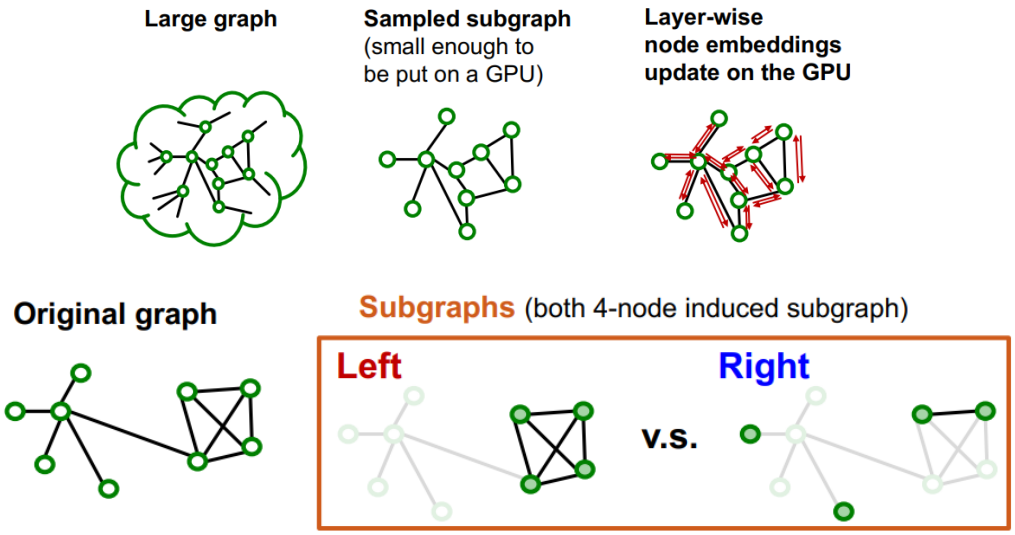
- Embedding Utilization: optimal.
- The key is to re-use nodes' embeddings as many as possible.
- Focus on dense parts of the graph.

References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Graph-wise Sampling: ClusterGCN 50

➢ **Idea**: Apply graph clustering algorithm (e.g., METIS) to identify dense subgraphs.



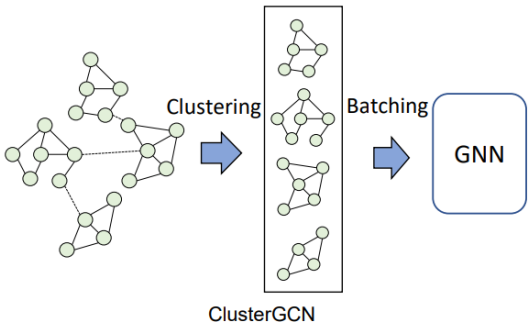
네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Graph-wise Sampling: ClusterGCN 51

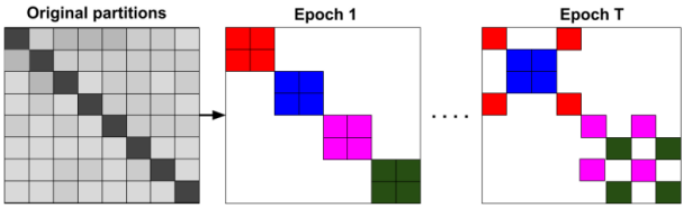
➢ Extract small clusters based efficient clustering algorithms.

$$\bar{G} = [G_1, \cdots, G_c] = [\{\mathcal{V}_1, \mathcal{E}_1\}, \cdots, \{\mathcal{V}_c, \mathcal{E}_c\}],$$

$$\bar{A} = \begin{bmatrix} A_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & A_{cc} \end{bmatrix}, \Delta = \begin{bmatrix} 0 & \cdots & A_{1c} \\ \vdots & \ddots & \vdots \\ A_{c1} & \cdots & 0 \end{bmatrix},$$



➢ Random batching at the subgraph level.

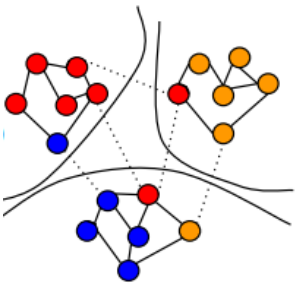


Graph-wise Sampling: ClusterGCN 52

➢ **Idea**: Apply graph clustering algorithm (e.g., METIS) to identify dense subgraphs.

➢ **Cluster-GCN**

- Partition the graph into several clusters, remove between-cluster edges.
- Each subgraph is used as a mini-batch in SGD.
- Embedding utilization is optimal because nodes' neighbors stay within the cluster.

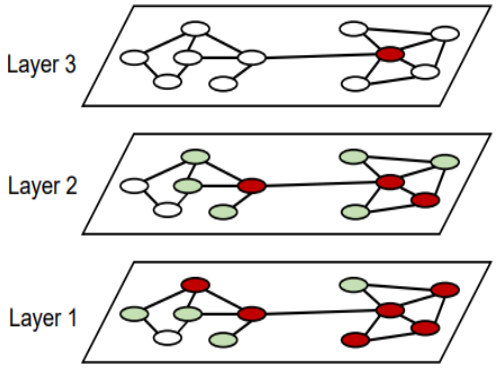


References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

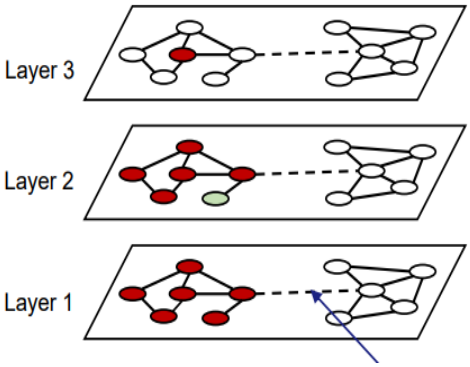
Graph-wise Sampling: ClusterGCN 53

➢ Neighbor expansion control:

Only consider the nodes in the same clusters.



Fix-size neighbor sampling $S = 2$



Only sample the nodes in the clusters

Cluster-GCN in summary 54

Contributions

+ Nodes outside the batch's sub-graph are never sampled, so **no neighborhood expansion**.
+ Practical: good performance and **good memory usage**.
+ Uses an unmodified GNN on each sub-graph.

Limitations

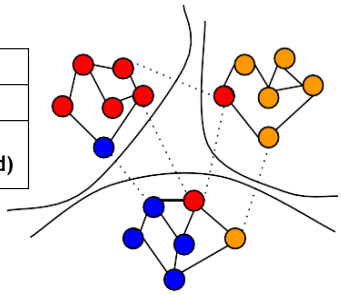
– No analysis of **sampling quality**: bias and variance of the scheme are not studied.
– Performance is **highly correlated with the clustering algorithm**.

References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

Graph-wise Sampling: ClusterGCN 55

- Partition the graph into several clusters, remove between-cluster edges.
- Each subgraph is used as a mini-batch in SGD.
- Embedding utilization is optimal because nodes' neighbors stay within the cluster.
- Even though 20% edges are removed, the accuracy of GCN model remains similar.

CiteSeer	Random partitioning	Graph partitioning
1 (no partitioning)	72.0	72.0
100 partitions	46.1	71.5 (~20% edges removed)

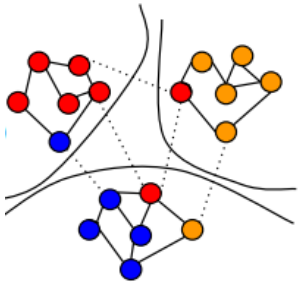


References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Graph-wise Sampling: ClusterGCN 56

- **Issues**: **Imbalanced** label distribution.
  - Nodes with similar labels are clustered together.
  - Hence, the label distribution within a cluster could be different from the original data.
  - Leading to a biased SGD.



References: Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, Cho-Jui Hsieh: Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. KDD 2019: 257-266

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

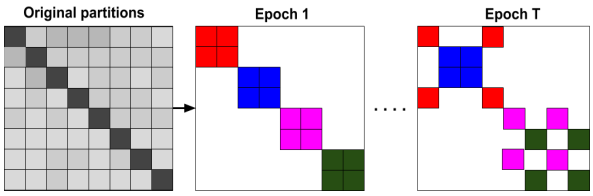
Graph-wise Sampling: ClusterGCN 57

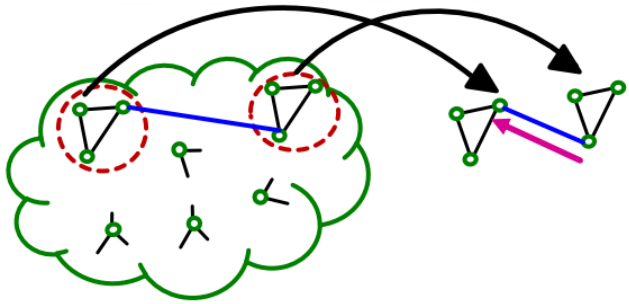
➢ **Multiple Clusters:**

- A random selection of multiple clusters as a batch has been proposed.

➢ **Two advantages:**

- Balance label distribution within a batch.
- Recover some missing edges between-cluster.





ClusterGCN: algorithm 58

Mini-batch training:

- For each mini-batch, **randomly sample a set of q node groups**: $\{V_{t_1}, \ldots, V_{t_q}\} \subset \{V_1, \ldots, V_C\}$.
- **Aggregate all nodes across the sampled node groups**: $V_{aggr} = V_{t_1} \cup \cdots \cup V_{t_q}$
- Extract the **induced subgraph** $G_{aggr} = (V_{aggr}, E_{aggr})$, where $E_{aggr} = \{(u, v) \mid u, v \in V_{aggr}\}$
  - **E_{aggr} also includes between-group edges!**

Algorithm 1: Cluster GCN

Input: Graph A, feature X, label Y;
Output: Node representation $\bar{X}$

Nodes Clustering
1 Partition graph nodes into c clusters $\mathcal{V}_1, \mathcal{V}_2, \cdots, \mathcal{V}_c$ by METIS;

2 **for** $iter = 1, \cdots, max_iter$ **do**

Random train q clusters
3 Randomly choose q clusters, $t_1, \cdots, t_q$ from $\mathcal{V}$ without replacement;
4 Form the subgraph $\bar{G}$ with nodes $\bar{\mathcal{V}} = [\mathcal{V}_{t_1}, \mathcal{V}_{t_2}, \cdots, \mathcal{V}_{t_q}]$ and links $A_{\bar{\mathcal{V}},\bar{\mathcal{V}}}$;

Train and optimize
5 Compute $g \leftarrow \nabla \mathcal{L}_{A_{\bar{\mathcal{V}},\bar{\mathcal{V}}}}$ (loss on the subgraph $A_{\bar{\mathcal{V}},\bar{\mathcal{V}}}$);
6 Conduct Adam update using gradient estimator g

7 Output: $\{W_l\}_{l=1}^{L}$

Graph-wise sampling

Sample whole sub-graphs, then run an ordinary full GNN on each one as a mini-batch.

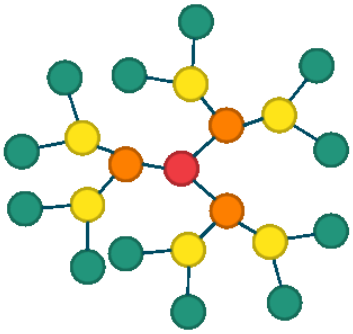
Representative: Cluster-GCN, GraphSAINT



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

GraphSAINT: Problem Statements 59

- *"Neighbour explosion"*, GCNs rely on aggregating neighbor information to update nodes.
- It is very intuitive that the more layers of GCNs, the more neighbors need to be considered when updating nodes; while the number of layers is fixed.
- The higher the average degree, the more neighbors need to be considered.

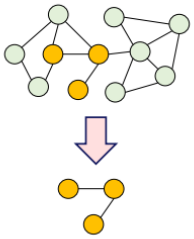


References: Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, Viktor K. Prasanna: GraphSAINT: Graph Sampling Based Inductive Learning Method. ICLR 2020

Graph-wise Sampling: GraphSAINT 60

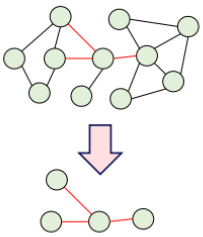
- Directly sample a subgraph for mini-batch training according to subgraph sampler.
- Sampler construction.

Node sampler



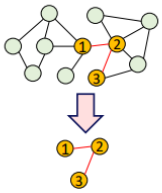
Uniformly sample nodes.

Edge sampler



Sample edge with probably $p_{u,v} \propto 1/d_u + 1/d_v$

Random walk sampler

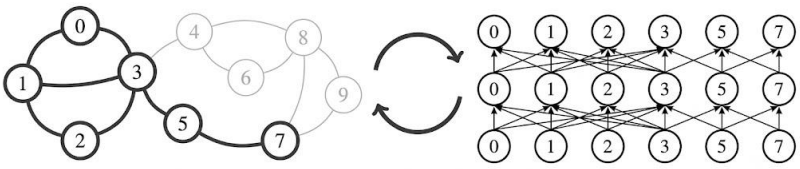


Sample edge with probably $p_{u,v} \propto \boldsymbol{B}_{u,v} + \boldsymbol{B}_{v,u}$

- $B_{u,v}$: the probability of a random walk to start at u and end at v in L hops.

Graph-wise Sampling: GraphSAINT 61

- Sample a small subgraph, then build a complete GNN.
- Constant neighborhood size.
- Not an I.I.D data sampler.
- **Why?**
  - Popular users will be sampled more frequently, i.e., influencers, …
- **How?**
  - Normalize aggregation and mini batch loss by edge/node sampling frequency.



$\mathcal{G}_s = \text{SAMPLE}(\mathcal{G})$ Full GCN on $\mathcal{G}_s$

References: Graph Embedding – DATA SCIENCE LAB. (2021). Usc.Edu. https://sites.usc.edu/dslab/projects/graph-embedding/

Graph-wise Sampling: GraphSAINT 62

➢ How to eliminate the **bias** introduce by the sampler?

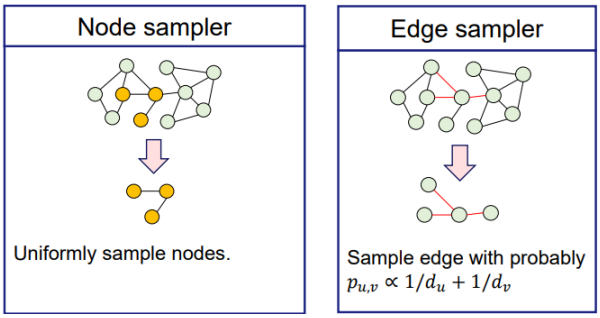
- Normalize minibatch loss by probability of sampling each node:

$$\mathcal{L}_{\text{batch}} = \sum_{v \in G_S} L_v / \lambda_v, \lambda_v = |V| p_v.$$

- p_v: the probability of a node $v \in V$ being sampled.
- p_{uv}: the probability of an edge $u, v \in E$ being sampled.

- Normalize neighbor aggregation by probability of sampling each edge:

$$a(u, v) = p_{u,v}/p_v$$



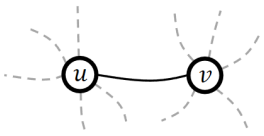
Graph-wise Sampling: GraphSAINT 63

➢ Another issue: Need to preserve connectivity → **Variance reduction**.

➢ How?

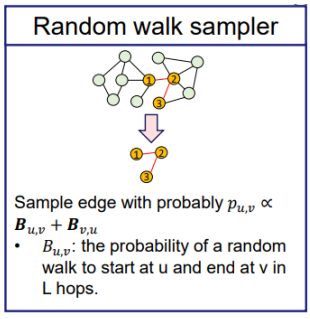
- **Important neighbors** to be sampled more frequently.
- Independent edge sampling: optimal edge probability for variance minimization.

$$p_{u,v} \propto \frac{1}{d_u} + \frac{1}{d_v}$$





Sample "—" more
Sample "—" less



➢ Extension

- Multi-layer version of random edge sampling → Random walk sampler.
- Variations of random walk sampler → Multi-dimensional RW, etc.

Why are GCNs limited to shallow structures? 14

Obstacle	What goes wrong	Root cause	Grows with
Over-smoothing	Node representations converge; classes become indistinguishable	Repeated low-pass filtering by $\hat{A}$ (Laplacian smoothing)	Number of layers; well-connected graphs
Over-squashing	Information from distant nodes is compressed and lost	Exponentially growing receptive field squeezed into fixed-width vectors through bottlenecks	Long-range dependencies; bottleneck topology
Under-reaching	Relevant nodes are never seen at all	Fewer layers than the problem radius r	Too few layers
Optimization	Vanishing gradients, overfitting, unstable training	Many stacked weight matrices and nonlinearities	Number of layers; few labeled nodes

네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Over-smoothing vs. Over-squashing

Both make deep models fail, through opposite mechanisms: features mix too much, or the information that must be mixed cannot fit.

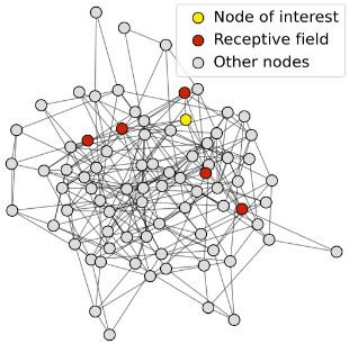


네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

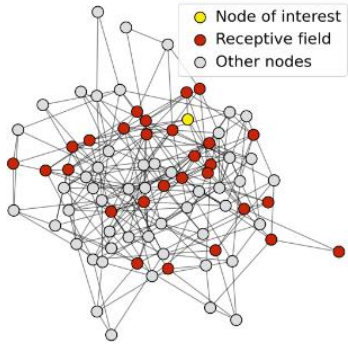
Why current GCNs cannot go deeper? 16

- **Over-smoothing problem**: All the node embeddings converge to the same value
  - This is bad because we want to use node embeddings to differentiate nodes.
- Why does the over-smoothing problem happen?
  - **Receptive field**: The set of nodes that determine the embedding of a node of interest.
    - In a K-layer GNN, each node has a receptive field of K-hop neighborhood.

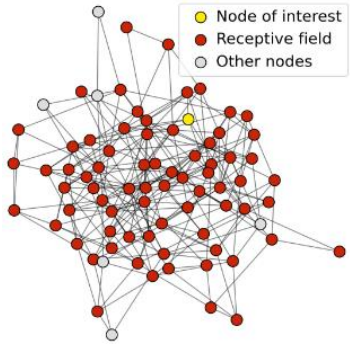
Receptive field for 1-layer GNN



Receptive field for 2-layer GNN



Receptive field for 3-layer GNN

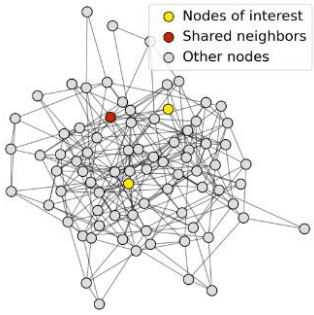


References: Jure Leskovec, Stanford CS224W: Machine Learning with Graph

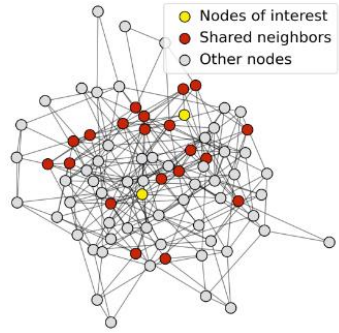
Why current GCNs cannot go deeper? 17

- **Over-smoothing**:
  - Receptive field overlap for two nodes: shared neighbors quickly grows when increase the number of hops (number of GNN layers).
    - If two nodes have highly overlapped receptive fields, then their embeddings are highly similar.

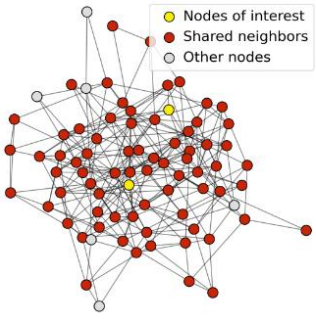
1-hop neighbor overlap
Only 1 node



2-hop neighbor overlap
About 20 nodes



3-hop neighbor overlap
Almost all the nodes!



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graph

Defining Over-smoothing (The Spatial View) 18

- **Concept:** Over-smoothing occurs when a node's receptive field expands to encompass the entire graph, causing local structural features to be washed out by global graph statistics.
- **Mechanism**: GNN message passing is mathematically analogous to a random walk on the graph. The layer propagation step (ignoring non-linearities) operates similarly to applying a Markov chain transition matrix

$$\tilde{A}_{rw} = \tilde{D}^{-1}\tilde{A}$$

- **The Infinite Depth Limit**: Due to the expander-graph properties and small-world nature of most real-world datasets (resulting in short mixing times), the random walk rapidly mixes.
- **Result**: As the number of layers $l \to \infty$, the node representations converge to the graph's stationary distribution. Every node's feature vector becomes strictly proportional to its degree (or the square root of its degree, depending on the normalization), destroying all individual semantic identity.

References: Qimai Li, Zhichao Han, Xiao-Ming Wu: Deeper Insights Into Graph Convolutional Networks for Semi-Supervised Learning. AAAI 2018: 3538-3545

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Defining Over-smoothing (The Spectral View) 19

- **Concept:** Graph convolution acts as a **strict low-pass filter** on graph signals. Over-smoothing is the exponential attenuation of all high-frequency signals across the network.
- **The Graph Laplacian Spectrum:** Consider the normalized graph Laplacian

$$\tilde{L} = I - \tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$$

with eigenvalues $0 = \lambda_0 \le \lambda_1 \le \cdots \le \lambda_{n-1} \le 2$. The eigenvectors associated with larger eigenvalues represent “high-frequency” signals (values that change rapidly across edges).

- **Frequency Attenuation**: The standard GCN aggregation matrix $I - \tilde{L}$ has eigenvalues $1 - \lambda_i$. Multiplying the node feature matrix by this filter l times scales the projection of the signal on the i-th eigenvector by $(1 - \lambda_i)^l$.
- **Result**: For all $\lambda_i > 0$, the magnitude $|1 - \lambda_i| < 1$. Therefore, high-frequency components, which capture the critical differences between connected nodes needed to establish class boundaries, decay exponentially. As $l \to \infty$, only the zero-frequency (perfectly smooth) component survives.

References: Hoang NT, Takanori Maehara: Revisiting Graph Neural Networks: All We Have is Low-Pass Filters. CoRR abs/1905.09550 (2019)

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Quantifying Over-smoothing 20

- To evaluate mitigation strategies theoretically and empirically, over-smoothing is quantified using representation distance and energy metrics.
- **Metric 1: Dirichlet Energy**
  - Measures the total variation (smoothness) of the node embeddings $H^{(l)}$ across the edges of the graph.

$$E\left(H^{(l)}\right) = \frac{1}{2}\sum_{(u,v)\in\mathcal{E}} A_{uv}\left\|h_u^{(l)} - h_v^{(l)}\right\|^2 = \mathrm{Tr}\left(\left(H^{(l)}\right)^T \tilde{L} H^{(l)}\right)$$

  - An energy limit of $\lim_{l\to\infty} E\left(H^{(l)}\right) = 0$ indicates complete spatial collapse and severe over-smoothing.
- **Metric 2: Mean Average Distance (MAD)**
  - Calculates the average cosine distance between node representations to quantify representation similarity independently of feature magnitude.

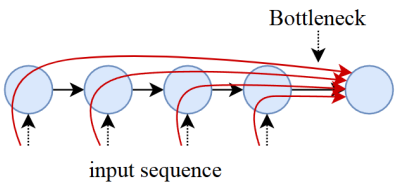
$$D\left(h_i, h_j\right) = 1 - \frac{h_i \cdot h_j}{\|h_i\|\|h_j\|}$$

  - A decreasing MAD score as layer depth increases provides a direct empirical measure that nodes are collapsing into a single point in the latent space.

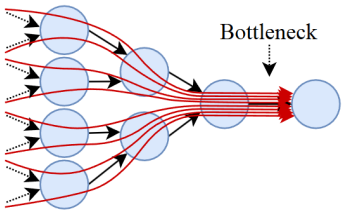
References:
- Chen Cai, Yusu Wang: A Note on Over-Smoothing for Graph Neural Networks.
- Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, Xu Sun: Measuring and Relieving the Over-Smoothing Problem for Graph Neural Networks from the Topological View. AAAI 2020: 3438-3445

Over-squashing 21

- **The Receptive Field Explosion**: In many real-world graphs (especially tree-like or scale-free graphs), the volume of a node's k-hop neighborhood grows exponentially with the radius k.
- **The Core Conflict**: To capture long-range interactions, a GNN requires multiple layers of message passing. However, all the information from this exponentially growing neighborhood must be compressed into a single, fixed-dimensional node feature vector $h_i^{(l)}$.
- **The Result (Over-squashing):** Crucial signals from distant nodes are **"squashed"** and lost during the successive aggregation steps. The GNN fails to propagate localized, long-range information.



(a) The bottleneck of RNN seq2seq models



(b) The bottleneck of graph neural networks

References: Uri Alon, Eran Yahav: On the Bottleneck of Graph Neural Networks and its Practical Implications. ICLR 2021

Over-squashing 22

Theoretical Definition & The Jacobian Matrix

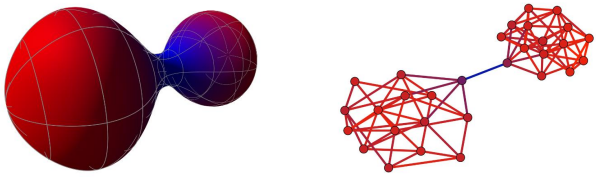
- How do we mathematically prove that information is not reaching its destination? We analyze the sensitivity of a node's output to a distant node's input.
- **The Jacobian Sensitivity**: Over-squashing is quantified by the norm of the Jacobian matrix of the final representation at node i with respect to the initial features of node j:

α and β: Upper bounds on the gradients of the update (ϕ) and message (ψ) functions.

$$\left|\frac{\partial h_i^{(r+1)}}{\partial x_s}\right| \leq (\alpha\beta)^{r+1}\left(\hat{A}^{r+1}\right)_{is}$$

Specific matrix entry isolating the topological influence between nodes i and s.

- **Exponential Decay**: If the communication path passes through a graph bottleneck, theoretical bounds show that this Jacobian norm decays exponentially with the distance $d(i,j)$ and the graph's mixing properties.
- f $\frac{\partial h_i^{(L)}}{\partial x_j} \approx 0$, the network cannot learn from the interaction between i and j.

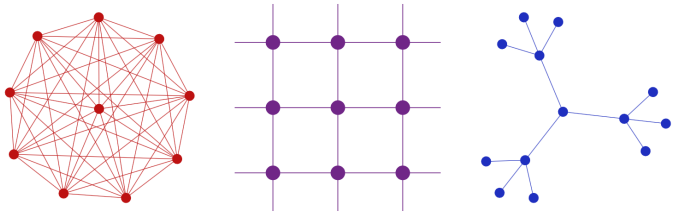


References: Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, Michael M. Bronstein: Understanding over-squashing and bottlenecks on graphs via curvature. ICLR 2022

Over-squashing 23

The Topological View (Graph Curvature)

- Over-squashing is fundamentally a problem of graph geometry.
- **Discrete Ricci Curvature**: Borrowed from differential geometry, edge curvature measures how neighborhoods of two connected nodes relate to each other.
  - **Positive Curvature (Cliques/Grids):** Neighborhoods overlap heavily. Information flows easily through multiple pathways.
  - **Negative Curvature (Trees/Bridges):** Neighborhoods fan out exponentially with no overlap. These edges act as structural bottlenecks where traffic jams occur, causing severe over-squashing.



(a) Clique (> 0) (b) Grid ($= 0$) (c) Tree (< 0)

References: Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, Michael M. Bronstein: Understanding over-squashing and bottlenecks on graphs via curvature. ICLR 2022

A tension: connectivity helps one, hurts the other 24

CHEEGER INEQUALITY (CHUNG, 1997)

$$\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2\lambda_2}$$

where $h(G)$ is the Cheeger constant (how narrow the graph's tightest cut is) and λ_2 the second-smallest eigenvalue of the normalized Laplacian, i.e., the spectral gap.

Small gap ↔ bottleneck.

Bottlenecked graph: small gap

Slow mixing: features stay distinct for many layers (little over-smoothing), but distant signals cannot get through (strong over-squashing).

Expander-like graph: large gap

Fast mixing: long-range signals arrive quickly (little over-squashing), but features converge after few layers (fast over-smoothing).

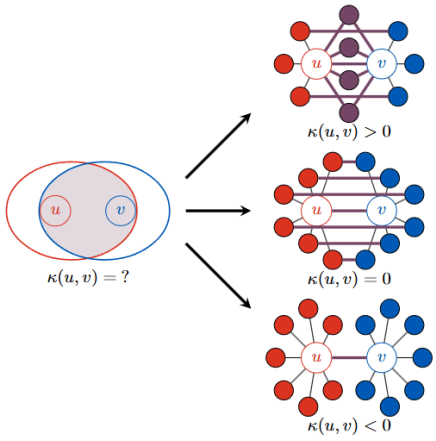
References: Chung, F. R. K. (1997). Spectral Graph Theory. American Mathematical Society.

A tension: connectivity helps one, hurts the other 25

Ollivier-Ricci Curvature on Graphs

- **Concept**: Adapts the notion of Ricci curvature to discrete spaces using optimal transport (Wasserstein distance).
- **Mechanism**: Evaluates the optimal ***transport cost*** $W_1(\mu_u, \mu_v)$ between the neighborhood distributions μ_u and μ_v of two adjacent nodes u and v.
- **Curvature Edge Metric:**

$$\kappa(u,v) = 1 - \frac{W_1(\mu_u, \mu_v)}{d(u,v)}$$



References: Khang Nguyen, Nong Minh Hieu, Vinh Duc Nguyen, Nhat Ho, Stanley J. Osher, Tan Minh Nguyen: Revisiting Over-smoothing and Over-squashing Using Ollivier-Ricci Curvature. ICML 2023: 25956-25979

A tension: connectivity helps one, hurts the other 26

Positive Curvature Induces Over-smoothing

- **Intuition:** Highly positive $\kappa(u,v)$ means N_u and N_v heavily coincide. Exchanging messages between them rapidly diminishes variance.
- **Structural Bound**: The curvature provides a lower bound on shared neighbors:

$$\frac{|N_u \cap N_v|}{\max(m,n)} \geq \kappa(u,v)$$

- **Feature Convergence**: Under standard MPNN update rules, the feature distance between nodes at layer $k+1$ is bounded:

$$\|X_u^{k+1} - X_v^{k+1}\| \leq \big(1-\kappa(u,v)\big)h\big(\kappa(u,v)\big)$$

  where $\lim_{x\to 1}(1-x)\,h(x) = 0$.

- **Conclusion**: Edges with $\kappa(u,v) \to 1$ force local representations to collapse, **accelerating over-smoothing**.

References: Khang Nguyen, Nong Minh Hieu, Vinh Duc Nguyen, Nhat Ho, Stanley J. Osher, Tan Minh Nguyen: Revisiting Over-smoothing and Over-squashing Using Ollivier-Ricci Curvature. ICML 2023: 25956-25979

A tension: connectivity helps one, hurts the other 27

Negative Curvature Induces Over-squashing

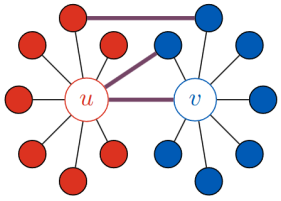
- **Intuition:** Very negative $\kappa(u,v)$ indicates a high optimal transport cost, signifying a lack of pathway edges connecting N_u and N_v.
- **Pathway Bound:** The number of pathway edges $|S|$ is strictly constrained by negative curvature:

$$|S| \leq \frac{n(\kappa(u,v)+2)}{2}$$

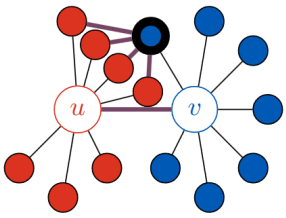
- **Vanishing Jacobian:** The sensitivity of node u at layer $k+2$ to a distant node q is bounded by a scaling term α:

$$\alpha \leq \frac{|S|+2}{\sum_{w\in\tilde{N}_v}(\deg(w)+1)}$$

- **Conclusion**: Negative curvature restricts $|S|$, driving $\alpha \to 0$. Information from distant nodes is structurally suppressed.



(a) Violet edges connecting $\tilde{N}_u \setminus \{v\}$ and $\tilde{N}_v \setminus \{u\}$ serve as information pathways.



(b) Edges may exacerbate the situation by creating new bottlenecks.

References: Khang Nguyen, Nong Minh Hieu, Vinh Duc Nguyen, Nhat Ho, Stanley J. Osher, Tan Minh Nguyen: Revisiting Over-smoothing and Over-squashing Using Ollivier-Ricci Curvature. ICML 2023: 25956-25979

Under-reaching

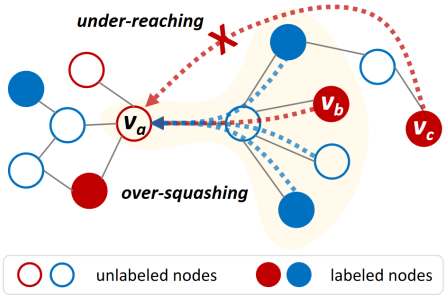
The opposite failure: a GNN with K layers sees only each node's K-hop neighbourhood, so information from farther nodes never arrives.



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Defining Under-reaching 29

- **The Receptive Field Limit:** A standard Graph Neural Network with K layers computes node representations by recursively aggregating information from K-hop neighborhoods.
- **The Strict Boundary**: Let $\mathcal{N}_K(v)$ be the set of nodes reachable from v in K or fewer steps. The K-th layer representation $h_v^{(K)}$ is strictly a function of the subgraph induced by $\mathcal{N}_K(v)$.
- **The Definition**: Under-reaching occurs when the structural or feature information required to correctly predict a target variable for node v resides at a node u, but the shortest path distance $d(u,v) > K$.
- **Mathematical Consequence**: The Jacobian sensitivity is identically zero: $\frac{\partial h_v^{(K)}}{\partial x_u} = 0$. The network is completely blind to the dependency.



References: Qingyun Sun, Jianxin Li, Haonan Yuan, Xingcheng Fu, Hao Peng, Cheng Ji, Qian Li, Philip S. Yu: Position-aware Structure Learning for Graph Topology-imbalance by Relieving Under-reaching and Over-squashing. CIKM 2022: 1848-1857

Theoretical Limits (Depth vs. Width) 30

- Can we solve under-reaching by making a shallow network infinitely wide (increasing hidden dimension size)?
- **Theoretical Bound**: *Loukas (2020)* proved that a message-passing GNN cannot compute basic global graph properties (like cycle detection, diameter, or shortest paths) unless the product of its depth and width is proportional to the graph size $\mathcal{O}(N)$.
- **Conclusion**: If depth K is treated as a small constant to prevent over-smoothing, no amount of node feature capacity (width) can overcome the geometric under-reaching problem. The GNN is fundamentally limited to local combinatorial properties.

References: Andreas Loukas: What graph neural networks cannot learn: depth vs width. ICLR 2020

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교

Task Dependency: When is it a problem? 31

- When Under-reaching is **NOT** a problem:
  - **Citation Networks / Social Networks**: High homophily. A node's immediate neighbors strongly predict its class. *(This is why 2-layer GCNs perform so well on Cora/Citeseer).*
- When Under-reaching is **FATAL**:
  - **Molecular Chemistry**: Predicting molecular properties where two distant functional groups on a polymer chain interact.
  - **Algorithmic Tasks**: Solving Traveling Salesperson Problem (TSP), Boolean Satisfiability Problem (SAT) problems, or finding the shortest path on a graph.
  - **Heterophilic Graphs**: Where similar nodes are structurally separated by disassortative connections.

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교

Recap: GCN is a low-pass filter, which is why depth over-smooths 6

$$\hat{A} = I - \tilde{L}_{\text{sym}} \Longrightarrow \hat{A}^K x = U\big(I - \tilde{\Lambda}\big)^K U^\top x, \quad g(\lambda) = (1-\lambda)^K$$

Filtering

$(1-\lambda)^K \approx 1$ for small λ and decays for larger λ. Self-loops shrink the spectrum, so high frequencies are damped too (Wu et al., 2019).

Over-smoothing

As $K \to \infty$, $\hat{A}^K x$ converges to the top eigenvector ($\propto \tilde{D}^{1/2}\mathbf{1}$): nodes in a component become indistinguishable up to degree (Li et al., 2018).

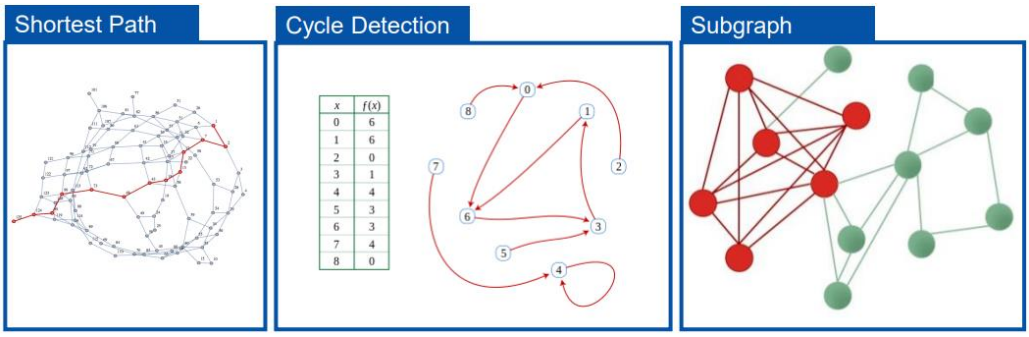
Consequence

GCN = denoising by local averaging. Great if labels are smooth (homophily); harmful if labels flip across edges (heterophily).

References:
- Wu, Felix, et al. "Simplifying graph convolutional networks." International conference on machine learning. Pmlr, 2019.
- Li, Qimai, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 32. No. 1. 2018.

The Power of Deeper DNNs 9

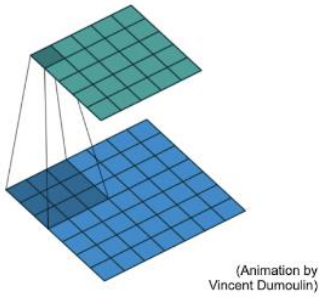
- Do GNNs need deeper structures to enable larger receptive fields, too?
- What limits the expressive power of GNNs?
  - The depth d.
  - The width w.
- GNNs significantly lose their power when capacity, dw, is restricted

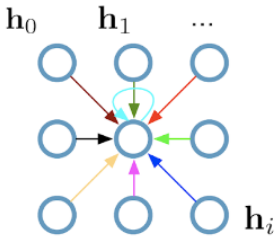


References: Andreas Loukas: What graph neural networks cannot learn: depth vs width. ICLR 2020

Recap: CNN vs. GCN 10

- Single CNN layer with 3x3 filter:





Update for a single pixel:

- Transform messages individually $\mathbf{W}_i\mathbf{h}_i$
- Add everything up $\sum_i \mathbf{W}_i\,\mathbf{h}_i$

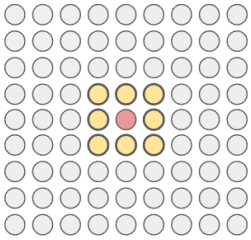
$\mathbf{h}_i \in \mathbb{R}^F$ are (hidden layer) activations of a pixel/node

Full update:

$$\mathbf{h}_4^{(l+1)} = \sigma\left(\mathbf{W}_0^{(l)}\mathbf{h}_0^{(l)} + \mathbf{W}_1^{(l)}\mathbf{h}_1^{(l)} + \cdots + \mathbf{W}_8^{(l)}\mathbf{h}_8^{(l)}\right)$$

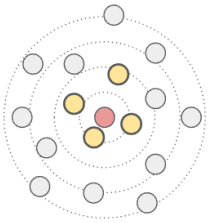
Recap: CNN vs. GCN 11

- The key differences between CNNs and GCNs is how the convolution is computed.
- For CNNs, we simply do a convolution with our neighbors on the grid with a given kernel size.
- On graphs, we need to define our neighbors first, and then do the convolution in the neighborhood. This is done by finding the k nearest neighbors (with respect to some distance metric).
- **So, where are the deep GCNs?**



$$\sigma\left(\sum_i w_i x_i + b\right)$$

Convolutional Neural Network (CNN)



$$\sigma\left(\sum_{u \in \mathcal{N}(v)} w\, x_u + w' x_v + b\right)$$

Graph Convolutional Network (GCN)

The depth paradox 12

Vision: deeper is usually better

Residual CNNs train reliably with 100+ layers, and accuracy generally improves with depth (He et al., 2016).

Graphs: deeper is often worse

Classic GCNs perform best with 2–3 layers; naively stacking more layers degrades accuracy (Kipf & Welling, 2017; Li, Han & Wu, 2018).

Residual connections alone are not enough

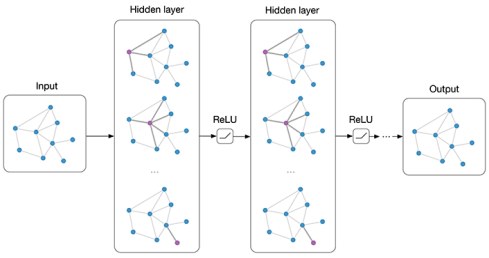
They ease optimization (DeepGCNs trained 56-layer GCNs with residual and dilated connections; Li et al., 2019), but they do not change what repeated propagation does to node features or to long-range signals.

Central question: what breaks when message-passing layers are stacked, and how can a node see far without the network being deep?

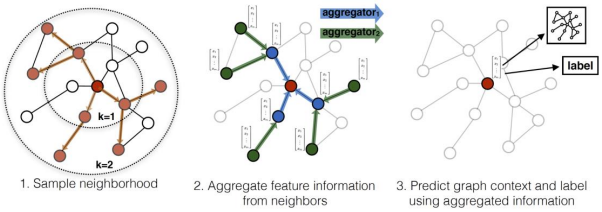
Why current GCNs cannot go deeper? 13

➢ Most SOTA GCN models are normally ***no deeper than 3 or 4*** layers.

GCN [1]



GraphSage [2]



Read more:

- GCN: https://tkipf.github.io/graph-convolutional-networks/
- GraphSage: https://snap.stanford.edu/graphsage/

References:
[1] Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017
[2] William L. Hamilton, Zhitao Ying, Jure Leskovec: Inductive Representation Learning on Large Graphs. NIPS 2017: 1024-1034

네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Optimization Problem

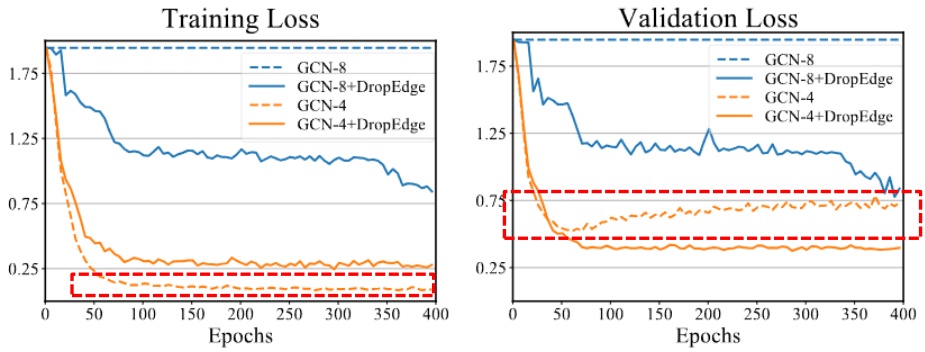
Unlike over-smoothing, these failures are not specific to graphs: overfitting and vanishing gradients limit every deep network.



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Why current GCNs cannot go deeper? 33

- **Overfitting**: GNNs suffer from Overfitting
  - Too many parameters are established but only few of data points are provided



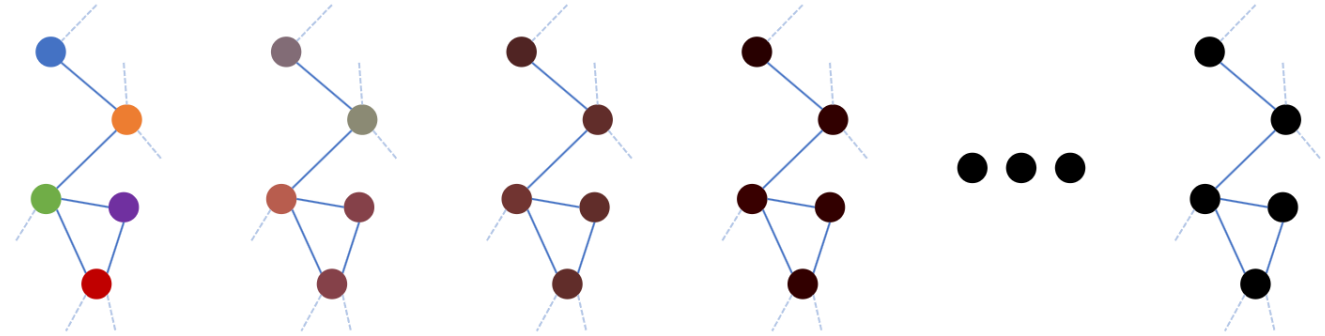
References: Yu Rong, Wenbing Huang, Tingyang Xu, Junzhou Huang: DropEdge: Towards Deep Graph Convolutional Networks on Node Classification. ICLR 2020

Why current GCNs cannot go deeper? 34

- **Vanishing Gradient**:
  - l-layers gradients

$$\frac{dH_{l+1}}{dH_l} \cdot \frac{dH_l}{dH_{l-1}} \cdot \ldots \cdot \frac{dH_0}{dW_0} \leq (s_l \lambda_{max}) \cdot (s_{l-1} \lambda_{max}) \cdot \ldots \cdot \frac{dH_0}{dW_0}$$

  - The gradients vanish as the model go deeper because $s_{1..l} \lambda_{m+1} < 1$



References: Jure Leskovec, Stanford CS224W: Machine Learning with Graph

Remedy Families

Architectural, training, and graph-level techniques that let GNNs go deeper.



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Solutions for training Deeper GNNs 36

Remedy family	Representative models	Main target
Skip connections	DeepGCNs (2019), GCNII (2020), JKNet (2018)	Optimization, over-smoothing
Normalization	PairNorm (2020)	Over-smoothing
Regularization	Dropout, DropEdge (2020), DropNode (2020)	Overfitting
Graph rewiring	SDRF (2022), FoSR (2023), BORF (2023), DIGL (2019)	Over-squashing
Global aggregation	Virtual node (2017), GraphGPS (2022)	Over-squashing, under-reaching
Multi-hop / multi-scale aggregation	MixHop (2019), APPNP (2019), GPR-GNN (2021), JKNet (2018)	Under-reaching, over-smoothing

Recap: message passing and its hidden assumption

4

$$h_v^{(\ell+1)} = \sigma\left(W^{(\ell)} \cdot \mathrm{AGG}\left(\left\{h_u^{(\ell)} : u \in \mathcal{N}(v) \cup \{v\}\right\}\right)\right)$$

$$H^{(\ell+1)} = \sigma\left(\hat{A} H^{(\ell)} W^{(\ell)}\right), \qquad \hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$$

What one layer does

Each node becomes a degree-normalized **average** of itself and its neighbors, followed by a shared linear map and a nonlinearity. After K layers a node sees its K-hop computation graph.

The implicit assumption

Averaging helps only if neighbors are **informative about**, and usually **similar to**, the center node. This is the **homophily** (smoothness) assumption.

References:
- Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Recap: Dirichlet energy

5

$$\mathcal{E}(X) = \mathrm{tr}(X^\top L X) = \frac{1}{2}\sum_{i,j} A_{ij} \|x_i - x_j\|_2^2, \quad L = D - A$$

$$L = U \Lambda U^\top, \quad \hat{x} = U^\top x \Longrightarrow \mathcal{E}(x) = \sum_k \lambda_k \hat{x}_k^2$$

Energy

Sums squared differences across edges. Small energy ⇒ connected nodes carry similar values.

Frequency

Eigenvalue λ_k is the frequency of eigenvector u_k: $\lambda \approx 0$ is near-constant on clusters; large λ flips sign across edges.

Smooth signal

A signal is smooth iff its spectral energy concentrates on **low** frequencies.

References:
- Cai, Chen, and Yusu Wang. "A note on over-smoothing for graph neural networks." arXiv preprint arXiv:2006.13318 (2020).
- Wu, Felix, et al. "Simplifying graph convolutional networks." International conference on machine learning. Pmlr, 2019.
- Li, Qimai, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 32. No. 1. 2018.

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

GCNII: GCN via Initial residual and Identity mapping 47

➢ **Idea**: Introduces two key modifications to solve the over-smoothing problem.

- **Initial Residual Connection**: Adds a skip connection from the input layer to each subsequent layer, ensuring the original features of nodes are retained through multiple layers.
- **Identity Mapping**: Modifies the weight matrix by adding an identity matrix, which helps retain the expressive power of deep networks by reducing overfitting and feature degradation.

References: Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li: Simple and Deep Graph Convolutional Networks. ICML 2020: 1725-1735

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

GCNII: GCN via Initial residual and Identity mapping 48

➢ **GCNII** modifies the standard message passing architecture by decoupling feature aggregation and nonlinear transformation.

➢ **GCNII Layer Formulation:**

$$H^{(\ell+1)} = \sigma\left(\left((1-\alpha_\ell)\tilde{P}H^{(\ell)} + \alpha_\ell H^{(0)}\right)\left((1-\beta_\ell)I_n + \beta_\ell W^{(\ell)}\right)\right)$$

Where:

- $\tilde{P}H^{(\ell)}$: The standard smoothed graph representation.
- $H^{(0)}$: The initial node features (output of the first MLP layer).
- I_n: The identity matrix.
- $W^{(\ell)}$: The learnable weight matrix for layer ℓ.
- α_ℓ, β_ℓ: Hyperparameters controlling the decay and influence of initial features and weight transformations.

References: Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li: Simple and Deep Graph Convolutional Networks. ICML 2020: 1725-1735

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

GCNII: GCN via Initial residual and Identity mapping 49

➢ Vanilla GCN

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\tilde{\mathbf{P}}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)}\right)$$

➢ GCN + **Initial residual**

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\left((1-\alpha_\ell)\tilde{\mathbf{P}}\mathbf{H}^{(\ell)} + \alpha_\ell\mathbf{H}^{(0)}\right)\mathbf{W}^{(\ell)}\right)$$

➢ **GCNII**: GCN + Initial residual **+ Identity mapping**

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\left((1-\alpha_\ell)\tilde{\mathbf{P}}\mathbf{H}^{(\ell)} + \alpha_\ell\mathbf{H}^{(0)}\right)\left((1-\beta_\ell)\mathbf{I}_n + \beta_\ell\mathbf{W}^{(\ell)}\right)\right)$$

References: Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li: Simple and Deep Graph Convolutional Networks. ICML 2020: 1725-1735

GCNII: GCN via Initial residual and Identity mapping 50

Why GCNII Prevents Over-Smoothing

1. **Initial Residual Connection**: $(1-\alpha_\ell)\tilde{P}H^{(\ell)} + \alpha_\ell H^{(0)}$

   Theory: Unlike standard residual connections (e.g., ResNet) that only carry over the previous layer $H^{(\ell)}$, GCNII maintains a persistent skip connection to the initial input $H^{(0)}$.

   Effect: It ensures that even as depth approaches infinity, the final distribution carries information from the original input features, strictly preventing the model from collapsing into the graph's stationary vector.

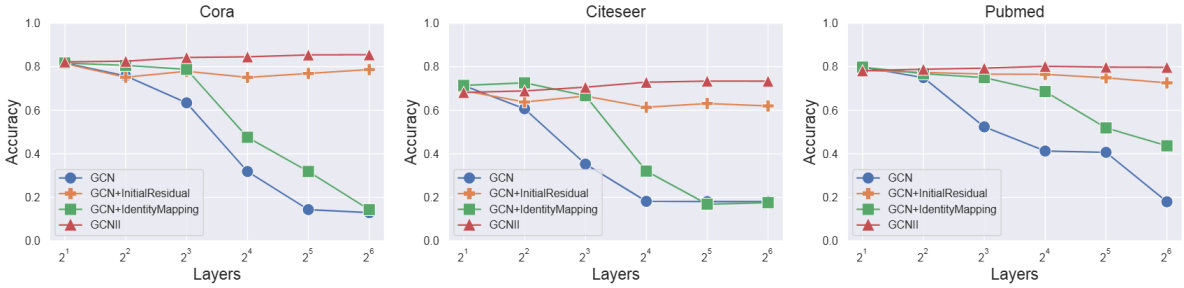
2. **Identity Mapping**: $(1-\beta_\ell)I_n + \beta_\ell W^{(\ell)}$

   Theory: Adds an identity matrix to the weight matrix. As layers stack, frequent multiplication of unregularized weight matrices disrupts the feature distribution.

   Effect: Ensures that deep models maintain at least the same performance as shallow models. β_ℓ is decayed adaptively ($\beta_\ell \approx \log(\lambda/\ell + 1)$) to regularize the transformation.

References: Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li: Simple and Deep Graph Convolutional Networks. ICML 2020: 1725-1735

GCNII: GCN via Initial residual and Identity mapping 51



Ablation study on initial residual and identity mapping.

References: Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, Yaliang Li: Simple and Deep Graph Convolutional Networks. ICML 2020: 1725-1735

Skip connections

Shortcuts that carry earlier-layer features forward: residual, dense, initial residual and jumping knowledge.



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

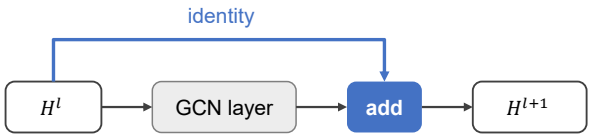
Skip connections 38

	Source of the skip	Merge	Update rule	Used in
Residual	Previous layer $H^{(\ell)}$	Add	$H^{(\ell+1)} = \sigma\left(\tilde{P}H^{(\ell)}W^{(\ell)}\right) + H^{(\ell)}$	ResGCN (DeepGCNs)
Dense	All previous layers	Concat	$H^{(\ell+1)} = \left[F\left(H^{(\ell)}\right), H^{(\ell)}, \ldots, H^{(0)}\right]$	DenseGCN
Initial residual	Input layer $H^{(0)}$	Convex mix	$H^{(\ell+1)} = \sigma\left(\left((1-\alpha)\tilde{P}H^{(\ell)} + \alpha H^{(0)}\right)W^{(\ell)}\right)$	GCNII
Jumping knowledge	All layers, output only	Concat / Max / LSTM	$h_v^{final} = \mathrm{AGG}\left(h_v^{(1)}, \ldots, h_v^{(L)}\right)$	JKNet

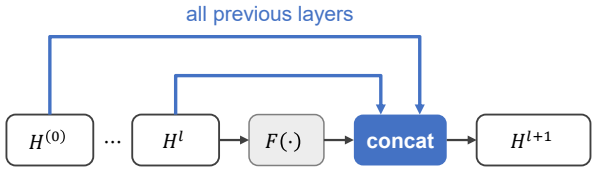
네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Skip connection architectures 39

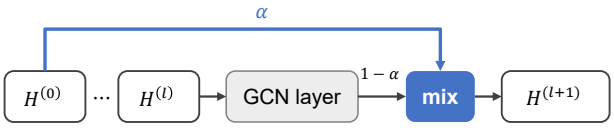
Residual ResGCN



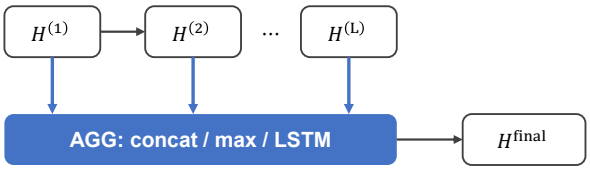
Dense DenseGCN



Initial residual GCNII



Jumping knowledge JKNet



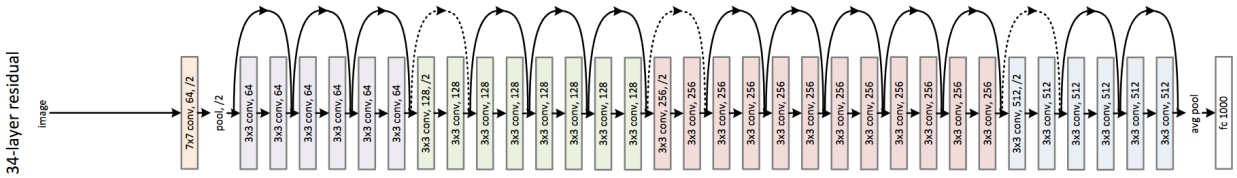
DeepGCNs framework 41

Residual Graph Connections (ResGCN)

- **Concept:** Adapted from *ResNet*. Instead of learning a direct mapping, the network learns a residual mapping F that is added to the identity of the input graph.
- **Mathematical Formulation:**

$$G_{l+1} = H(G_l, W_l) = F(G_l, W_l) + G_l = G_{l+1}^{res} + G_l$$

- **Theoretical Impact:**
  - The vertex-wise addition $(+G_l)$ acts as an identity skip connection.
  - This provides a direct path for gradient flow during back-propagation, mitigating vanishing gradients and allowing the graph convolution F to focus strictly on learning the residual structural information (G_{l+1}^{res}).



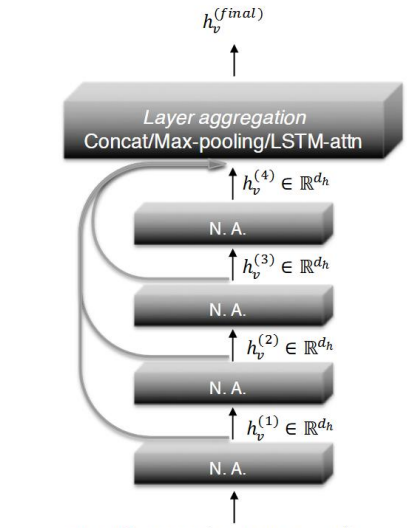
Residual network with 34 parameter layers

References: Guohao Li, Matthias Müller, Ali K. Thabet, Bernard Ghanem: DeepGCNs: Can GCNs Go As Deep As CNNs? ICCV 2019: 9266-9275

JK-Net networks 53

- **Idea:**
  1. Makes each layer increase the size of the influence distribution by aggregating neighborhoods from the previous layers.
  2. Combines some of the previous layers' representations independently for each node.

- At the last layer, for each node, carefully select from all of those intermediate representations (which "jump" to the last layer).
- If this is done independently for each node, then the model can adapt the effective neighborhood size for each node as needed, resulting in exactly the desired adaptivity.



N.A.stands for neighborhood aggregation.

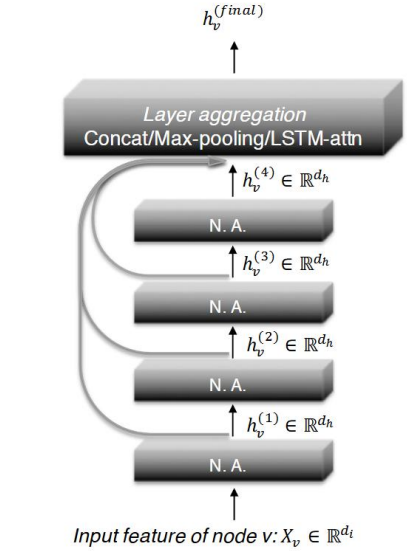
References: Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, Stefanie Jegelka: Representation Learning on Graphs with Jumping Knowledge Networks. ICML 2018: 5449-5458

JK networks: Aggregation Mechanisms with Skip Connections 54

- **Concatenation**:

$$\left[h_v^{(1)}, \ldots, h_v^{(k)}\right]$$

- **Max-pooling**:
  - Select the most informative layer for each feature coordinate.
- **LSTM-Attention**:
  - Input $h_v^{(1)}, \ldots, h_v^{(k)}$ into a bi-directional LSTM to generate forward and backward $f_v^{(l)}$ and $b_v^{(l)}$ for each layer.



N.A.stands for neighborhood aggregation.

References: Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, Stefanie Jegelka: Representation Learning on Graphs with Jumping Knowledge Networks. ICML 2018: 5449-5458

Homophily metric families 9

Family	Metrics	Uses features?	Linear? (*)
Graph-label consistency	$H_{edge}, H_{node}, H_{class}, H_{adj}$	no	linear
Similarity-based	H_{GE}, H_{agg}	yes	linear
Neighborhood informativeness	$LI, H_{neighbor}$	no	nonlinear
Hypothesis testing	CPM	yes	nonlinear
Heterogeneous (meta-path)	$MSHR, HHR, MLH, MDE$	(**)	linear

(*) Linear describes simple, direct relationships that can be separated by a straight line, whereas non-linear describes complex, interacting patterns requiring flexible, curved decision boundaries.
(**): Meta-Path Subgraph Homophily Ratio (MSHR), Heterogeneous Graph Homophily Ratio (HHR) and Meta-Path Based Label Homophily (MLH) **do not use node features**. While Meta-Path Based Dirichlet Energy (MDE) **does use features**.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

Classifier-based Performance Metric (CPM) 10

- **Core Concept:** CPM evaluates the true utility of graph structural information by measuring the node distinguishability of aggregated features (H) versus original features (X). Instead of defining a static formula, it uses the p-value of hypothesis testing as the performance metric, successfully capturing nonlinear, feature-dependent information without requiring iterative GNN training.
- **Methodology**
  - **Feature Extraction**: Compute original features X and aggregated features $H = \hat{A}X$ for a subset of sampled nodes (e.g., 500 nodes, 60/40 train/test split).
  - **Classification Strategy**: Feed X and H into a non-iterative classifier, such as *Gaussian Naïve Bayes (GNB)* or *Kernel Regression (KR)*.
  - **Sampling**: Repeat the process multiple times (e.g., 100 iterations) to generate distributions of prediction accuracy for both X and H.

References: Luan, Sitao, et al. "When do graph neural networks help with node classification? investigating the homophily principle on node distinguishability." Advances in Neural Information Processing Systems 36 (2023): 28748-28760.

Classifier-based Performance Metric (CPM) 11

- **Hypothesis Formulation:** The statistical test is defined to compare the accuracy distributions:

$$H_0: \mathrm{Acc}\big(\mathrm{Classifier}(H)\big) \geq \mathrm{Acc}\big(\mathrm{Classifier}(X)\big)$$
$$H_1: \mathrm{Acc}\big(\mathrm{Classifier}(H)\big) < \mathrm{Acc}\big(\mathrm{Classifier}(X)\big)$$

- **Theoretical Interpretation**
  - **The Metric:** The resulting p-value acts as a statistical threshold indicating whether structural aggregation provides a significant classification advantage.
  - **Significance:** Rejecting H_0 *(a low p-value)* suggests that aggregating messages over the graph structure is detrimental compared to using raw features alone, effectively identifying malignant heterophily.

References: Luan, Sitao, et al. "When do graph neural networks help with node classification? investigating the homophily principle on node distinguishability." Advances in Neural Information Processing Systems 36 (2023): 28748-28760.

Local and Global Consistency Metrics 12

Edge Homophily (h_{edge}):

- **Definition**: The global fraction of edges in the network that connect nodes of the same class.
- **Equation**:

$$h_{edge} = \frac{|\{(u,v) \in E : y_u = y_v\}|}{|E|}$$

- **Theoretical Implication**: While intuitive, h_{edge} strictly evaluates graph-wide consistency. It is highly ***sensitive to graph density*** and is easily ***skewed by a few high-degree hubs***, often masking local neighborhood heterophily.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

Local and Global Consistency Metrics 13

Node Homophily (h_{node}):

- **Definition**: The average proportion of intra-class neighbors, computed on a per-node basis and then averaged across all nodes.
- **Equation**:

$$h_{node} = \frac{1}{|V|}\sum_{v \in V}\frac{|\{u \in \mathcal{N}(v): y_u = y_v\}|}{d(v)}$$

- **Theoretical Implication**: By normalizing at the node level (dividing by $d(v)$), this metric treats the local context of every node equally. It ***prevents high-degree nodes from dominating*** the metric, offering a more balanced view of how homophilic the graph is from the perspective of an average node.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Adjusting for Class Imbalance and Density 14

Class Homophily (h_{class}):

- **Definition**: Accounts for severe class imbalances by evaluating homophily within each specific class, subtracting baseline random chance, and averaging the results.
- **Equation**:

$$h_{class} = \frac{1}{C}\sum_{k=1}^{C}\max\left(0, h_k - \frac{|V_k|}{|V|}\right)$$

where $h_k = \frac{|\{(u,v)\in E: y_u = y_v = k\}|}{|\{(u,v)\in E: y_u = k\}|}$

- **Theoretical Implication**: In a highly skewed dataset, a node might connect to the majority class purely by random chance, artificially inflating standard edge homophily. Subtracting the prior $\frac{|V_k|}{|V|}$ isolates genuine class-driven affinity.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Adjusting for Class Imbalance and Density 15

Adjusted Homophily (h_{adj}):

- **Definition**: Normalizes the global edge homophily by factoring in the expected homophily of a completely random graph that shares the same degree distribution.
- **Equation**:

$$h_{adj} = \frac{h_{edge} - \sum_{k=1}^{C} p_k^2}{1 - \sum_{k=1}^{C} p_k^2}$$

where $p_k = \frac{\sum_{v \in V_k} d(v)}{2|E|}$ represents the degree-weighted probability of connecting to class k.

- **Theoretical Implication**: Ensures standard bounds. $h_{adj} = 1$ indicates perfect homophily, and $h_{adj} = 0$ implies the edge connectivity is entirely independent of the node labels *(purely random mixing)*.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Information-Theoretic and Structural Metrics 16

Label Informativeness (LI):

- **Definition**: Quantifies heterophily through mutual information, capturing predictable cross-class patterns that standard consistency metrics misinterpret as noise.
- **Equation**:

$$LI = \frac{I(Y_u; Y_v)}{H(Y_u)}$$

- **Theoretical Implication**: Evaluates how much knowing the label of a neighbor Y_v reduces the entropy (uncertainty) $H(Y_u)$ of a target node's label Y_u. If class A always connects to class B, edge homophily is 0, but Label Informativeness is perfectly high (1.0), correctly signaling that the graph is highly structured and learnable.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Information-Theoretic and Structural Metrics 17

Structural Homophily (h_S):

- **Definition**: Shifts the measurement from raw label matching to the consistency of topological contexts among nodes of the same class.
- **Equation**:

$$h_S = \frac{1}{C}\sum_{c=1}^{C}\left(1 - \frac{\sigma_c}{\sigma_{max}}\right)$$

where σ_c is the standard deviation of structural features *(like neighborhood label distributions)* within class c.

- **Theoretical Implication**: Even in fiercely heterophilic datasets, nodes of the same class often exhibit identical types of neighborhoods. High structural homophily indicates that high-order neighbor mixing or structural encoding will allow GNNs to easily distinguish between classes despite immediate label clashes.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

Homophily vs. heterophily

When does “neighbors look alike” fail, and what then?



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Recap: message passing and its hidden assumption 4

$$h_v^{(\ell+1)} = \sigma\left(W^{(\ell)} \cdot \mathrm{AGG}\left(\left\{h_u^{(\ell)} : u \in \mathcal{N}(v) \cup \{v\}\right\}\right)\right)$$

$$H^{(\ell+1)} = \sigma\left(\hat{A}H^{(\ell)}W^{(\ell)}\right), \qquad \hat{A} = \tilde{D}^{-1/2}(A+I)\tilde{D}^{-1/2}$$

What one layer does

Each node becomes a degree-normalized **average** of itself and its neighbors, followed by a shared linear map and a nonlinearity. After K layers a node sees its K-hop computation graph.

The implicit assumption

Averaging helps only if neighbors are **informative about**, and usually **similar to**, the center node. This is the **homophily** (smoothness) assumption.

References:
- Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Recap: Dirichlet energy 5

$$\mathcal{E}(X) = \mathrm{tr}(X^\top L X) = \frac{1}{2}\sum_{i,j} A_{ij}\,\|x_i - x_j\|_2^2, \quad L = D - A$$

$$L = U\Lambda U^\top, \quad \hat{x} = U^\top x \Longrightarrow \mathcal{E}(x) = \sum_k \lambda_k\,\hat{x}_k^2$$

Energy

Sums squared differences across edges. Small energy ⇒ connected nodes carry similar values.

Frequency

Eigenvalue λ_k is the frequency of eigenvector u_k: $\lambda \approx 0$ is near-constant on clusters; large λ flips sign across edges.

Smooth signal

A signal is smooth iff its spectral energy concentrates on **low** frequencies.

References:
- Cai, Chen, and Yusu Wang. "A note on over-smoothing for graph neural networks." arXiv preprint arXiv:2006.13318 (2020).
- Wu, Felix, et al. "Simplifying graph convolutional networks." International conference on machine learning. Pmlr, 2019.
- Li, Qimai, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 32. No. 1. 2018.

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Recap: GCN is a low-pass filter, which is why depth over-smooths 6

$$\hat{A} = I - \tilde{L}_{\mathrm{sym}} \Longrightarrow \hat{A}^K x = U\big(I - \tilde{\Lambda}\big)^K U^\top x, \quad g(\lambda) = (1-\lambda)^K$$

Filtering

$(1-\lambda)^K \approx 1$ for small λ and decays for larger λ. Self-loops shrink the spectrum, so high frequencies are damped too (Wu et al., 2019).

Over-smoothing

As $K \to \infty$, $\hat{A}^K x$ converges to the top eigenvector ($\propto \tilde{D}^{1/2}\mathbf{1}$): nodes in a component become indistinguishable up to degree (Li et al., 2018).

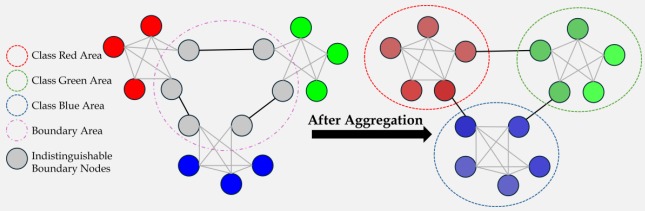
Consequence

GCN = denoising by local averaging. Great if labels are smooth (homophily); harmful if labels flip across edges (heterophily).

References:
- Wu, Felix, et al. "Simplifying graph convolutional networks." International conference on machine learning. Pmlr, 2019.
- Li, Qimai, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 32. No. 1. 2018.

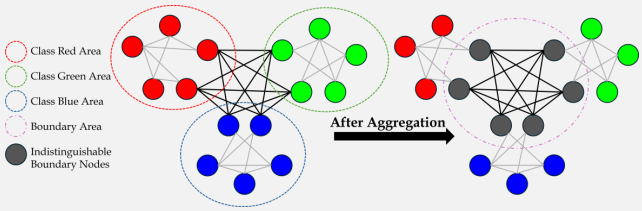
네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Two regimes of “who links to whom” 7



Homophily

“Birds of a feather”: citations, co-purchases, friendships. Neighbors share labels.



Heterophily

Fraudster ↔ victim, buyer ↔ seller, student ↔ faculty page, protein ↔ binding partner.

References:
- Thomas N. Kipf, Max Welling: Semi-Supervised Classification with Graph Convolutional Networks. ICLR (Poster) 2017
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

Homophily and Heterophily in Real-World Graphs 8

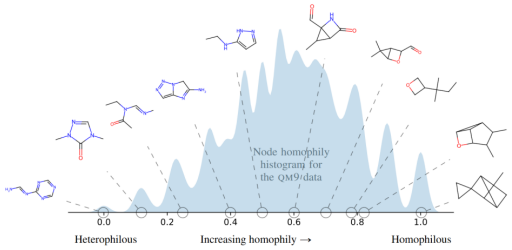


Figure: Homophily distribution of molecules in QM9.

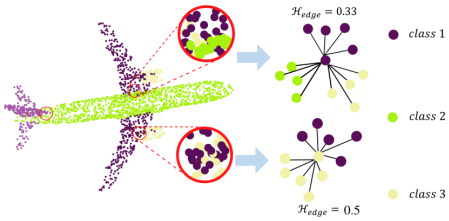


Figure: Local heterophilic structures in point-cloud segmentation. The red circle enlarges the segmentation boundary.

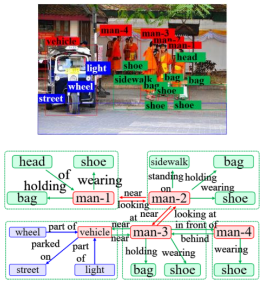


Figure: Scene graph generation.

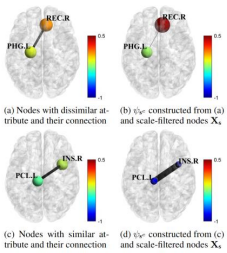
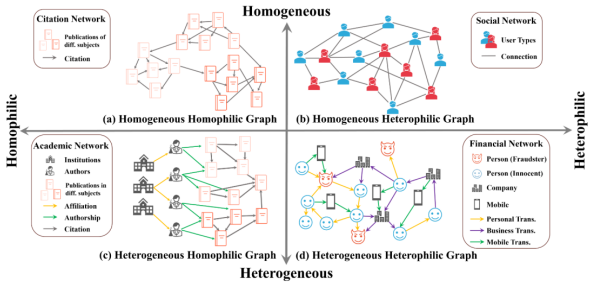


Figure: In brain networks, heterophily (a) and homophily (c) coexist.

References: LoG2024 Tutorial. (2024). Google.Com. https://sites.google.com/mila.quebec/heterophilic-graph-learning/home

Before we continue to *Homophily Metrics on Heterogeneous Graphs* 18

Feature	Heterogeneity	Heterophily
What it describes	Diversity of node and edge types	Similarity of labels/features between connected nodes
Opposite concept	Homogeneity (single node/edge type)	Homophily (connected nodes share similar labels/features)
Impact on GNNs	Standard GNNs fail because they assume a single uniform message-passing space.	Standard GNNs fail because neighborhood aggregation mixes dissimilar/conflicting labels.
Example	Author-Paper-Venue academic graph	Fraudster-Victim transaction graph



References: Lin, Junhong, et al. "When heterophily meets heterogeneity: Challenges and a new large-scale graph benchmark." Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2. 2025.

Before we continue to *Homophily Metrics on Heterogeneous Graphs* 19

A Heterogeneous Graph (or HIN) is defined as a tuple:

$$G = (V, E, \mathcal{T}_v, \mathcal{T}_e, \phi, \psi)$$

Where:

- V is the set of nodes and E is the set of edges.
- $\mathcal{T}_v$ is the set of predefined node types.
- $\mathcal{T}_e$ is the set of predefined edge (relation) types.
- $\phi: V \to \mathcal{T}_v$ is a node-type mapping function that assigns each node to a specific type.
- $\psi: E \to \mathcal{T}_e$ is an edge-type mapping function that assigns each edge to a specific relation type.

The Mathematical Constraint for Heterogeneity:

A graph is mathematically heterogeneous if and only if:

$$|\mathcal{T}_v| + |\mathcal{T}_e| > 2$$

(If $|\mathcal{T}_v| = 1$ and $|\mathcal{T}_e| = 1$, the graph is **strictly homogeneous**).

References: https://graph-neural-networks.github.io/static/file/chapter16.pdf

Heterogeneous Graph Homophily Ratio (HHR) 20

Meta-Path Subgraph Homophily Ratio (MSHR):

- **Definition**: Measures the edge-label consistency exclusively within the subgraph induced by a specific meta-path P.
- **Equation**:

$$H(\mathcal{G}_P) = \frac{\sum_{(v_i,v_j)\in\mathcal{E}_P} \mathbf{1}\left(y_i = y_j\right)}{|\mathcal{E}_P|}$$

where $\mathbf{1}(\cdot)$ is the indicator function, y_i is the label of node v_i, and $|\mathcal{E}_P|$ is the total number of edges in the meta-path induced subgraph $\mathcal{G}_P$.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Meta-Path Based Label Homophily and Dirichlet Energy 21

Meta-Path Based Label Homophily (MLH):

- **Definition**: Averages the homophily ratios across all possible length-2 meta-paths for a specific target node type A.
- **Equation**:

$$\mathrm{MLH}_A(\mathcal{G}) = \frac{1}{|\mathcal{P}_A|}\sum_{P\in\mathcal{P}_A} H\left(\mathcal{G}_P\right)$$

where $\mathcal{P}_A = \{AA_iA \mid A_i \in \mathcal{A}\}$ represents the set of all possible length-2 meta-paths starting from node type A, passing through any node type A_i, and ending back at node type A.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Meta-Path Based Label Homophily and Dirichlet Energy 22

Meta-Path Based Dirichlet Energy (MDE):

- **Definition**: Evaluates feature-level or representation-level smoothness across the same length-2 meta-paths using Dirichlet Energy E_D.
- **Equation**:

$$\mathrm{MDE}_A(\mathcal{G}) = \frac{1}{|\mathcal{P}_A|} \sum_{P \in \mathcal{P}_A} E_D\left(\mathcal{G}_P\right)$$

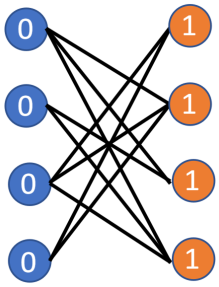
- **Theoretical Implication**: While MLH focuses on discrete labels, MDE captures the continuity of node features over the heterogeneous topology. A larger $\mathrm{MLH}_A(\mathcal{G})$ or a smaller $\mathrm{MDE}_A(\mathcal{G})$ implies stronger homophily for the target node type A. This framework allows researchers to evaluate whether the continuous features align with the discrete label homophily across complex relational structures.

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

Diagnosis – Revising the Assumption 28

Is homophily necessary? Good vs. bad heterophily

- **The Toy Example**:



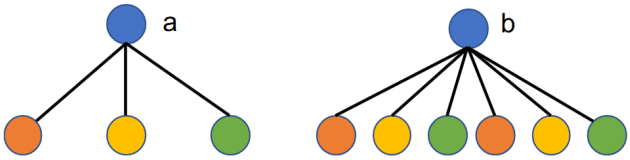
- **Setup**: Blue nodes (feature 0) only connect to Orange nodes (feature 1), and vice versa. The edge homophily ratio is 0.
- **GCN Operation**: After one GCN aggregation step, every Blue node averages its neighbors (all 1s) and becomes 1. Every Orange node averages its neighbors (all 0s) and becomes 0.
- **Result**: Despite absolute heterophily, the representations remain perfectly distinguishable. GCN achieves 100% accuracy because the features of nodes with the same label are “corrupted in the exact same way.”

References: Yao Ma, Xiaorui Liu, Neil Shah, Jiliang Tang: Is Homophily a Necessity for Graph Neural Networks? ICLR 2022

Diagnosis – Revising the Assumption

When does GCN learn similar embeddings for same-label nodes?

- **The Toy Example**:



- This figure shows two distinct nodes (a and b) of the same class that share the ***exact same neighborhood distribution***.
- **Key Insight**: Strong homophily is just one specific scenario that creates consistent neighborhood distributions.
- **The General Rule**: GCNs will succeed as long as nodes of the same label share similar neighborhood patterns, even if those neighbors belong to different classes. This represents **"good heterophily."**

References: Yao Ma, Xiaorui Liu, Neil Shah, Jiliang Tang: Is Homophily a Necessity for Graph Neural Networks? ICLR 2022

Beyond Local Neighborhoods

Under heterophily, same-class nodes are often not adjacent.
Reach them through higher-order hops, a latent space, or paths.



네트워크 과학 연구실 NETWORK SCIENCE LAB
가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

The H2GCN Framework 52

Theoretical Formulation of GNNs

The Local Markov Property in GNNs

- From a probabilistic perspective, GNN models assume the class label y_v depends only on the ego-feature x_v and neighbor-features $\{x_u : u \in \mathcal{N}(v)\}$.
- The iterative representation learning is defined as:

$$r_v^{(k)} = f\left(r_v^{(k-1)}, \left\{r_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)$$

$$r_v^{(0)} = x_v$$

$$y_v = \arg\max\left\{\text{softmax}\left(r_v^{(K)} W\right)\right\}$$

References: Zhu, Yan, Zhao, Heimann, Akoglu & Koutra, "Beyond Homophily in Graph Neural Networks", NeurIPS 2020

네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

The H2GCN Framework

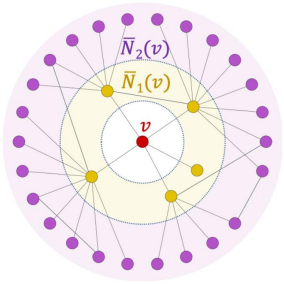
53

Theoretical Formulation of GNNs

The Local Markov Property in GNNs

- **The Bottleneck:**

  If the embedding function f relies on simple neighborhood averaging (like standard GCNs), a heterophilous neighborhood will result in representations that are completely uninformative for the ego node v.



References: Zhu, Yan, Zhao, Heimann, Akoglu & Koutra, "Beyond Homophily in Graph Neural Networks", NeurIPS 2020

The H2GCN Framework

54

Design 1 (D1) - Ego- and Neighbor-Embedding Separation

Preserving Node Identity

Under strong heterophily, the features of a node's neighbors are fundamentally different from the node itself. Forcing them into the same embedding space prematurely degrades performance.

Theoretical Solution: Keep the ego-representation mathematically distinct from the aggregated neighbor-representation.

Instead of a coupled update $r_v^{(k)} = \sigma\left(W \sum_{u \in \mathcal{N}(v) \cup \{v\}} r_u^{(k-1)}\right)$, D1 enforces separation:

$$r_v^{(k)} = f_{\text{ego}}\left(r_v^{(k-1)}\right) + f_{\text{neighbor}}\left(\left\{r_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)$$

References: Zhu, Yan, Zhao, Heimann, Akoglu & Koutra, "Beyond Homophily in Graph Neural Networks", NeurIPS 2020

The H2GCN Framework 55

Design 2 (D2) - Higher-Order Neighborhoods

Finding Homophily in Heterophily

In networks with strong heterophily, 1-hop neighbors ($N_1(v)$) are strictly dissimilar. However, 2-hop neighbors ($N_2(v)$) often exhibit homophily.

***Proof of Concept**: In a purely bipartite graph (extreme heterophily, $h = 0$), a node v and its 1-hop neighbors always belong to different classes. But any node $u \in N_2(v)$ is guaranteed to share the same class as v.*

Theoretical Solution: Explicitly aggregate over higher-order neighborhoods.

$$r_{v,\text{high-order}}^{(k)} = \text{AGG}\left(\left\{r_u^{(k-1)} : u \in N_1(v)\right\}, \left\{r_u^{(k-1)} : u \in N_2(v)\right\}, \ldots\right)$$

References: Zhu, Yan, Zhao, Heimann, Akoglu & Koutra, "Beyond Homophily in Graph Neural Networks", NeurIPS 2020

The H2GCN Framework 56

Synthesizing D1, D2, and D3

Let $S_1 = D^{-1/2}AD^{-1/2}$ and $S_2 = D^{-1/2}A^2D^{-1/2}$ (normalized 1-hop and 2-hop adjacencies).

The H2GCN Forward Pass:

1. **Feature Embedding:**

$$H^{(0)} = \text{ReLU}(XW_0)$$

(Addresses D1 by explicitly embedding initial features).

2. **Neighborhood Aggregation:**

For each step k, aggregate separately without self-loops:

$$R_1^{(k)} = S_1 H^{(k-1)}$$
$$R_2^{(k)} = S_2 H^{(k-1)}$$

(Addresses D1 and D2: separated ego/neighbors and higher-order hops).

3. **Combination:**

$$H^{\text{final}} = \text{CONCAT}\left(H^{(0)}, R_1^{(1)}, R_2^{(1)}, \ldots, R_1^{(K)}, R_2^{(K)}\right)$$

(Addresses D3).

References: Zhu, Yan, Zhao, Heimann, Akoglu & Koutra, "Beyond Homophily in Graph Neural Networks", NeurIPS 2020

Remedies for Heterophily

Both make deep models fail, through opposite mechanisms: features mix too much, or the information that must be mixed cannot fit.



네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Where each remedy intervenes 31

$$\hat{Y} = f_\theta(A, X)$$

A standard GNN applies a fixed **low-pass, local** operator to the given A. Each remedy relaxes one of these three words: **low-pass**, **local**, or **given**.

Remedy	Relaxes	Core idea
1. Beyond low-frequency	“low-pass”	learnable high-pass / signed filters, per-node channel mixing
2. Beyond local	“local”	2-hop and higher, latent-space and path-based neighbors
3. Advanced architectures	“fixed operator”	expressive spectral bases, similarity-guided and selective messages
4. Structure learning	“given A”	rewire or learn edges that are predictive of labels
5. Graph transformers	“local” + “given A”	global attention = a learned dense graph

References:
- Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크 과학 연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

More ways to reach further

64

High-order, latent-space and path-based neighbors

High-order mixing

Aggregate 1-, 2-, …, k-hop neighbors in separate channels; 2-hop sets are homophily-dominant.

MixHop · H2GCN · UGCN · TDGNN

Latent space

Embed nodes, then treat nodes close in the embedding space as neighbors, however far apart in the graph.

Geom-GCN · NL-GNN · GPNN · HOG-GCN · DC-GNN

Path-based

Sample paths (e.g., BFS for homophily, DFS for heterophily) and encode their order and distance.

PathNet · RAW-GNN · PathMLP

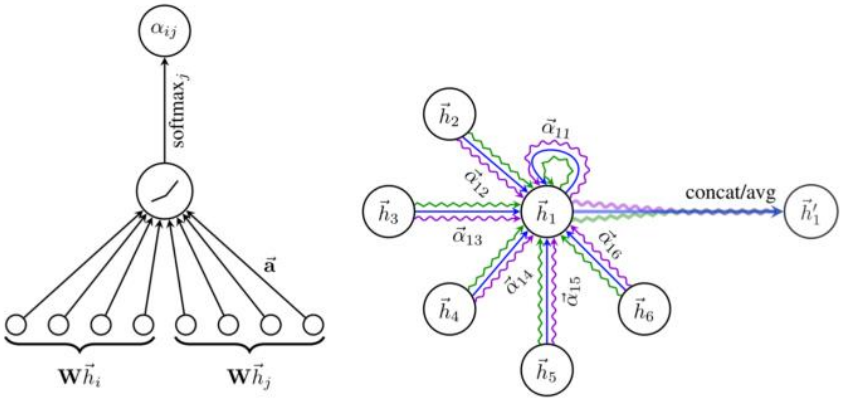
Trade-off: farther reach means larger receptive fields, so cost and over-squashing grow. Latent-space neighbors bridge to Remedy 4 (Graph Structure Learning).

References: Luan, Sitao, et al. "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges." arXiv preprint arXiv:2407.09618 (2024).

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Graph Neural Networks (GNNs) with Attention 14

➢ **Attention means:** Assign an attention coefficient to each neighbor, indicating the importance of that neighbor's features for the feature update of the node.



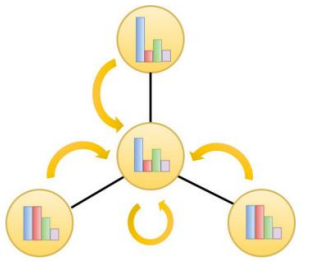
[Figure from Veličković et al. (ICLR 2018)]

References: Monti et al. (CVPR 2017), Hoshen (NIPS 2017), Veličković et al. (ICLR 2018)

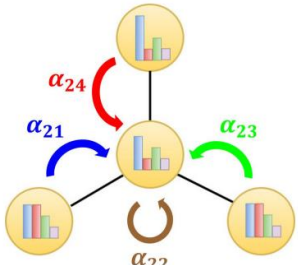
Comparison between GCN and GAT 15

➢ The key difference between GAT and GCN is how the information from the one-hop neighborhood is aggregated.

Vanilla GCN updates the information of neighbor nodes with the same importance	The attention mechanism enables GCN to update nodes with different importance.



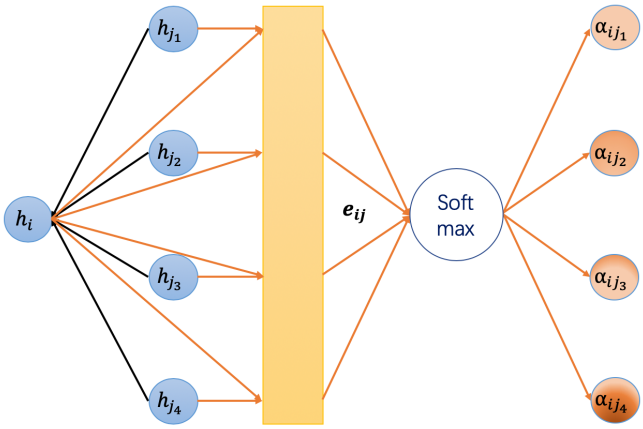
$$H^{(l+1)} = \sigma\left(\sum_{j \in N(i)} H_j^{(l)} W^{(l)}\right)$$



$$H^{(l+1)} = \sigma\left(\sum_{j \in N(i)} \alpha_{ij} H_j^{(l)} W^{(l)}\right)$$

Analysis of Graph Attention: Attention Heads 16

- In Graph Attention Networks (GATs), the concept of multiple attention heads is similar to the idea of multiple filters in Convolutional Neural Networks (CNNs).
- Each attention head can potentially learn to pay attention to different types of neighborhood information.



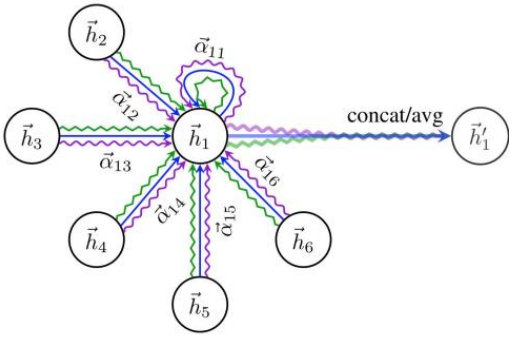
Analysis of Graph Attention: Computation Steps (1) 17

- Input node features: Each node in the graph has a feature vector.

$$\mathbf{h} = \{\vec{h}_1, \vec{h}_2, \ldots, \vec{h}_N\}, \vec{h}_i \in \mathbb{R}^F$$

- Calculate energy (co-efficient) between two nodes.

$$e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j) \quad \text{a: attention function}$$



Analysis of Graph Attention: Computation Steps (2) 18

➢ Attention score (over the neighbors): Normalize over all the neighbors

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T\left[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_j\right]\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T\left[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_k\right]\right)\right)}$$

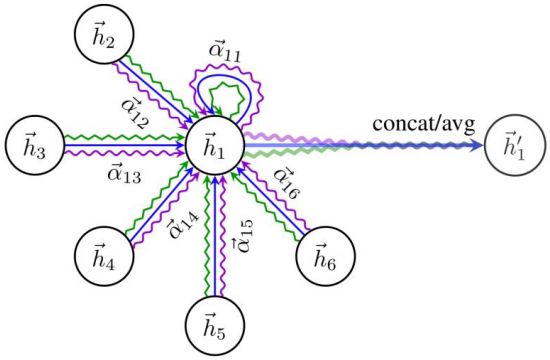
➢ Multi-head attention

- Feature concatenation

$$\vec{h}_i' = \|_{k=1}^{K}\sigma\left(\sum_{j\in\mathcal{N}_i}\alpha_{ij}^k\,\mathbf{W}^k\vec{h}_j\right)$$

- Feature averaging (for the final layer)

$$\vec{h}_i' = \sigma\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{j\in\mathcal{N}_i}\alpha_{ij}^k\,\mathbf{W}^k\vec{h}_j\right)$$



Analysis of Graph Attention 19

➢ Pros:

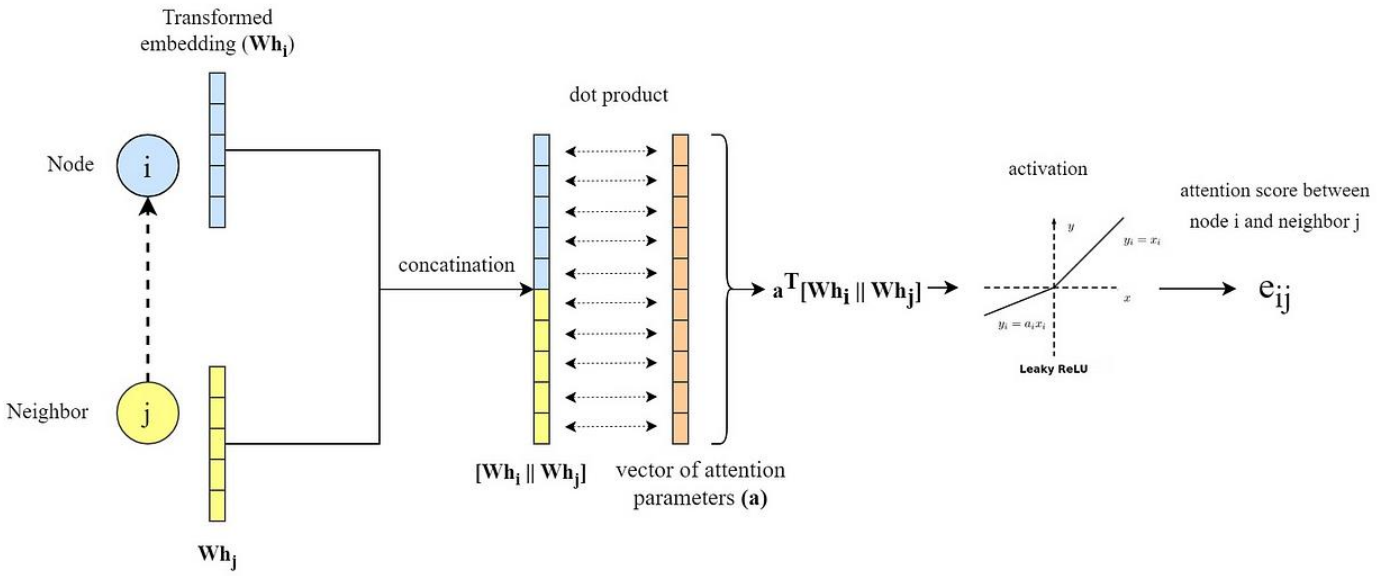
- No need to score intermediate edge-based activation vectors (when using dot product attention).
- Slower than GCNs but faster than GNNs with edge embeddings.

➢ Cons:

- Can be more difficult to optimize.

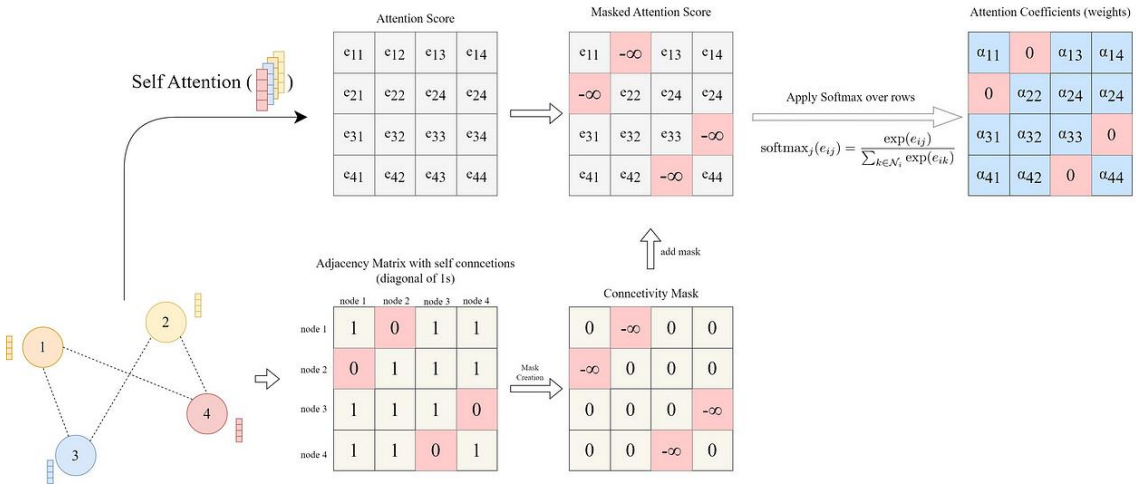
GAT (Graph Attention Networks) Summary 20

➢ The whole operation is illustrated below:



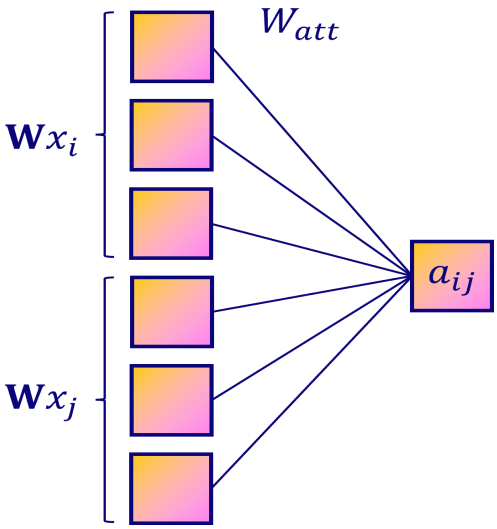
Graph Attention Networks: Example 21

➢ Applying masking mechanism to the masked attention score, then apply Softmax function:



Graph Attention Networks: Steps by steps 22

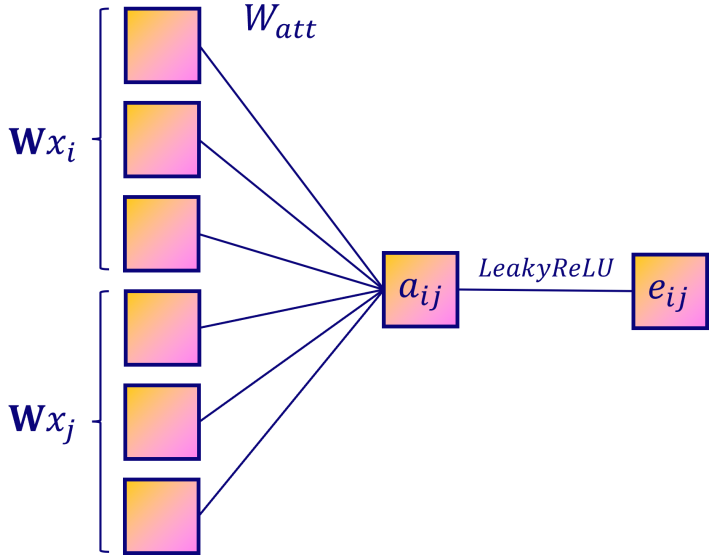
- **Linear transformation:** To calculate the attention coefficient, we need to consider pairs of nodes. An easy way to create these pairs is to concatenate attribute vectors from both nodes.



$$a_{ij} = W_{att}^{t}\left[\mathbf{W}x_i \parallel \mathbf{W}x_j\right]$$

Graph Attention Networks: Steps by steps 23

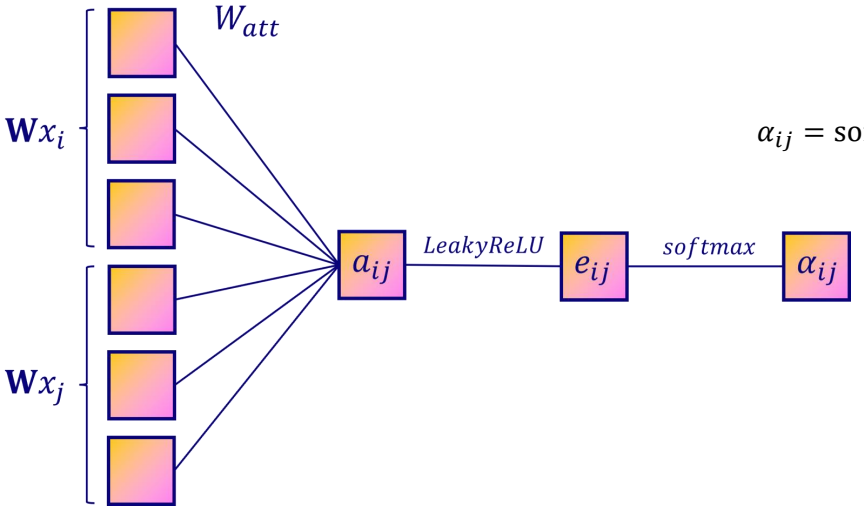
- **Activation function:** Add nonlinearity with an activation function. In this case, the paper's authors chose the LeakyReLU function.



$$e_{ij} = \text{LeakyReLU}\left(a_{ij}\right)$$

Graph Attention Networks: Steps by steps 24

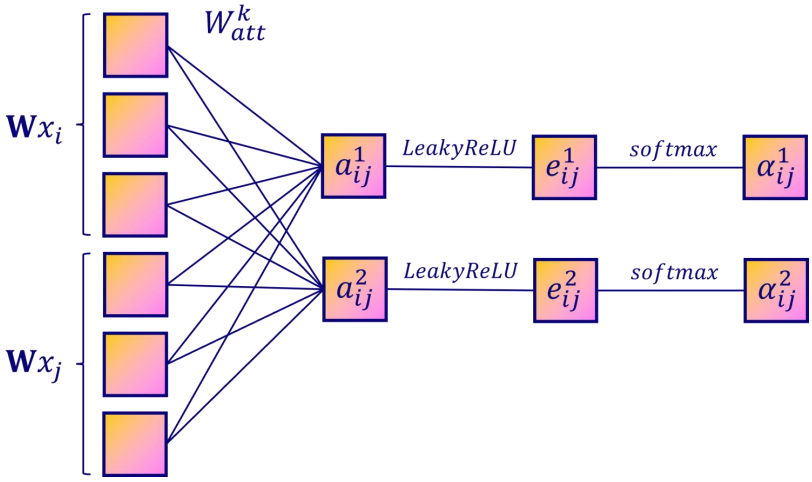
- **Softmax normalization**: The output of our neural network is not normalized, which is a problem since we want to compare these coefficients.
- A common way to do it with neural networks is to use the softmax function.



$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} exp(e_{ik})}$$

Graph Attention Networks: Steps by steps 25

- Multi-head attention: In GATs, multi-head attention consists of replicating the same three steps several times in order to average or concatenate the results.



Average:

$$h_i = \frac{1}{n}\sum_{k=1}^{n} h_i^k$$

Concatenation:

$$h_i = \Big\|_{k=1}^{n} h_i^k$$

Deconstructing the GAT Equation 27

- **Splitting the Weight Vector**: We can split the attention vector $\mathbf{a} \in \mathbb{R}^{2d'}$ into two equal halves corresponding to the query and the key: $\mathbf{a} = [\mathbf{a}_1 \| \mathbf{a}_2]$, where $\mathbf{a}_1, \mathbf{a}_2 \in \mathbb{R}^{d'}$.
- **Expanding the Expression:**

  By substituting the split vector into the pre-activation term, we get:

$$\mathbf{a}^\top \left[\mathbf{W}h_i \| \mathbf{W}h_j\right] = \mathbf{a}_1^\top \mathbf{W}h_i + \mathbf{a}_2^\top \mathbf{W}h_j$$

- **The Implication:**

  The operation inside the LeakyReLU function is simply the sum of two strictly independent terms:

  - $\mathbf{a}_1^\top \mathbf{W}h_i$ relies only on the query node i.
  - $\mathbf{a}_2^\top \mathbf{W}h_j$ relies only on the key node j.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

The “Static Attention” Bottleneck 28

- **Reformulating the Score:**

  Let $C(i) = \mathbf{a}_1^\top \mathbf{W}h_i$ (a constant for a fixed query i) and $K(j) = \mathbf{a}_2^\top \mathbf{W}h_j$. The GAT scoring function becomes:

$$e(h_i, h_j) = \text{LeakyReLU}\big(C(i) + K(j)\big)$$

- The Monotonicity Problem:
  - LeakyReLU is a strictly monotonically increasing function.
  - For any fixed query i, $C(i)$ acts merely as a uniform constant shift added to all neighbors.
  - Therefore, the relative ranking of attention scores among neighbors j is entirely dictated by $K(j)$.
- **Conclusion:**

  Because $K(j)$ is unconditioned on i, the ranking of neighbors $\text{argsort}_j \left(e(h_i, h_j)\right)$ is identical for every single node in the graph. GAT computes Static Attention.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

Visualizing the Limitation (GAT) 29

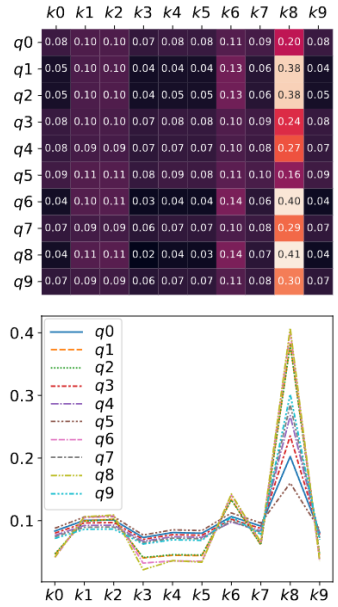


Figure: Attention in standard GAT (Veličković et al. (2018))

- **The Setup**: A complete bipartite graph with query nodes $(q_0 \dots q_9)$ attending to key nodes $(k_0 \dots k_9)$. The task requires dynamic attention.
- **Observation:**
  - Notice the vertical color bands in the heatmap.
  - Look at key $\boldsymbol{k_8}$: It receives the highest attention score from q_0, q_1, q_2, and so on.
- **Takeaway**: Standard GAT fails to map specific queries to specific keys. Every query node behaves identically, confirming the mathematical proof that the ranking is global and unconditioned on the query.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

The Theoretical Fix 30

- **The Goal:** To achieve Dynamic Attention, the query i must be able to alter the ranking of the keys j. This requires the query and key representations to interact non-linearly.
- **The Structural Flaw in GAT:**

  Applying $\mathbf{a}^{\top}$ before the *non-linearity collapses the query and key into isolated scalar values* too early.
- **The Solution:**

  Swap the order of the non-linear activation (LeakyReLU) and the final linear projection ($\mathbf{a}$).
  - **Step 1:** Apply a linear transformation to the concatenated features.
  - **Step 2:** Apply the non-linear activation to the joint vector, allowing cross-dimensional interaction.
  - **Step 3:** Project the non-linearly activated vector to a scalar score.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

Deriving GATv2 31

- **The New Equation:**

$$e(h_i, h_j) = \mathbf{a}^{\top}\text{LeakyReLU}(\mathbf{W}[h_i \| h_j])$$

- **Updated Operations**:
  1. **Concatenation**: $[h_i \| h_j] \in \mathbb{R}^{2d}$
  2. **Joint Linear Projection**: A single weight matrix $\mathbf{W} \in \mathbb{R}^{d' \times 2d}$ projects the concatenated features.
  3. **Non-linearity**: LeakyReLU is applied to the resulting d'-dimensional vector
  4. **Scalar Projection**: The attention vector $\mathbf{a} \in \mathbb{R}^{d'}$ projects the activated joint representation to a final scalar score.
- **Cost**: This modification requires **zero** additional parameters and negligible computational overhead compared to standard GAT.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

Why GATv2 Computes Dynamic Attention 32

- **The Multi-Layer Perceptron (MLP) Connection:**

  The GATv2 scoring function mathematically mirrors a standard feed-forward neural network with a single hidden layer:

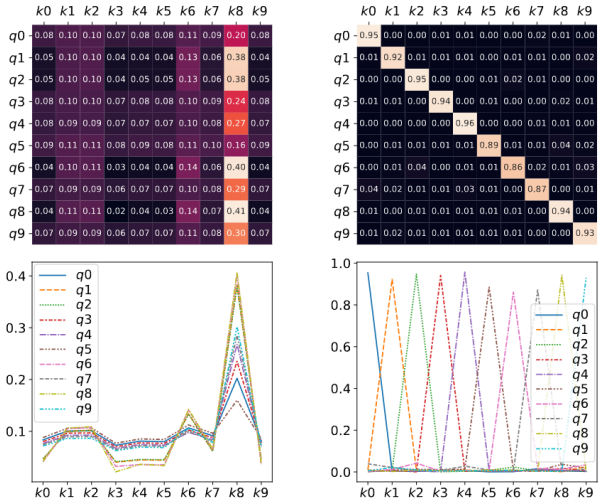
  - **Input**: $[h_i \| h_j]$
  - **Hidden Layer**: $\mathbf{W}$ (weights) followed by LeakyReLU (activation)
  - **Output Layer**: $\mathbf{a}^{\top}$ (weights mapping to a 1D scalar)
- **Universal Approximation Theorem**:

  Because GATv2's scoring mechanism is an MLP with a non-linear activation, it acts as a universal approximator. It can approximate any continuous function over a compact domain.
- **Conclusion**:GATv2 possesses the absolute capacity to learn complex, non-linear interactions where $f(q_i, k_{\phi(i)}) > f(q_i, k_j)$, satisfying the formal requirements for ***Dynamic Attention***.

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

Visualizing the Improvement (GATv2) 33

- The GATv2 model performs better than the first version GAT, because it uses a dynamic graph attention variant that has a universal approximator attention function, it is more expressive than the other model, based on a static attention.



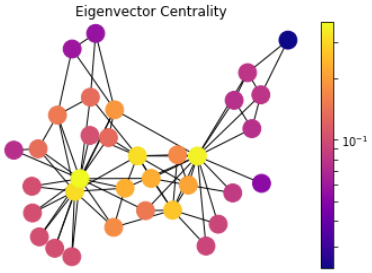
(a) Attention in standard GAT (b) Attention in GATv2

References: Shaked Brody, Uri Alon, Eran Yahav: How Attentive are Graph Attention Networks? ICLR 2022

Graph Attention Networks: Idea 9

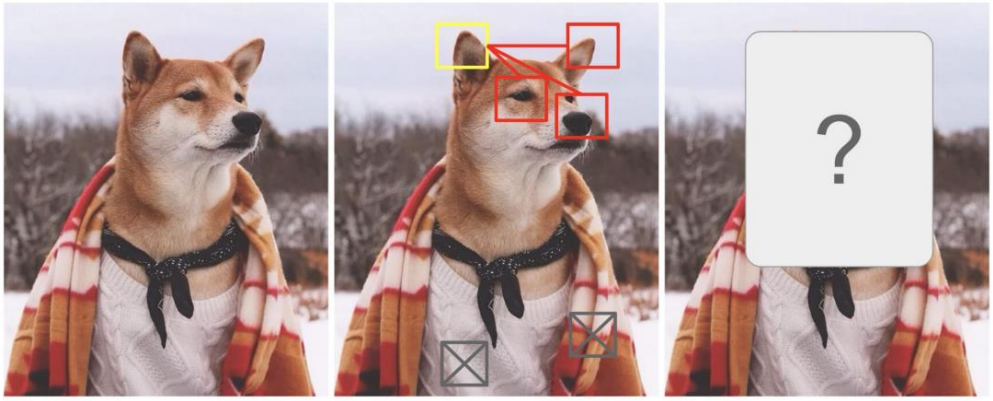
Not all node's neighbors are equally important

- Attention is a mechanism that allows a network to focus on certain parts of the input when processing it.
- The attention focuses on the important parts of the input data and fades out the rest.
  - **Idea**: The neural network should devote more computing power on that small but important part of the data.
  - Which part of the data is more important depends on the context and is learned through training.



Attention mechanism in Image processing 10

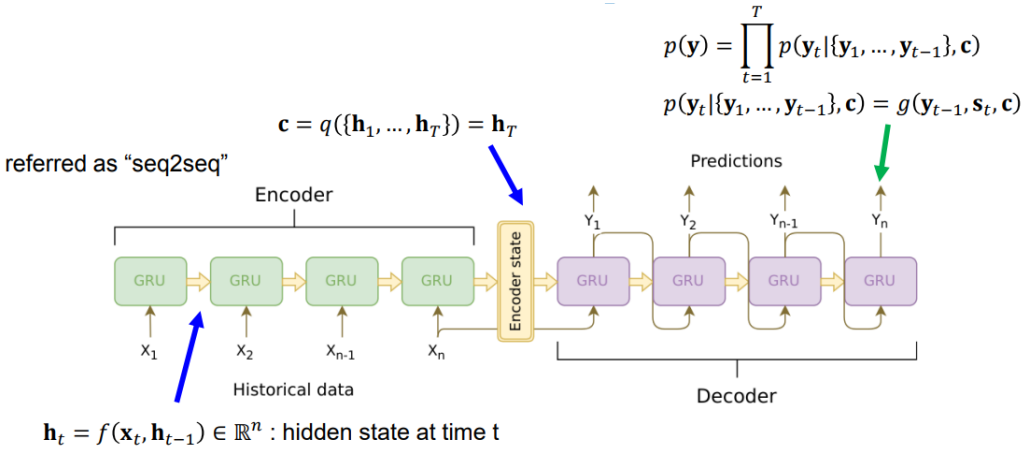
- We deduce something by paying attention to something that is relatively more important.



References: Weng, L. (2018, June 24). Attention? Attention! Github.Io; Lil'Log. https://lilianweng.github.io/posts/2018-06-24-attention/

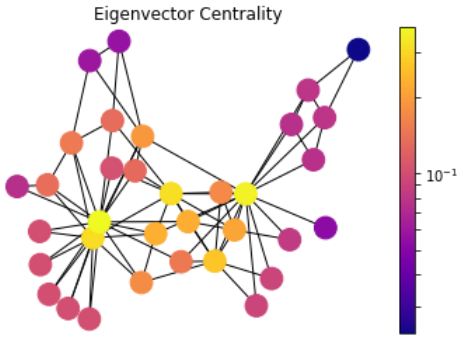
Attention mechanism in NLP (Seq2Seq) 11

- RNN encoder-decoder for neural machine translation:
  - In capability of remembering long sentences: Often it has forgotten the first part once it completes processing the whole input. The attention mechanism was born to resolve this problem.



Why Attention in GNN? 12

- GNN compute node representations from representations of neighbours.
- Nodes can have largely different neighbourhood sizes.
- Not all neighbours have relevant information for a certain node.
- Attention mechanism allow to adaptively weight the contribution of each neighbour when updating a node.



Analyzing Attention Forms (GO vs. DP) 38

1. **Graph Attention Original (GO):**

$$e_{ij}^{GO} = \text{LeakyReLU}\left(\mathbf{a}^T\left[\mathbf{W}h_i || \mathbf{W}h_j\right]\right)$$

***Strength**: Highly expressive for capturing label-agreement (nodes with the same label).*

2. **Dot Product (DP):**

$$e_{ij}^{DP} = (\mathbf{W}h_i)^T(\mathbf{W}h_j)$$

***Strength**: Mathematically aligned with graph autoencoders; highly predictive of edge presence.*

References: Dongkwan Kim, Alice Oh: How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. ICLR 2021

Advanced SuperGAT Variants (SD and MX) 39

To resolve the trade-off between label agreement and link prediction, two advanced variants was introduced:

3. **Scaled Dot-Product (SD):**

$$e_{ij}^{SD} = \frac{(\mathbf{W}h_i)^T(\mathbf{W}h_j)}{\sqrt{d}}$$

Theory: Prevents the dot product from exploding in high-dimensional feature spaces (d), ensuring stable gradients for the $\sigma(e_{ij})$ function during self-supervision.

4. **Mixed Attention (MX):**

Theory: Decouples the mechanisms. Uses GO to compute α_{ij} for node feature aggregation (maximizing label agreement) but applies a DP-based projection strictly for the self-supervised edge prediction branch.

References: Dongkwan Kim, Alice Oh: How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. ICLR 2021

The Role of Graph Characteristics 40

The effectiveness of the four SuperGAT variants depends fundamentally on two macro-level graph properties:

- **Homophily (H):** The ratio of edges connecting nodes of the same label.

$$H = \frac{|\{(i,j) \in \mathcal{E} : y_i = y_j\}|}{|\mathcal{E}|}$$

- **Average Degree ($\bar{D}$):** The density of the graph.

$$\bar{D} = \frac{2|\mathcal{E}|}{|\mathcal{V}|}$$

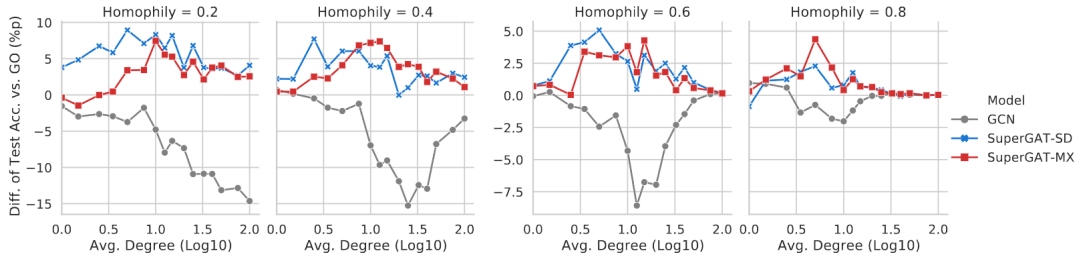


Figure: Mean test accuracy gains (of 5 runs) against GAT_{GO} on synthetic datasets, varying homophily and average degree of the input graph.

References: Dongkwan Kim, Alice Oh: How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. ICLR 2021

The SuperGAT Design Recipe 41

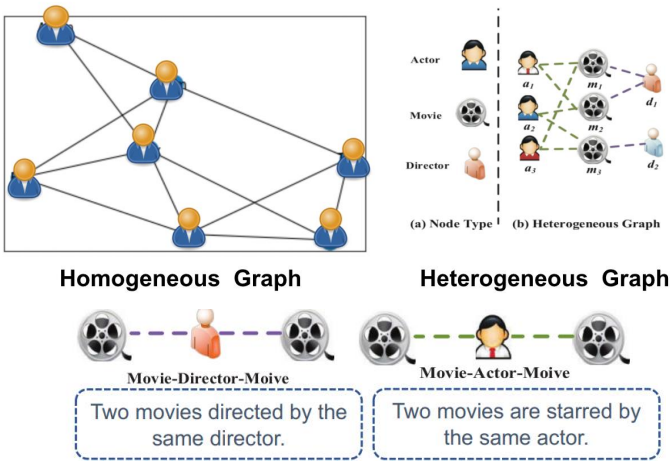
The theoretical analysis provides a deterministic design recipe for practitioners to choose the right attention mechanism based on graph statistics:

- **High Homophily + High Degree**: ***DP*** *or* ***SD*** *thrive*. Structural proximity heavily correlates with label similarity, and dense edges provide rich, stable self-supervision for the dot product.
- **Low Homophily (Heterophily):** ***GO*** *is required.* The dot-product strictly enforces similarity, which degrades representations when neighbors have different labels.
- **Low Degree (Sparse Graphs): MX** or **GO** are preferred. Sparse graphs lack enough edges to properly train DP representations without collapsing, requiring the robustness of GO.

References: Dongkwan Kim, Alice Oh: How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. ICLR 2021

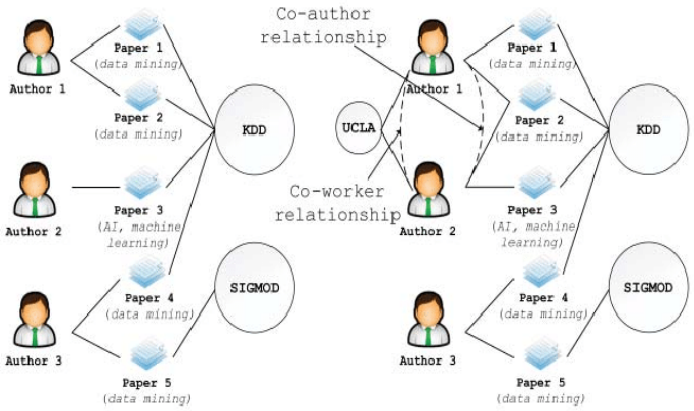
Attention in Heterogeneous Graphs 70

- Multiple types of nodes or links
- Rich semantic information
  - Meta-path: a relation sequence connecting objects
    (e.g., Movie-Actor-Movie)



Meta-paths: High level representation of relationship 71

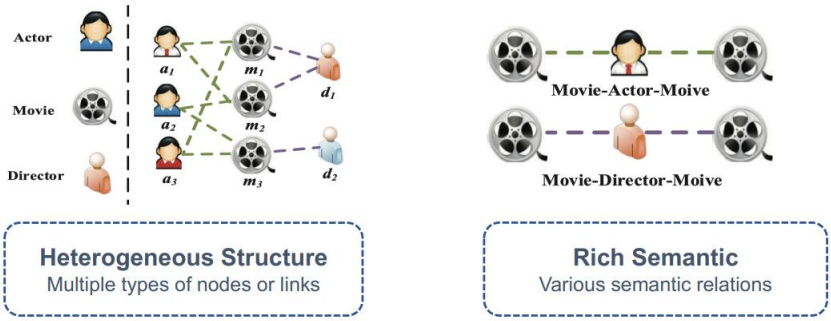
- DBLP Bibliographic network
  - Node (type)
    - KDD (Venue)
    - Author 1
  - Link (Type)
    - Write (Author - Paper)
    - Publish (Paper – Venue)



Why Attention in Heterogeneours Graphs 72

➢ Existing GNNs focus on homogeneous graphs

- Cannot handle multiple types of nodes and edges.
- Cannot capture rich semantic information.



Challenge: How to handle the heterogeneity of graph?

Heterogeneous Graph Attention Networks (HAN) 73

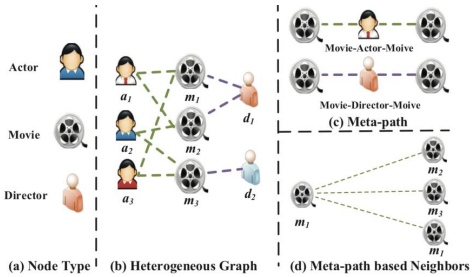
Recap: Heterogeneous Graphs & Meta-Paths

➢ **Heterogeneous Graph Definition**: A graph $G = (V, E)$ containing diverse nodes and relations, governed by a node type mapping function $\phi: V \rightarrow A$ and an edge type mapping function $\psi: E \rightarrow R$, where $|A| + |R| > 2$.

➢ **The Heterogeneity Challenge**: Different types of nodes possess varying traits, meaning their features exist in fundamentally disjoint feature spaces.

➢ **Meta-Paths for Semantic Extraction:** A meta-path Φ is a composite relation sequence connecting object types (e.g., Movie → Actor → Movie). Each meta-path extracts distinct semantic relations between nodes.

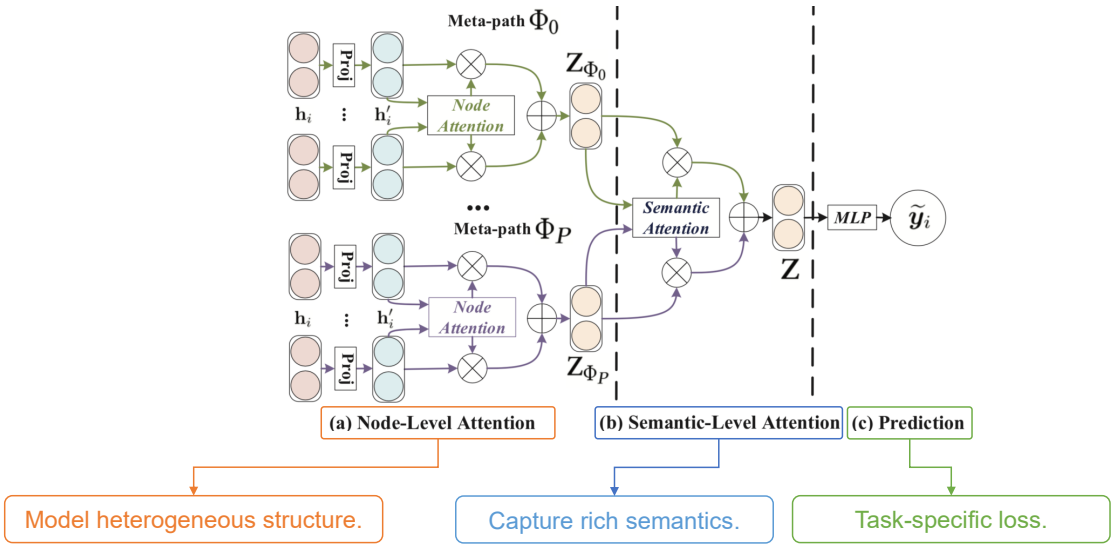
➢ **Requirement for Attention:** Standard aggregation treats all neighbors **equally**. In heterogeneous settings, attention must dynamically learn weights at two distinct granularities: the topological neighborhood (nodes) and the relational context (meta-paths).



References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 74

Heterogeneous Graph Attention Networks (HAN)



References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 75

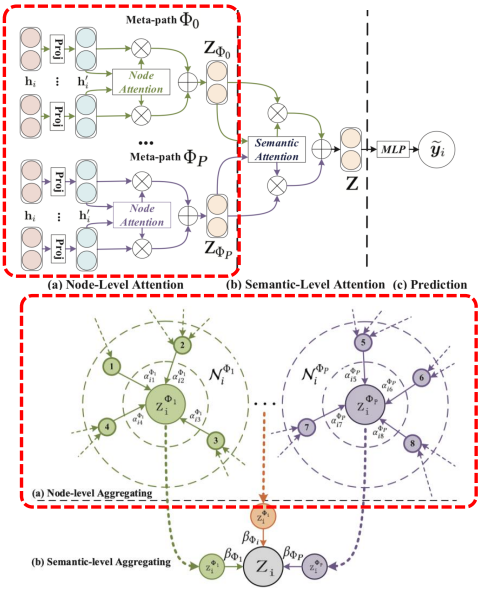
The Hierarchical Attention Framework

- **Core Architecture: The network processes heterogeneous data through a sequential, bottom-up hierarchical structure.**
  - **Step 1: Feature Projection:** Aligns disparate feature spaces into a unified embedding dimension.
  - **Step 2: Node-Level Attention:** Evaluates and aggregates the structural importance of meta-path based neighbors for a specific semantic context.
  - **Step 3: Semantic-Level Attention:** Evaluates and fuses the global importance of different meta-paths to generate the final, optimal node representations.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 76

➢ Node-level Attention and Aggregating



➢ Type-specific information

$$\mathbf{h}_i' = \mathbf{M}_{\phi_i} \cdot \mathbf{h}_i$$

Type-specific transformation matrix

➢ Importance of Neighbors

$$e_{ij}^{\Phi} = \mathrm{att}_{node}(\mathbf{h}_i', \mathbf{h}_j'; \Phi)$$

$$\alpha_{ij}^{\Phi} = \mathrm{softmax}_j(e_{ij}^{\Phi}) = \frac{\exp\left(\sigma\left(\mathbf{a}_{\Phi}^{\mathrm{T}} \cdot [\mathbf{h}_i' \,\|\, \mathbf{h}_j']\right)\right)}{\sum_{k \in \mathcal{N}_i^{\Phi}} \exp\left(\sigma\left(\mathbf{a}_{\Phi}^{\mathrm{T}} \cdot [\mathbf{h}_i' \,\|\, \mathbf{h}_k']\right)\right)}$$

Node-level attention vector

➢ Node-level Aggregating

$$\mathbf{z}_i^{\Phi} = \Big\|_{k=1}^{K} \sigma\Big(\sum_{j \in \mathcal{N}_i^{\Phi}} \alpha_{ij}^{\Phi} \cdot \mathbf{h}_j'\Big).$$

Node weight

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 77

Feature Space Alignment ***(Type-Specific Projection)***

- **Objective**: Resolve the dimensional and spatial mismatch of node features across different types before computing attention.
- **Transformation**: For a given node i of type ϕ_i, a type-specific linear projection matrix maps its raw features into a shared latent space.
- **Equation**:

$$h_i' = M_{\phi_i} \cdot h_i$$

where:

- $h_i \in \mathbb{R}^{d_{\phi_i}}$: Original feature vector of node i.
- $h_i' \in \mathbb{R}^{d'}$: Projected feature vector in the unified space.
- $M_{\phi_i} \in \mathbb{R}^{d' \times d_{\phi_i}}$: Trainable transformation matrix specific to node type ϕ_i.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 78

Feature Space Alignment *(Neighbor Importance Formulation)*

- **Objective**: For a specific meta-path Φ, calculate the relevance of a neighboring node j to a target node i.
- **Mechanism:** A self-attention mechanism operates on the projected features of node pairs connected by Φ.
- **Equation**:

$$e_{ij}^{\Phi} = \text{LeakyReLU}\left(\left(\mathbf{a}^{\Phi}\right)^{T}\left[h_i' || h_j'\right]\right)$$

where:

- e_{ij}^{Φ}: Raw, unnormalized attention score (importance) of node j to node i.
- $\mathbf{a}^{\Phi} \in \mathbb{R}^{2d'}$: Parameterized attention vector specific to meta-path Φ.
- $||$: Concatenation operation.
- $\left[h_i' || h_j'\right]$: The combined representation of the node pair in the shared space.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Heterogeneous Graph Attention Networks (HAN) 79

Feature Space Alignment *(Normalization & Aggregation)*

- **Objective**: Normalize the raw attention scores across the meta-path neighborhood and aggregate the features to form a semantic-specific embedding.
- **Normalization Equation:**

$$\alpha_{ij}^{\Phi} = \frac{\exp\left(e_{ij}^{\Phi}\right)}{\sum_{k \in N_i^{\Phi}} \exp\left(e_{ik}^{\Phi}\right)}$$

- **Aggregation Equation:**

$$z_i^{\Phi} = \sigma\left(\sum_{j \in N_i^{\Phi}} \alpha_{ij}^{\Phi} \, h_j'\right)$$

where:

- N_i^{Φ}: The neighborhood set of node i strictly defined by meta-path Φ.
- α_{ij}^{Φ}: Normalized attention coefficient.
- $\sigma(\cdot)$: Non-linear activation function.
- z_i^{Φ}: The resulting embedding for node i, capturing only the semantics of meta-path Φ.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA

Heterogeneous Graph Attention Networks (HAN) 80

Multi-Head Extension for Node-Level Attention

- **Objective**: Stabilize the learning process and capture diverse neighborhood information by executing multiple independent attention mechanisms in parallel.
- **Equation:**

$$z_i^{\Phi} = \|_{k=1}^{K} \sigma\left(\sum_{j \in N_i^{\Phi}} \left(\alpha_{ij}^{\Phi}\right)^k h_j'\right)$$

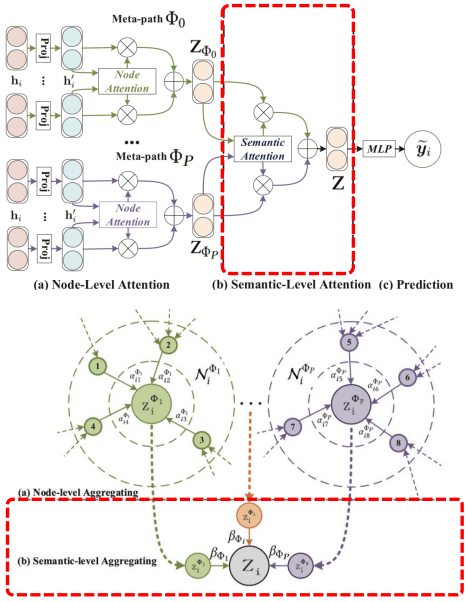
where:

- K: Total number of independent attention heads.
- $\left(\alpha_{ij}^{\Phi}\right)^k$: The normalized attention coefficient computed by the k-th attention head.
- $\|_{k=1}^{K}$: Concatenation of the outputs from all K heads.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 81

➢ Semantic-level Attention and Aggregating



➢ Semantic-Level Attention

$$\left(\beta_{\Phi_0}, \beta_{\Phi_1}, \ldots, \beta_{\Phi_P}\right) = \mathrm{att}_{sem}\left(\mathbf{Z}_{\Phi_0}, \mathbf{Z}_{\Phi_1}, \ldots, \mathbf{Z}_{\Phi_P}\right)$$

➢ Importance of Meta-path

Semantic-level attention vector

$$w_{\Phi_i} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbf{q}^{\mathrm{T}} \cdot \tanh\left(\mathbf{W} \cdot \mathbf{z}_i^{\Phi} + \mathbf{b}\right)$$

$$\beta_{\Phi_i} = \frac{\exp\left(w_{\Phi_i}\right)}{\sum_{i=1}^{P} \exp\left(w_{\Phi_i}\right)}$$

➢ Semantic-Level Aggregating

$$\mathbf{Z} = \sum_{i=1}^{P} \beta_{\Phi_i} \cdot \mathbf{Z}_{\Phi_i}$$

Semantic weight

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 82

Semantic-Level Attention *(Meta-Path Importance Evaluation)*

- **Objective**: Since different meta-paths contribute variably to specific downstream tasks, the network must learn a global weight for each meta-path.
- **Equation:**

$$w_{\Phi_p} = \frac{1}{|V|}\sum_{i \in V} \mathbf{q}^T \cdot \tanh\left(\mathbf{W} \cdot z_i^{\Phi_p} + \mathbf{b}\right)$$

where:

- w_{Φ_p}: Unnormalized global importance score for meta-path Φ_p.
- $\mathbf{W}, \mathbf{b}$: Trainable weight matrix and bias vector.
- $\mathbf{q}$: A shared semantic-level attention vector.
- $|V|$: Total number of target nodes, used to average the semantic contribution globally.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 83

Semantic-Level Attention *(Final Optimal Fusion)*

- **Objective**: Normalize the semantic-level importance scores and linearly combine the semantic-specific node embeddings into one comprehensive vector.
- **Normalization Equation:**

$$\beta_{\Phi_p} = \frac{\exp\left(w_{\Phi_p}\right)}{\sum_{p=1}^{P} \exp\left(w_{\Phi_p}\right)}$$

- **Fusion Equation:**

$$\mathbf{Z} = \sum_{p=1}^{P} \beta_{\Phi_p} \mathbf{Z}^{\Phi_p}$$

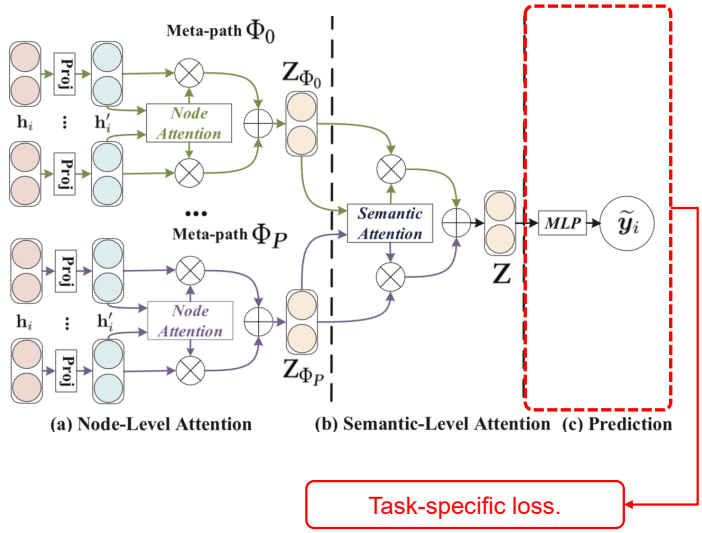
where:

- β_{Φ_p}: The learned, normalized weight of meta-path Φ_p, interpreted as its contribution to the task.
- $\mathbf{Z}^{\Phi_p}$: The matrix of node embeddings generated under meta-path Φ_p.
- $\mathbf{Z}$: The final, unified matrix of node representations that perfectly balances complex structural constraints and rich, multi-faceted semantics.

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Heterogeneous Graph Attention Networks (HAN) 84

➢ Prediction



➢ Semi-supervised Loss

Parameter of classifier

$$L = -\sum_{l \in \mathcal{Y}_L} \mathbf{Y}^l \ln\left(\mathbf{C} \cdot \mathbf{Z}^l\right)$$

Labeled data

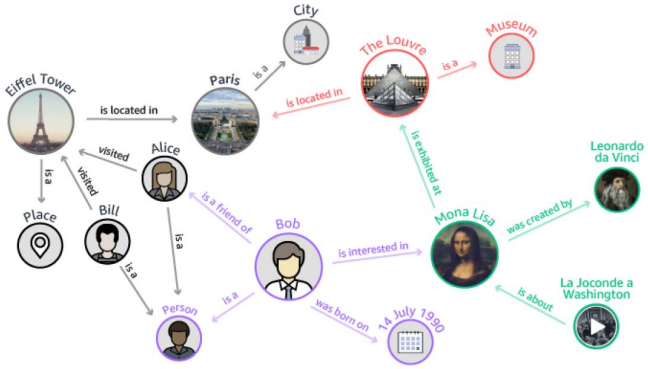
Optimize for the specific task
(e.g. node classification)

References: Wang, Xiao, et al. "Heterogeneous graph attention network." The world wide web conference. 2019.

Attention in Heterogeneous Graphs 69

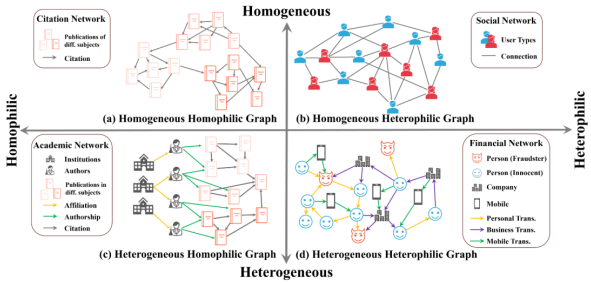
➢ Graph in real world:

- Many node types, link types
- Non- ordered
- Complex connections



Before we continue to *Homophily Metrics on Heterogeneous Graphs* 18

Feature	Heterogeneity	Heterophily
What it describes	Diversity of node and edge types	Similarity of labels/features between connected nodes
Opposite concept	Homogeneity (single node/edge type)	Homophily (connected nodes share similar labels/features)
Impact on GNNs	Standard GNNs fail because they assume a single uniform message-passing space.	Standard GNNs fail because neighborhood aggregation mixes dissimilar/conflicting labels.
Example	Author-Paper-Venue academic graph	Fraudster-Victim transaction graph



References: Lin, Junhong, et al. "When heterophily meets heterogeneity: Challenges and a new large-scale graph benchmark." Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2. 2025.

Before we continue to *Homophily Metrics on Heterogeneous Graphs* 19

A Heterogeneous Graph (or HIN) is defined as a tuple:

$$G = (V, E, \mathcal{T}_v, \mathcal{T}_e, \phi, \psi)$$

Where:

- V is the set of nodes and E is the set of edges.
- $\mathcal{T}_v$ is the set of predefined node types.
- $\mathcal{T}_e$ is the set of predefined edge (relation) types.
- $\phi: V \to \mathcal{T}_v$ is a node-type mapping function that assigns each node to a specific type.
- $\psi: E \to \mathcal{T}_e$ is an edge-type mapping function that assigns each edge to a specific relation type.

The Mathematical Constraint for Heterogeneity:

A graph is mathematically heterogeneous if and only if:

$$|\mathcal{T}_v| + |\mathcal{T}_e| > 2$$

(If $|\mathcal{T}_v| = 1$ and $|\mathcal{T}_e| = 1$, the graph is **strictly homogeneous**).

References: https://graph-neural-networks.github.io/static/file/chapter16.pdf

네트워크과학연구실 NETWORK SCIENCE LAB 가톨릭대학교 THE CATHOLIC UNIVERSITY OF KOREA